CocoaMojo

Writing native macOS applications in Mojo, on a compiler that reads the SDK.

CocoaMojo is the Cocoa layer of MojoCocoa, an unofficial fork of Mojo frozen at one commit. It is two things at once: a small standard-library package, `std.objc`, and a compiler hook, `cocoakb`, that answers questions about macOS *while your program is being compiled*.

The result is that a Cocoa binding states a name and the compiler supplies everything else. A selector typo is a compile error. A struct that has drifted from the SDK fails to build. An argument in the wrong register file is a compile error rather than a corrupted call.

```
comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()

var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

Getting it

CocoaMojo installs from a signed, notarized disk image published at CocoaMojoInstaller — one for Apple Silicon, one for the 2019 Intel Mac Pro. Open it, double-click *Install Roast*, press Install.

You get the compiler, the standard library, the language server, the debugger, the examples, and **Roast**, a Mac IDE for Mojo written in Mojo. The installer also builds the SDK database from the machine it lands on, which takes about twelve seconds and is the reason none of this is downloaded prebuilt.

Chapter 1 takes it from there. Building the fork from source is Appendix A, and you do not need it to write anything.

These documents

Document	What it is
Programmer's Guide	Read this first. Ten chapters and an appendix: installing the toolchain, the frozen dialect, Mojo's own object model, Cocoa from the first message to a walked-through Life implementation, and the fork's own IDE read as a program.
Reference Manual	Look things up here. The frozen language dialect, every `std.objc` entry point, every `cocoakb` query, and every diagnostic.
GPU programming	Writing Mojo functions that run on the Apple GPU: the execution model, threads and memory, and how to build, run and verify them.
The examples	A walkthrough of all eighteen example projects, and an honest statement of what each one teaches — including the ones that teach nothing.

A warning about versions

Mojo has been changing quickly, and most writing about it — including parts of its own published documentation — describes a language this compiler does not accept. `@parameter if` and `@parameter for` have given way to `comptime if` and `comptime for`, and `alias` is deprecated in favour of `comptime`.

This fork has also stopped following upstream's declaration model and named itself **cocoa-mojo**. Three keywords carry revived, narrower meanings: **fn** is now the foreign-callable function — thin, non-raising, C ABI, exactly the Objective-C `IMP` contract — **let** is an immutable, scope-bound binding, and **class**, which upstream reserved for a Python-style class that never came, declares a real Objective-C class. None of the three means what it meant in older Mojo, and none means what upstream would mean by it.

Everything in these documents was checked against the source tree of this frozen compiler, not against published documentation. Where the two disagree, the compiler is right and the documentation is stale. The Language reference lists the differences explicitly, because they are the single largest source of code that looks correct and will not build.

Which of those differences are *this fork's decisions* — and which are upstream's own churn, inherited by freezing — is set out in Deviations from Modular's compiler, with the argument for each and what it costs. It is the place to start if you are weighing whether to use this rather than learn from it.

The state of the port

The Cocoa layer described here works and is verified: nine of nine verification spikes pass, and the example applications build and run.

The GPU layer is a working vertical slice: kernels compile, run and agree with a CPU reference, and 96 of 119 in-scope GPU tests pass, with the failures triaged and named. It has its own section, and nothing in the Cocoa chapters depends on it.

Conventions

Code shown in these documents is either taken directly from the fork's standard library and verification spikes, or is reduced from them. Where a listing is abbreviated, an ellipsis comment says so.

The database is a build input, not a runtime dependency. A compiled CocoaMojo program has no dependency on `cocoa.sqlite`. By the time code is generated, every query has already collapsed to a constant.

The examples, and what each one is for

The distribution ships eighteen example projects, about 12,100 lines of Mojo. They are not a gallery. Most of them were written to answer a question, and a few were carried in from elsewhere to prove that they still run. This section walks through every one and says what it teaches.

It also says when an example teaches **nothing**. Several here are smoke tests or compatibility checks, and pretending otherwise would waste your time. Each section ends with a plain verdict, and some of those verdicts are "none".

The whole set

Example	Size	The lesson
hello	7 lines	**None.** It proves the toolchain works.
window	134 lines	The fork's whole thesis, at the smallest size it can be stated
operators	544 lines	**None specific to this fork** — upstream's example, running unmodified
process	128 lines	**None specific to this fork** — same, for child processes
life-python	231 lines	**No code lesson** — the source is upstream's. The Python workflow is the lesson
vector-add	80 lines	The canonical first kernel: one thread, one element
grayscale	115 lines	Two-dimensional indexing. Little else beyond `vector-add`
tiled-matmul	345 lines	Shared memory, and the *second* barrier everyone omits
life	643 lines	A real application, and a program right to use no GPU at all
mandelbrot	488 lines	One dispatch per frame, computing *and* colouring, with no shader
fluid	911 lines	Thirty-five dependent dispatches per step, where launch cost dominates
othello	1,024 lines	The honest answer about game trees: one search suits a GPU, one does not
chip	2,159 lines	A Mojo `fn` serving as a C function pointer on a real-time thread
abcplayer	3,344 lines	The largest example: a real parser, and exact timing without floats
bifurcation	246 lines	What Python is genuinely better at, measured rather than assumed
fern	289 lines	Wide work with no deadline — correctly on the CPU
ferns	659 lines	A deadline, but work too narrow to move
fernwind	778 lines	Wide *and* out of time — the crossing, measured at 104×

The last three have a chapter of their own, Three ferns, because together they make an argument no single example can.

The chapters

Chapter	What it covers
1. The small ones	`hello`, `window`, `operators`, `process`, `life-python`
2. The GPU primers	`vector-add`, `grayscale`, `tiled-matmul`
3. The applications	`life`, `mandelbrot`, `fluid`, `othello`
4. Sound, and time	`chip`, `abcplayer`
5. Python, used well	`bifurcation`, and what it measures about the boundary

Two questions the set keeps asking

Read end to end, the examples circle the same two decisions.

Where does the work run? The GPU chapters and the ferns settle this with a rule worth carrying into the rest of the section:

> Move work to the GPU when it is **wide** and you are **out of time**. Width without a deadline does not need one. A deadline with narrow work cannot use one.

`life` declines a GPU, `mandelbrot` needs exactly one dispatch, `fluid` needs thirty-five and discovers that launch cost is the whole problem, and `othello` splits its own players down the middle. Four different answers, all correct.

Who calls whom? A Cocoa application is not a program that runs; it is a program that is *called*. `window` shows the smallest version — a button that sends a message to a `class` you declared. `life` shows three such classes. `chip` shows the case where the caller is not Objective-C at all, but CoreAudio on a real-time thread, and the thing it wants is a bare C function pointer.

How to run one

Every example is a folder with a `main.mojo` in it. In Roast, **File ▸ Open Folder…**, then ⌘R. There is no project file and nothing to generate.

The GPU examples guard themselves with `has_accelerator` and will say so rather than fail if there is no Metal device. `life-python` and `bifurcation` need a Python environment; each README explains what and why, and Roast builds one on demand — only for projects that actually use Python.

```
And the largest example is not in this folder: File ▸ Open IDE Source
opens Roast's own source — the editor you are reading this in, written
in the language it edits. It has a chapter too:
Guide, chapter 10.
```

1. The small ones

Five short projects. One of them states the fork's entire argument; two of them state nothing at all, on purpose; and two are here to prove a negative.

`hello`

Seven lines, one file.

```
# The smallest thing that runs. ⌘R.
def main():
    print("Hello from cocoa-mojo.")
    var total = 0
    for i in range(1, 101):
        total += i
    print("The first hundred integers sum to", total)
```

It prints a greeting and `5050`.

The loop is there for one reason: a program that only prints a constant cannot tell you whether it *ran*. A wrong sum would mean the compiler produced code and the code is broken, which is a different failure from the toolchain not working. Seven lines, two distinguishable failure modes.

The lesson: none. This is a smoke test. If it prints 5050 your compiler, your SDK database and your run button all work, and you can stop thinking about them. That is the whole of its ambition, and it is worth having for exactly the five seconds it takes.

`window`

134 lines, and the most important small file in the distribution. A window, a label, a button; click the button and the label counts. In any other language-to-Cocoa bridge this is a chapter about generated headers. Here it is one `class` declaration:

```
class ExampleActions:
    """The button's target.

    `buttonClicked_` becomes the selector `buttonClicked:` -- an underscore is
    a colon -- and the compiler derives its `v@:@` encoding, because this is a
    selector we invented rather than one the SDK declares. There is no `_cmd`
    argument to write and no IMP to register: `ExampleActions()` builds the
    class in the runtime and hands back an instance.
    """

    def buttonClicked_(self, sender: ObjCObject):
        clicks()[] += 1
```

Three things are absent, and their absence is the point. There is no bridging header. There is no `"v@:@"` typed out anywhere — the compiler derived it. And there is no `ObjCClassBuilder` call registering the class by hand, because `ExampleActions()` does that.

The button is wired with the selector named as a string, checked at compile time:

```
let button = msg_send[
    ObjCObject, "NSButton", "buttonWithTitle:target:action:", is_class=True,
](
    NSButton.as_object(),
    nsstring(String("Click me")).ptr(),
    actions.ptr(),
    sel["buttonClicked:"]().ptr(),
)
```

Misspell `buttonWithTitle:target:action:` and the build fails. In Objective-C it would compile and crash at run time; here `cocoakb` is asked during elaboration whether `NSButton` responds to that selector.

Two details in this file are load-bearing for every Cocoa program you write afterwards.

AppKit must be loaded explicitly. The comment says why:

```
# AppKit is not linked into a JIT process; without this NSApplication is
# nil and the app exits having drawn nothing.
if not load_framework["AppKit"]():
    raise Error("could not load AppKit")
```

The failure this prevents is silent — no crash, no message, just a program that exits having done nothing visible.

A callback cannot reach a local. `buttonClicked_` needs the click count and the label, and neither can be a variable in `main`, because Cocoa calls the method with no path back to `main`'s frame. So both live in named globals:

```
comptime clicks = named_global["example.clicks", Int]
comptime label_addr = named_global["example.label", Int]
```

This constraint reappears in every larger example. `life`, `mandelbrot` and `othello` all hand-roll their event loops or park state in globals for exactly this reason, and `othello`'s header states the general form of it: a `DeviceContext` cannot live in a `named_global`, and a Cocoa callback cannot reach a local, so the loop that owns the GPU has to be the loop that drives the app.

The lesson: this is the fork's thesis at the smallest size it can be written. A real Objective-C class, a selector you invented, an encoding nobody typed, and a compile error for a name that does not exist. Read this one even if you skip the rest of the section. The full treatment is Guide, chapter 6.

operators

544 lines across three files: `main.mojo`, `my_complex.mojo`, and `test_my_complex.mojo` beside them. A `Complex` struct wearing the whole operator set — arithmetic, comparison, indexing, conversion — with a `std.testing` suite that exercises it.

It is Modular's own example, from `mojo/examples/operators`, carried into this collection **unmodified**. Not adapted, not ported: the same source.

The lesson: none that belongs to this fork. What you learn here is Mojo's object model, and you would learn it identically from upstream. There is no Cocoa in it and no GPU in it.

Its value is a negative result, and a genuinely useful one: this fork froze the language, revived three keywords with narrower meanings, and replaced `alias` with `comptime` — and ordinary Mojo still compiles and runs here unchanged. `my_complex.mojo` and `test_my_complex.mojo` are byte-identical to upstream's; `main.mojo` differs by four lines of comment. When you are wondering whether some upstream snippet will work, this example is the standing answer for the non-Cocoa parts of the language.

The pattern worth stealing is the four-overload operator:

```
def __add__(self, rhs: Self) -> Self:      # Complex + Complex
def __add__(self, rhs: Float64) -> Self:   # Complex + Float64
def __radd__(self, lhs: Float64) -> Self:  # Float64 + Complex
def __iadd__(mut self, rhs: Self):         # +=
```

Same-type, mixed-type, *reflected* mixed-type, and in-place. Miss `__radd__` and `2.0 + c` fails while `c + 2.0` works, which is a confusing afternoon.

One practical caveat. The test suite beside it cannot be run from the IDE. Roast's entry-point rule returns `main.mojo` whenever a project has one, and its test-file exclusion matches `_test.mojo`, which `test_my_complex.mojo` does not. So ⌘R in this project always builds `main.mojo`. To run the suite, point the compiler at the test file directly.

`process`

128 lines. Modular's `std.os.Process` example, also carried in unmodified. It spawns `sleep 1` and waits for it, spawns `sleep 2` and polls it, and kills a child, printing the exit code or terminating signal at each step.

The lesson: none that belongs to this fork. Same as `operators` — it is here to prove that the standard library beneath the Cocoa layer is intact.

It is worth knowing it exists for one practical reason: it is the example to crib from when you want to shell out to a tool from a Mojo application, and it shows the three shapes you actually need — block until done, check without blocking, and terminate.

`life-python`

231 lines across `main.mojo` and `gridv1.mojo`. Conway's Life for the second time in this distribution — but the window belongs to **pygame**, not Cocoa. The grid logic is Mojo; the drawing goes through CPython:

```
var pygame = Python.import_module("pygame")
pygame.init()
var window = pygame.display.set_mode(
    Python.tuple(window_width, window_height)
)
```

The comparison is the demo. Put it beside the native `life` example — the same game, the same rules, two windowing worlds — and the difference is immediate. The README does not oversell it, and neither will I:

> the first thing everyone says is how much faster the Cocoa one feels. That is the demo working. (To be fair to Python, upstream's loop also sleeps 0.1s per generation; to be fair to Cocoa, nobody has ever asked it to slow down.)

That parenthesis matters. A good part of the visible gap is a deliberate sleep in upstream's loop, not Python's cost, and a comparison that hid this would be a worse example. The honest claim is narrower than the dramatic one: the Cocoa path is the one that can be asked for 60 frames a second, and the pygame path is the one that gets you a window in four lines.

This is also the only example that needs a Python environment. It ships a `requirements.txt` naming `pygame-ce` rather than classic pygame, for a concrete reason worth repeating — the community build publishes wheels for current CPython, where classic pygame had none for 3.14 and tried to compile itself against an SDL that was not there. It imports as `pygame` either way.

Two menu items, once, before the first run:

1. Python menu → **Create or Repair Environment**
2. Python menu → **Install Project Dependencies**

Roast builds an environment only for projects that actually use Python, so opening the other sixteen examples will not create one.

The lesson: interop is real, and it is a trade rather than a win. Python gets you a window and a drawing call immediately, and costs you the frame rate and a dependency you have to install. Knowing which of those two you are buying is the entire content of the example.

2. The GPU primers

Three GPU examples come from Modular's own collection and run here **unmodified** — the fork added a comment to the top of each and changed nothing else. `vector-add/main.mojo` and `grayscale/main.mojo` differ from upstream by four and three comment lines respectively; `tiled-matmul/main.mojo` by four.

That is the point of having them. This fork wrote its own Metal backend for the GPU path, and the question a reader most wants answered is whether ordinary Mojo GPU code survives it. These three answer it without the fork getting a vote: upstream's `DeviceContext`, `TileTensor`, `enqueue_function`, `barrier()` and `AddressSpace.SHARED` compile and run here as written.

Read them for the idioms. Do not read them for a lesson about this fork, and be aware of how little two of the three actually check.

`vector-add`

80 lines. Three ten-element buffers, one thread per element, `1.25 + 2.5` ten times. It prints `Resulting vector:` and ten copies of `3.75`.

The launch is the whole reason to read the file:

```
ctx.enqueue_function[vector_addition](
    lhs_tensor, rhs_tensor, out_tensor, Int32(VECTOR_WIDTH),
    grid_dim=grid_dim,
    block_dim=BLOCK_SIZE,
)

with out_buffer.map_to_host() as host_buffer:
    var host_tensor = TileTensor(host_buffer, layout)
    print("Resulting vector:", host_tensor)
```

The kernel is a compile-time parameter, its arguments are positional, and the geometry is keyword-only at the end. Reading the result back is a `with` block. That five-line shape is the entire API surface a first GPU program needs, and it does not get more complicated in the larger examples — `mandelbrot` and `fluid` call `enqueue_function` exactly like this, just more often.

Note the guard, because the three examples do not agree on it:

```
comptime assert has_accelerator(), "This example requires a supported GPU"
```

That is a *compile-time* assert. On a machine with no Metal device this example fails to build rather than degrading to a message.

The lesson: none specific to this fork. It is a hello-world that happens to touch the GPU. It is also worth saying plainly that **it never checks its own answer** — the ten values are printed for a human to look at, and nothing would fail if they were wrong. As a compatibility proof it is the weakest of the three. Read it for the launch idiom, then move on.

grayscale

115 lines, and the 2D sibling of `vector-add`. It builds a 10×5 RGB image on the host, converts it to single-channel grey on the GPU, and prints the result as a table of integers.

One idea, and it is the one everybody gets wrong once:

```
var row = global_idx.y
var col = global_idx.x

if col < WIDTH and row < HEIGHT:
    var red = rgb_tensor[row, col, 0].cast[float_dtype]()
    ...
    gray_tensor[row, col] = gray.cast[int_dtype]()
```

`global_idx.y` is the row and `global_idx.x` is the column. Transpose those two and you get a kernel that runs, writes plausible-looking numbers, and is wrong in a way that only shows up as a picture that looks sheared. The bounds check matters here too: the grid is one block of 16×16 for a 50-pixel image, so about 80% of the threads launched exist only to fail that `if`.

Two things to know before you trust it as a reference. The luma weights are `0.21 / 0.71 / 0.07`, which sum to 0.99 and are not the Rec.601 coefficients (`0.299 / 0.587 / 0.114`) — an approximation, and upstream's. And because the synthetic image spans so little range, every output value lands between 17 and 29: a near-uniform grey. The example is exercising the plumbing, not producing an image.

The lesson: none beyond vector-add, except the y-is-row convention. Like `vector-add` it never validates anything. If you have read `vector-add`, the only new information here is two-dimensional indexing, and you now have it.

tiled-matmul

345 lines, and the one carried example that teaches something real. It multiplies two 64×64 matrices using 16×16 shared-memory tiles, 4×4 blocks of 16×16 threads.

The lesson is the barrier — specifically that there are **two** of them per iteration:

```
# Ensure all threads finish loading tiles before any thread starts computing
barrier()

comptime for k in range(TILE_K):
    var a_element = tile_a_shared[thread_y, k]
    var b_element = tile_b_shared[k, thread_x]
    accumulator += a_element * b_element

# Ensure all threads finish computing before any thread loads next tiles
barrier()
```

The first barrier is obvious: do not compute on a tile that is still being filled. The second is the one people omit, and omitting it is a genuinely nasty bug — a fast thread races ahead to the next iteration and overwrites shared memory that a slower thread in the same block is still reading. The kernel still runs. The answer is wrong in a way that depends on scheduling.

This pattern generalises to every shared-memory algorithm you will write, and it is the only idea in this chapter that does.

`tiled-matmul` also guards itself differently from the other two:

```
comptime if not has_accelerator():
    print("No GPU detected...")
```

A message at run time, not a failed build. Worth knowing that the two styles exist and that upstream uses both.

The lesson: real, and not fork-specific. Shared-memory tiling with correct double barriers is worth understanding wherever you write GPU code.

One caveat, and it matters. The program prints:

```
Validating GPU results against CPU reference...
```

There is no CPU reference implementation in the file. What actually runs is five hard-coded closed-form spot checks — `(0,0)`, `(0,1)`, `(1,0)`, `(1,1)` and `(3,3)` — against `C[i,j] = (i+1) × 64 × (j+1)`. That is 5 of 4,096 elements, all inside the first block, none of them at a tile boundary. The `✓ Validation PASSED` line is real but it is checking far less than its own wording claims, and a bug in the later K-loop iterations or across tile boundaries would sail straight past it.

It is still the strongest compatibility proof in the carried set, because it is the only one that checks its arithmetic at all. Just do not upgrade "five spot checks passed" into "verified against a CPU reference" — the program does that for you, and it should not.

3. The applications

Four real programs: a window with a simulation in it, twice on the GPU, once not, and one that plays a board game. Between them they answer a question the smaller examples cannot, and it is worth stating before the walkthroughs because it is the reason to read all four rather than one.

> The moment you own a `DeviceContext`, you lose `[NSApp run]`.

`life` is the only one of the four that hands control to Cocoa in the ordinary way. The other three hand-roll an event pump, and all three give the same reason: a `DeviceContext` cannot live in a `named_global`, and a Cocoa callback cannot reach a local — so the loop that owns the GPU has to be the loop that drives the application. Neither file teaches that alone. The pair does.

`life`

643 lines. Conway's Life at 180×120, cells coloured by age, mouse drawing, keyboard control, a 60 Hz timer, presented through a `CAMetalLayer`.

This chapter will not walk it line by line, because Guide, chapter 9 already does — that chapter exists to read this exact program. What belongs here is why it earns a place next to three GPU applications while using **no GPU at all**.

Metal appears in `life` only to put pixels on screen. `evolve()` is a plain double loop over 21,600 cells on the CPU. That is the correct decision, and it follows the rule the ferns chapter argues for: 21,600 cells at 60 Hz is 1.3 million cell-updates a second, which a single core does without noticing. The work is wide, but there is no deadline being missed, so there is nothing for a kernel to buy.

The architectural payoff is that because nothing is a local that a callback needs, `life` can use the normal Cocoa shape — and it is the only example here that shows it:

class	How Cocoa reaches it
`LifeDelegate`	`applicationShouldTerminateAfterLastWindowClosed:`, via `setDelegate:`
`LifeActions`	`lifeTick:`, the target of a 60 Hz `NSTimer`
`LifeView`	`mouseDown:`, `mouseDragged:`, `rightMouseDown:`, `keyDown:` …

Three classes, three different ways Cocoa calls into your code, and `[NSApp run]` on the last line.

Two comments in this file are worth more than the simulation. The first states a Mojo rule that will eventually bite everyone:

```
"""Zeroed heap memory that nothing in Mojo owns.

The buffers must outlive `main`'s locals: Mojo destroys a value at its LAST
USE, not at end of scope, so a `List` whose `.unsafe_ptr()` we stash is
freed immediately and every stored pointer dangles -- which shows up much
later as a corrupted allocator, nowhere near the cause.
"""
```

Destruction at last use, **not** at end of scope. Stash a pointer into a `List` and the list can be gone before the next line.

The second is a diagnostic nobody would predict: the timer's selector is `lifeTick:` rather than the obvious `tick:`, because the SDK already declares `tick:` on `CASecureFlipBookLayer` taking a double. An invented selector that collides with a known one gets registered with the SDK's shape — `v@:d` where `v@:@` was meant. The compiler said so, which is the part worth noticing.

The lesson: this is what a normal CocoaMojo application looks like, and a demonstration that declining a GPU is a legitimate answer.

mandelbrot

488 lines. A live-zooming fractal at 60 fps, every pixel computed *and* coloured by one Mojo kernel. There is no shader anywhere in the pipeline.

One dispatch per frame does everything: `mandelbrot_color_kernel` computes the escape count and applies the cosine palette in the same kernel, so the CPU receives finished BGRA and its entire job is `map_to_host`, a copy, and `replaceRegion:` into the drawable's texture.

Before the window opens, it times itself honestly:

```
# Warm, then time: the first launch carries compilation and wiring.
ctx.enqueue_function(kern, dev, cx, cy, scale, grid_dim=(GRID), block_dim=(BLOCK))
ctx.synchronize()
var g0 = perf_counter_ns()
ctx.enqueue_function(kern, dev, cx, cy, scale, grid_dim=(GRID), block_dim=(BLOCK))
ctx.synchronize()
var g1 = perf_counter_ns()
```

A discarded warm-up launch, then the measured one. The first launch of any kernel carries compilation and pipeline setup; timing it and reporting the number is the most common way GPU benchmarks lie. The same file runs the identical arithmetic on one CPU core and prints both.

Three details worth stealing:

isFlipped instead of arithmetic. Cocoa's origin is bottom-left and the kernel counts pixels from the top. Rather than subtracting in every handler, the view says:

```
def isFlipped(self) -> Bool:
    # Origin at the top-left, exactly as the kernel counts pixels, so a
    # click converts to a pixel without anyone flipping an axis.
    return True
```

`life` does the same conversion by hand. This is better.

send rather than msg_send for Metal. The concrete classes behind `MTLDevice` and friends are private, so there is no public class name for `cocoakb` to check a selector against. That is the rule for choosing between the two dispatch helpers: typed `msg_send` when the database knows the class, dynamic `send` when it cannot.

A headless mode that does not steal focus. `MANDEL_FRAMES=N` renders N frames and exits, with the window brought up as an Accessory and unfocused so a benchmark run does not take the desktop from whoever is working. The header notes that this lesson "was learned the loud way." Frame rate goes to stdout as well as the title bar, because a title bar is invisible to a captured run.

Correctness is checked outside the example, in `spikes/mandelbrot/compute_smoke.mojo`, and the threshold is worth understanding: **exact agreement on more than 99.0% of pixels**, not a float tolerance. The GPU fuses multiply-add where the CPU does not, which shifts escape counts in the thin chaotic band at the set's boundary. That is inherent to a Float32 Mandelbrot, not a bug, and a tolerance-based check would have been the wrong instrument.

The lesson: one kernel can own the whole frame, and this is the smallest honest CPU-versus-GPU timing harness in the distribution.

`fluid`

911 lines. Stable Fluids — drag the mouse, dye swirls through a velocity field that is advected along itself and then made divergence-free by a Jacobi pressure solve. Six kernels, all Mojo, no shader.

It exists for a reason its own header states, and the reason is more interesting than the fluid:

> mandelbrot is one dispatch per frame and life is none, so neither measures launch cost.

A fluid step is roughly **35 dependent dispatches** — two advections, divergence, thirty Jacobi sweeps ping-ponged in pairs, projection, dye advection and a shade pass. Each waits on the one before. That makes per-dispatch round-trip the dominant term rather than a rounding error, and `fluid` is the only example in the set shaped to measure it.

The measurement has moved as the runtime improved, so take the *shape* of the result rather than any single ratio. The example's header records an early figure of 10.19 ms/step synchronous against 1.99 ms/step asynchronous on an M4, with the synchronous path also showing a 70% spread between cold and warm runs while the asynchronous path held to ±0.2%. Later measurements with dispatch batching added put the same 35-dispatch workload at 3.87 ms/step synchronous against 1.49 batched-and-asynchronous, and on an M4 Max batching alone moves 1.06 to 0.93. The ratio has fallen from 5.1× to 2.6× because the slow side got faster; the conclusion has not moved at all. Launch overhead, not arithmetic, is what a many-kernel step is made of.

> **Note.** The header's instruction to set `APPLEGPU_ASYNC_LAUNCH=1` is stale. Asynchronous, command-buffer-batched launch is now the default; `APPLEGPU_SYNC_LAUNCH=1` is what restores the old synchronous behaviour, and `APPLEGPU_BATCH_DISPATCHES=0` isolates batching.

Two ideas here are worth more than the benchmark.

The Jacobi solve is ping-ponged, and the comment says why:

```
Ping-ponged rather than updated in place: a Jacobi step reads the previous
iterate's whole neighbourhood, and writing in place would feed half-new
values back in, silently turning this into Gauss-Seidel with a
thread-order-dependent answer.
```

"Silently" is the operative word. In-place would converge to something plausible and wrong, and differently wrong on different hardware.

There are two run loops, and a hand-rolled pump only services one of them. This is the single most valuable paragraph in the four applications:

```
# Spin the run loop briefly. `nextEventMatchingMask:` with
# distantPast polls the event queue and returns at once -- it never
# services the Mach port Apple Events are delivered on, so without
# this the handlers registered above are simply never called.
```

`fluid` is scriptable — it registers an Apple Event handler for five verbs — and draining the AppKit event queue does not deliver Apple Events, which arrive on a different Mach port. Without an explicit `CFRunLoopRunInMode` spin, a correctly registered, correctly addressed, perfectly well-formed event is simply never delivered. The same applies to `finishLaunching`, which a hand-rolled pump must call itself because it is where AppKit attaches that port to the run loop.

Two things to keep in mind if you use this file as a reference. Its header claims the companion `spikes/fluid/fluid_smoke.mojo` checks mass conservation to 0.6% and post-projection divergence within ±0.05; the smoke test actually enforces three qualitative conditions — dye did not vanish, dye did not blow up, the splat landed — and prints those two figures for a human to read. They are observations, not asserted tolerances. And unlike `life` and `mandelbrot`, `fluid` declares no view class at all: it picks events apart by integer type inside the pump.

The lesson: dispatch cost is a real budget line, and a hand-rolled pump silently drops any input channel it does not explicitly service.

othello

1,024 lines across `board.mojo`, `ai.mojo` and `main.mojo`. The board game on a green felt board, with four computer players — and the only example in the distribution that makes a falsifiable engineering argument and then declines to take the flattering side of it.

The question is where a GPU helps a computer player. The answer is: for one of the two classic search algorithms, and not the other.

Alpha-beta does not want a GPU, and the file says so at the top of `ai.mojo`:

```
# Alpha-beta is the classic answer and it does NOT want a GPU. Its whole
# advantage is that a branch which cannot beat what you already have is never
# examined, so the work each thread does depends on what every other thread
# found -- the opposite of what a GPU is for. Threads would diverge on the
# first cutoff, and an 8x8 board at four ply is a few hundred microseconds of
# CPU anyway. Shipping that on the GPU would be slower and dishonest.
```

The measurements back it: depth 3 is 22 µs, depth 4 is 81 µs, depth 6 is 1,009 µs. There is no deadline being missed, and the pruning that makes the algorithm good is exactly what threads in lockstep would have to give up.

Monte-Carlo playouts do. Play the position out at random a few thousand times per candidate move and count wins: every game is independent, holds its whole state in two registers, touches no memory until it writes a single `Int32`, and runs the same instructions as its neighbours. Threads are laid out as `(move, playout)` so a whole warp shares a candidate. Measured here: **16,384 playouts take 136.8 ms on the CPU and 3.4 ms on the GPU — 40×**, and both pick the same move.

So the Master level is a GPU player and the other three are not. Master beats Advanced 6–0.

The reduction is done on the host on purpose: an atomic per playout "would serialise exactly the thing that is supposed to be parallel."

Two traps in this example are worth carrying away.

A legal construct that the Metal backend cannot link. The winner test is written as arithmetic rather than the obvious three-way branch:

```
# The obvious `if b > w: 1 elif w > b: -1 else: 0` is folded into LLVM's
# three-way compare intrinsic, and the Metal backend has no such
# instruction -- the kernel then fails to link with "Undefined symbols:
# llvm.scmp.i32.i64", which says nothing about what wrote it.
# `Int(b > w) - Int(w > b)` is ALSO folded: zext(a>b) - zext(b>a) is the
# canonical shape LLVM turns into scmp. This one is not that shape.
return Int(b > w) * 2 - Int(b != w)
```

Note that the *obvious rewrite is also wrong*. Both the branch and the subtraction fold into the same intrinsic; only the third form survives.

> **This has since been fixed in the backend.** AIR legalisation now expands `llvm.scmp` and `llvm.ucmp` into selects, so the obvious three-way branch compiles and runs, and a surviving intrinsic fails the build by rule rather than reaching the reader as an undefined symbol. The workaround is still in othello's source and still correct — it is simply no longer required. The lesson that outlives the bug is the shape of the failure: a legal construct the optimiser *introduced*, invisible to a target-agnostic verifier, and named only at pipeline creation.

A struct return that needs the typed msg_send. An `NSPoint` comes back in two registers, and the dynamic path does not describe that to the ABI:

```
# The point comes back through the TYPED msg_send. ... the dynamic `Obj[...]`
# path does not describe that to the ABI -- the call returns nothing usable
# and every click lands on the same wrong square, silently.
let at = msg_send[CGPoint, "NSEvent", "locationInWindow"](event)
```

The move generator underneath all this is 121 lines of bitboard shifts with no struct at all — two bare `UInt64`s are the board — validated against published perft counts (4, 12, 56, 244, 1396, 8200, 55092). That representation is what makes the parallel player *possible*, not merely faster: a playout fits in registers because the board does.

The lesson: "can this use a GPU?" is the wrong question. "Is this work wide, and am I out of time?" is the right one, and the same program can answer it differently for two of its own features.

4. Sound, and time

The two largest examples in the distribution make noise. They are here for different reasons: `chip` answers "what is `fn` actually *for*", and `abcplayer` is the closest thing the set has to a full application — a real parser, an exact clock, and two independent output backends.

Both run code on a real-time audio thread, which is the strictest execution environment in the collection. No allocation, no locks, no raising, no unbounded loops. Every one of those constraints is annotated in the source at the line that obeys it.

`chip`

2,159 lines across four files. A chip-tune synthesiser in the spirit of the 6581: three voices with sawtooth, triangle, pulse and noise, ring modulation, hard sync, ADSR envelopes and a shared resonant filter — driven by a 50 Hz player routine, with an oscilloscope on a C64 palette.

The lesson, stated plainly

This example exists to show that **one Mojo process can hold both halves of a foreign ABI with no shim**. `class` gives Cocoa an Objective-C object to send messages to. `fn` gives CoreAudio a bare C function pointer to call on a real-time thread. Neither needs a C file, a bridging header, or a block.

The type is spelled as a `comptime`:

```
# The callback's type. An AURenderCallback is a C function pointer, and `fn`
# in a type position is exactly that -- the same aliasing the Objective-C
# bindings use for an IMP.
comptime AURenderCallback = fn(P, P, P, UInt32, UInt32, P, /) -> Int32
```

and installing it is four lines:

```
var cbfn: AURenderCallback = render
let fn_addr = Pointer(to=cbfn).unsafe_bitcast[Int]()[]
var cbs = external_call["calloc", P](Int(2), Int(8))
cbs.unsafe_bitcast[Int]()[unsafe_offset=0] = fn_addr
cbs.unsafe_bitcast[Int]()[unsafe_offset=1] = Int(st)
```

That is an `AURenderCallbackStruct` — a function pointer and a `refCon` — built by hand and handed to `AudioUnitSetProperty`. Because `fn` *is* a C function pointer, the eight bytes in that slot are exactly what CoreAudio wants.

Note the second slot. All synthesiser state hangs off one pointer passed through `inRefCon`, so no global holds the chip and two of these could run at once. Even the UI's own flags live in the unused tail of that block, so the callback still needs only the one pointer. The full treatment of this pattern is Guide, chapter 6, under "When the caller is not Objective-C".

The old AudioUnit API is used rather than `AVAudioSourceNode` for a stated reason: the modern one takes an Objective-C block, and **Mojo cannot construct a block**. That is a real limitation of the fork, and it is why the older API is the right one here.

What the real-time thread costs you

`fn` is non-raising, and that has consequences that reach down into the DSP:

```
# std.math.sin is a `def`, and a `def` may raise. The render callback is an
# `fn` -- non-raising, C-callable -- so it cannot call one.
```

So the filter carries a hand-rolled nine-term sine series. And its range reduction is deliberately a single step rather than a loop, because the argument grows with the frame counter — a subtract-until-in-range loop would get one iteration longer every fifty frames, "on the audio thread, where the cost eventually shows up as a dropout and never as an error."

There is no lock in the callback, and the comment is explicit that this is a choice rather than an oversight: the main thread reads the same state to draw the meters, and "a torn read costs a wrong pixel for one frame, where a held lock would cost a click in the speaker." The oscilloscope's ring buffer is unsynchronised for the same reason.

The bug worth keeping

The example documents a freeze that took real time to find, and the annotation is preserved at the line that fixes it:

```
# `var`, not `let`. `let` names the storage of `i` rather than copying
# it, so the index recorded here would follow `i` as the loop below
# advances it -- and voice_part would then be handed 96 instead of 32,
# pointing this voice at another voice's events and, past the end of the
# block, at whatever the heap holds. That read is what hung the audio
# thread: a note of a few billion is an octave loop that never finishes.
var bass_first = i
```

This is the fork's `let`-binds-by-reference rule producing its worst possible symptom. A garbage note is not a wrong pitch — it is a hang, because the octave normalisation loops. And a hang on the audio thread stops the speaker without raising anything. The fix that stuck was a clamp in `midi_hz`, on the grounds that the audio thread must never be handed an input it can loop forever on.

Two other Cocoa traps are recorded here and are worth knowing:

- **The retain is not optional.** A Mojo-made `NSView` handed to AppKit is released at the end of the statement that made it, after which the first `drawRect:` traps inside AppKit with a stack that says nothing about ownership.
- **Build fonts once.** Asking for a font inside `drawRect:` thirty times a second eventually returns nil, and a nil into an attributes dictionary raises from inside AppKit's drawing machinery.

The lesson: fn is the C ABI, and a real-time thread means what it says. Both documented failures here were silence rather than crashes, which is the failure mode that makes you blame the audio system instead of your code.

abcplayer

3,344 lines across nine files — the largest example in the distribution. It reads ABC notation, parses it fully (repeats, first and second endings, tuplets, broken rhythm, ties, grace notes, chord symbols, key signatures and modes), schedules it to the sample, and plays it through either the chip synthesiser or General MIDI — or writes a standard MIDI file instead.

Time is an integer

The central design decision is stated in `model.mojo` and everything follows from it:

> **Time is an integer.** Durations are counted in ticks at 480 per quarter note — 1920 per whole note — and never in seconds or doubles. That number is the standard MIDI resolution and it divides exactly by everything ABC can ask for: a 1/64 note is 30 ticks, a triplet eighth is 160, a dotted quarter is 720. Nothing rounds, so a tune that should land on the bar line does, however many tuplets and dots came before it.

There is exactly **one** place where that integer clock becomes a sample position, and it rounds once:

```
fn tick_to_sample(tick: Int, bpm: Int, per_beat: Int, sample_rate: Int) -> Int:
    let denom = bpm * per_beat
    if denom <= 0:
        return 0
    return (tick * sample_rate * 60 + denom // 2) // denom
```

Choosing 480 also means the MIDI writer needs no conversion at all — the ticks it writes are the ticks the model holds.

Dispatch inside the buffer, not at its edges

The timing argument is the reason this example was written, and it is measurable. Notes are not started by waking a thread at the right moment; the schedule is compiled to sample offsets ahead of time, and the render callback splits its own buffer at each event:

```
span = min(frames - filled, next_event - now)
```

The measurement, taken with 512-frame buffers deliberately chosen so that no onset falls on a buffer boundary: scheduled at samples 0, 48000, 96000, 144000; measured at 0, 48001, 96001, 144000. **Worst error one sample — 0.021 ms** — and the source is candid that the one-sample discrepancies are the onset detector's threshold rather than the scheduler.

One ordering detail carries real musical weight. The schedule sorts NOTE_OFF before NOTE_ON at the same sample, "because releasing first leaves the voice free for the new note. The other order steals a voice that is about to be freed and drops a note."

Two backends from one model

The same schedule drives three outputs, which is the strongest argument for keeping the model free of audio concerns: the chip synthesiser (three voices, with stealing that takes the oldest sounding note "because it is the one furthest through its decay and the least missed"), Apple's DLS synth for General MIDI, and a standard MIDI file.

The file is the interesting one, and the README says why:

> The MIDI file is the proof. It opens in any notation program, so the pitches, the lengths and the bar positions can be checked by something that was not written here.

That is the right instinct about correctness. A parser that only feeds its own synthesiser can be wrong in a way that sounds fine.

What the port found

`abcplayer` is a port of a C++ program, and it fixed five defects in it. Four were findable by reading:

1. **Key signatures did nothing.** `applyKeySignature()` returned its argument unchanged, so every tune played in C major.
2. **Explicit accidentals were dropped.** The test looked for `#` and `b` rather than ABC's `^` and `_` — and was unreachable anyway, behind a guard.
3. **First and second endings were ignored**, giving `A B A B` where the music says `A B A C`.
4. **Middle C was an octave low**, from `base_octave * 12` with `base_octave` of 4 — "a C, an octave low, and sounds plausible enough to survive review."
5. **Chord symbols corrupted the melody's timeline**, because generated chord notes were appended to the melody voice and advanced its clock.

The first two meant the original played most folk tunes wrong from the opening bar.

A catalogue of compiler traps — most of them now historical

`abcplayer` records six failure modes it hit during the port, each annotated at the line that works around it, and each with its *symptom* rather than just its rule. That made it the most useful bug catalogue in the distribution.

Every one was retested against the shipped compiler in August 2026, and four no longer reproduce. The workarounds are still in the source, so the examples below describe code that is no longer necessary. They are kept here because the comments are still in the files, and a reader who meets one deserves to know it is a fossil.

What the source says	Retested
+= through a `List` subscript updates a temporary — rests occupied no time	**Fixed.** Reverting the workaround produces a byte-identical MIDI file
A `fn` returning a heap-owning type crashes in `DialectConversion`	**Fixed.** Now a clean diagnostic asking for `.copy()`
Passing a `mut` struct on to a second function crashes at the call site	**Fixed.** Works
Reading `self` while appending to a `List` `self` owns crashes	**Fixed.** Works
A `let` bound into a `List` that later *grows*	**Caught** — but reported as `error: failed to run the`

dangles	`pass manager`, which says nothing
A `let` naming a list slot that is later *written* breaks an insertion sort	**Still real, still silent**

That last row is the one to carry away, because it is not a bug — it is let binding by reference, working exactly as designed, in its most damaging form. The insertion sort in `midi.mojo` is four lines of ordinary code:

```
for a in range(1, len(times)):
    let t = times[a]        # names the slot; the shift below writes over it
    var b = a - 1
    while b >= 0 and times[b] > t:
        times[b + 1] = times[b]
```

Sorting `5 3 9 1` that way yields `5 5 9 9`. No error, no warning, and the comment in the source telling you to write `var` is the only thing standing between you and a sort that quietly loses entries.

One non-compiler caveat also stands: **rebuilding a string with chr() mangles UTF-8** — slice instead, or a title with an accent comes out as mojibake.

The cost of the historical traps was real: `music.mojo` is 823 lines *partly because* the chord emitter, bar mirror and broken-rhythm handler had to be written out inline. That shape is now a scar rather than a requirement, and the workarounds could be unwound with a test at each site.

The lesson: exact timing is a representation problem, not a scheduling problem. Pick an integer clock that divides by everything the input can ask for, round once, and dispatch inside the buffer. Everything else — three backends, a MIDI file that other programs can check — follows from the model staying clean.

> **One caveat if you are reading or editing this example.** `abcplayer` ships its own copy of `chip.mojo`, a verbatim duplicate of `chip/chip.mojo` with a provenance header asking that the two be kept in sync. That is a deliberate trade — an IDE example should be a folder that opens and runs with no include flags — but it is a real drift risk, and a fix to one will not reach the other.

5. Python, used well

Two examples in this distribution reach into CPython, and they answer different questions. life-python asks what it costs to borrow another windowing world. `bifurcation` asks something more useful: **what is Python actually better at than Mojo, and how do you arrange a program so each does that part?**

The answer this example argues for:

> Python is very good at everything that *surrounds* a result — axes, scales, legends, labels, colour maps, file formats. It is poor at the loop that produced the result. Put the loop in Mojo and the presentation in Python, and measure the boundary rather than assuming it.

`bifurcation`

The logistic map, `x → r x (1 − x)`, swept across 1,600 values of `r` with 2,000 iterations recorded per value. The attractor doubles, doubles again, and falls into chaos — with windows of order inside it. A second panel plots the Lyapunov exponent, which is positive exactly where the map is chaotic and dips below zero exactly where the windows are.

Why the compute stays in Mojo

The map is a **recurrence**: every iterate depends on the one before it. There is nothing to vectorise, and nothing to hand a GPU — this is the same shape the ferns chapter identifies in the chaos game, and it reaches the same conclusion for the same reason. It is simply a lot of arithmetic that has to happen in order, 6.4 million steps of it.

The program measures the boundary instead of asserting it, running **the same algorithm** on both sides — histogram write and logarithm included:

```
Mojo      23.8 ms   for all 1600 columns
CPython    8.5 ms   for 24 columns   ->   0.6 s extrapolated
ratio     20.2 x
```

Two things about that measurement are deliberate.

It uses `timeit`, which runs the loop **in a function scope with the collector off**. That is CPython's best case: the same loop written at module scope, where every name is a global lookup, is about twice as slow again. Choosing the favourable number is the point — a comparison you have tilted in your own favour proves nothing.

And the honest answer here is about **20×**, not the 100× a careless benchmark would report. Twenty is still the whole argument: half a second is tolerable once and intolerable in a loop, while the figure itself costs nothing worth measuring.

Why the picture comes from Python

`fern/` in this collection writes a PNG by hand. Its `png.mojo` is 125 lines of CRC, Adler-32 and deflate's stored mode, and what you get for that is pixels in a file: no axes, no ticks, no scale, no legend, and a 2 MB output because it compresses nothing. That is the right answer when a bitmap is all you want.

This example wants the other thing, and matplotlib has already solved it:

```
var im = top.imshow(
    dens,
    extent=extent, origin="lower", aspect="auto",
    cmap="magma", norm=mcolors.PowerNorm(0.35),
    interpolation="nearest",
)
var cb = fig.colorbar(im, ax=Python.list(top, bottom), pad=0.01)
```

Every keyword there is a decision somebody made well, and reimplementing the set of them is a career rather than an afternoon.

How the data crosses

Not one value at a time. `std.python.numpy` copies a flat Mojo `Span` straight into a real NumPy array:

```
var dens = copy_to_numpy_tensor(density, Coord(H, W))
var lam = copy_to_numpy_array(lyap)
```

Appending 1.4 million values across the bridge individually is the slow path, and avoiding it is precisely why that module exists — its own docstring names matplotlib as the case it was written for.

Two details worth stealing

Give the colorbar both axes. A colorbar takes its space from the axes you hand it, so `ax=top` alone makes the top panel narrower than the bottom, and two panels that share an x-axis but do not line up are worse than no colorbar at all. `ax=[top, bottom]` steals from both and keeps them aligned — which is what lets you read a window in the diagram and see its λ dip directly beneath.

PowerNorm, not a log norm. Most buckets in a bifurcation histogram are empty, and the log of zero is not a colour.

The lesson: measure the boundary, then put each half where it belongs. The interesting claim is not "Mojo is faster" — it is that the 20× matters for one half of this program and is irrelevant for the other, and that you can tell which is which by timing it rather than by taste.

GPU programming in Mojo

Mojo's premise is that one source file specialises to whatever silicon you point it at, and the GPU is where that claim is cashed. A kernel is an ordinary Mojo function. There is no separate shading language, no string of source compiled at run time, and no second toolchain — you write Mojo, and the same compiler that produced your CPU code produces the GPU code.

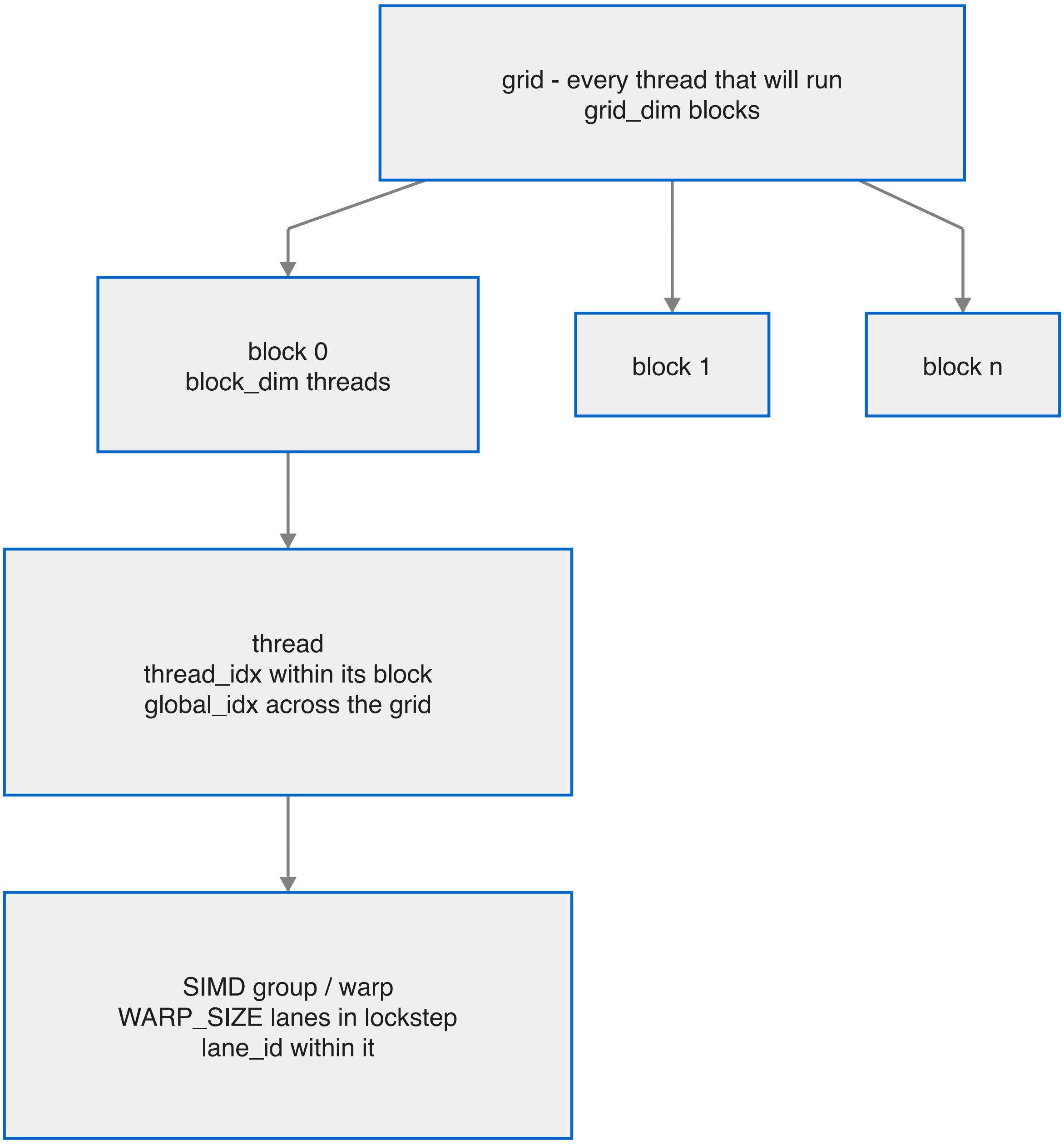
```
from std.gpu import global_idx
from max.gpu.host import DeviceContext

def scale(data: Pointer[Float32, MutAnyOrigin], factor: Float32, n: Int):
    var i = Int(global_idx.x)
    if i < n:
        data[unsafe_offset=i] = data[unsafe_offset=i] * factor
```

That is a complete GPU kernel. The rest of this section is about how to launch it, how to think about the threads that run it, and how to know the answer is right.

Document	What it covers
1. Your first kernel	The execution model, and a complete program from allocation to result
2. Threads, blocks and memory	Indexing, shared memory, barriers, and SIMD-group operations
3. Running and checking	Building, timing, verifying results, and what this hardware supports
4. Three ferns: when a GPU kernel is worth writing	A guided tour of `fern/`, `ferns/` and `fernwind/`: where the crossing from CPU to GPU actually happens, and why the first two are right to stay put

The execution model, briefly

The same function body runs on thousands of threads at once. Each thread discovers *which* piece of work it owns by asking for its own index, does that piece, and stops. Threads are grouped into **blocks**, blocks make up the **grid**, and only threads within a block can cheaply cooperate.



If you have written CUDA or Metal compute, this is the model you already know, with Mojo's names on it. If you have not, the next chapter builds it up from a working program.

Where this runs

This fork targets the **Apple Silicon GPU** through Metal. The reference machine is an M4 Max. Kernels are ordinary Mojo, so the same source is what you would run on another vendor's hardware through a different backend — but on this machine, Metal is the backend and `DeviceContext(api="metal")` is how you ask for it.

1. Your first kernel

A kernel is a function

There is no kernel keyword and no attribute. A kernel is a `def` whose parameters can cross to the device, which asks for its own thread index and does one element's worth of work:

```
from std.gpu import global_idx

def mandelbrot_kernel(
    escape: Pointer[UInt32, MutAnyOrigin],
    center_x: Float32,
    center_y: Float32,
    scale: Float32,
):
    var idx = Int(global_idx.x)
    if idx < PIXELS:
        var px = idx % WIDTH
        var py = idx // WIDTH
        var cx = center_x + (Float32(px) - Float32(WIDTH) * 0.5) * scale
        var cy = center_y + (Float32(py) - Float32(HEIGHT) * 0.5) * scale

        var zx = Float32(0)
        var zy = Float32(0)
        var n = UInt32(0)
        while n < UInt32(MAX_ITER) and zx * zx + zy * zy <= Float32(4):
            var nzx = zx * zx - zy * zy + cx
            zy = Float32(2) * zx * zy + cy
            zx = nzx
            n += 1

        escape[unsafe_offset=idx] = n
```

Three things in that signature are worth pausing on.

Pointer[UInt32, MutAnyOrigin] — device buffers use `MutAnyOrigin`, not the `MutUntrackedOrigin` that Cocoa code uses. The pointer is tracked; it just has no single named origin.

Scalars pass by value. `center_x`, `scale` and friends are copied into the launch, so small parameters cost nothing to pass and need no buffer.

The bounds check is not optional. Grids are rounded up to whole blocks, so the final block runs threads past the end of your data. Without `if idx < n` those threads write outside the buffer.

The host side, in five calls

```
from max.gpu.host import DeviceContext

def main() raises:
    var ctx = DeviceContext(api="metal")
    var dev = ctx.enqueue_create_buffer[DType.uint32](PIXELS)
    var f = ctx.compile_function[mandelbrot_kernel]()
```

```
    comptime block = 256
    comptime grid = (PIXELS + block - 1) // block

    ctx.enqueue_function(f, dev, cx, cy, scale,
                         grid_dim=(grid), block_dim=(block))
    ctx.synchronize()
```

Call	What it does
`DeviceContext(api="metal")`	Opens the device
`enqueue_create_buffer[DType.T](n)`	Allocates n elements of device memory
`compile_function[kernel]()`	Compiles the kernel and builds its pipeline
`enqueue_function(f, args…, grid_dim=, block_dim=)`	Launches it
`synchronize()`	Waits for completion

`compile_function` is the expensive call. Hoist it out of any loop — compile once, launch many times.

Choosing the grid

```
comptime block = 256
comptime grid = (PIXELS + block - 1) // block
```

`block_dim` is how many threads are in each block; `grid_dim` is how many blocks. The ceiling division is the standard idiom for "enough blocks to cover `n` items", and it is exactly why the bounds check inside the kernel is required.

256 is a reasonable default block size. Both take tuples for 2D and 3D work — `grid_dim=(gx, gy)`, `block_dim=(16, 16)` — and then your kernel reads `global_idx.x` and `global_idx.y`.

Getting the answer back

Apple Silicon has unified memory, so reading results is a mapping rather than a copy:

```
with dev.map_to_host() as host_view:
    for i in range(PIXELS):
        ...host_view[i]...
```

It is a context manager, so the mapping is released at scope exit.

For explicit transfers — and for portability to discrete GPUs — buffers also carry `enqueue_copy_to`, `enqueue_copy_from` and `enqueue_fill`, and the context can allocate a host buffer with `enqueue_create_host_buffer`.

The whole program

Putting it together, with a CPU reference to check against:

```
from std.gpu import global_idx
from max.gpu.host import DeviceContext
from std.time import perf_counter_ns

comptime WIDTH = 1024
comptime HEIGHT = 768
comptime PIXELS = WIDTH * HEIGHT
comptime MAX_ITER = 256


def main() raises:
    comptime cx = Float32(-0.75)
    comptime cy = Float32(0.0)
    comptime scale = Float32(3.0) / Float32(WIDTH)

    # CPU reference
    var cpu_list = List[UInt32](length=PIXELS, fill=0)
    var cpu_buf = cpu_list.unsafe_ptr().unsafe_origin_cast[MutAnyOrigin]()
    mandelbrot_cpu(cpu_buf, cx, cy, scale)

    # GPU
    var ctx = DeviceContext(api="metal")
    var dev = ctx.enqueue_create_buffer[DType.uint32](PIXELS)
    var f = ctx.compile_function[mandelbrot_kernel]()
    comptime block = 256
    comptime grid = (PIXELS + block - 1) // block

    ctx.enqueue_function(f, dev, cx, cy, scale,
                         grid_dim=(grid), block_dim=(block))
    ctx.synchronize()

    var agree = 0
    with dev.map_to_host() as gpu_buf:
        for i in range(PIXELS):
            if gpu_buf[i] == cpu_buf[i]:
                agree += 1
    print("agreement:", Float64(agree) / Float64(PIXELS) * 100.0, "%")
```

On the reference M4 Max the most recent run of this reports:

```
GPU: 0.413 ms   speedup: 197.96 x
exact agreement: 100.0 % ( 0 boundary-band pixels differ)
```

Before you take that as a benchmark, read Running and checking — both the timing and the comparison need more care than they look like they do.

Running it

From the CocoaMojo distribution, one command either way:

```
cocoamojo --run   mandelbrot.mojo      # JIT: compile and run
cocoamojo --build mandelbrot.mojo      # -> ./mandelbrot
```

Both work for GPU code. `--run` JITs the program, GPU kernels included, and `--build` produces an ordinary double-clickable binary.

No `-I` flags, no `MODULAR_*` variables, no Bazel — the distribution carries the compiler, the runtimes, the packages and the SDK database beside each other, and `cocoamojo` wires them up. Chapter 3 covers the details, including what to do when you are building against the source tree rather than a distribution.

2. Threads, blocks and memory

Chapter 1 used one index and one buffer. Real kernels need to know where they sit in the grid, cooperate with their neighbours, and use faster memory than the device buffer. This chapter is those three things.

Knowing where you are

```
from std.gpu import (
    thread_idx, block_idx, block_dim, grid_dim, global_idx,
    lane_id, warp_id, WARP_SIZE,
)
```

Name	What it tells you
`thread_idx.x/.y/.z`	This thread's position **within its block**
`block_idx.x/.y/.z`	This block's position within the grid
`block_dim.x/.y/.z`	How many threads per block
`grid_dim.x/.y/.z`	How many blocks in the grid
`global_idx.x/.y/.z`	Position across the whole grid
`lane_id`	This thread's lane within its SIMD group
`warp_id`	Which SIMD group within the block
`WARP_SIZE`	Lanes per SIMD group

`global_idx` is the convenience you will use most; it is `block_idx * block_dim

- thread_idx` computed for you.

For 2D work, index both axes:

```
def blur(out: Pointer[Float32, MutAnyOrigin], w: Int, h: Int):
    var x = Int(global_idx.x)
    var y = Int(global_idx.y)
    if x < w and y < h:
        var i = y * w + x
        ...
```

launched with `grid_dim=(gx, gy), block_dim=(16, 16)`.

A note on `WARP_SIZE`

Do not hardcode it. Apple SIMD groups are 32 lanes; AMD's wave64 is 64. `WARP_SIZE` is a compile-time constant for the target you are building for, and a kernel that assumes 32 or 64 is a kernel that silently produces wrong answers on the other vendor. This is one of the commonest portability bugs in GPU code and it costs nothing to avoid.

Shared memory

Every thread can reach the device buffers you passed in, but that memory is comparatively slow. Threads *within a block* also share a small, fast scratchpad — Metal calls it threadgroup memory, CUDA calls it shared memory — and using it well is most of what separates a fast kernel from a slow one.

```
from std.memory import unsafe_stack_allocation, AddressSpace

comptime TILE = 16

def tiled_matmul(...):
    var a_shared = unsafe_stack_allocation[
        TILE * TILE, Float32, address_space=AddressSpace.SHARED,
    ]()
    var b_shared = unsafe_stack_allocation[
        TILE * TILE, Float32, address_space=AddressSpace.SHARED,
    ]()
```

The allocation is per block, and its size must be a compile-time constant. Ask for too much and the pipeline will not create.

The classic use is tiling: each block cooperatively loads a tile of input into shared memory, then every thread in the block reads that tile many times without touching device memory again.

Barriers

Shared memory is only useful if the threads agree about when it is filled. That is what a barrier is for:

```
from max.gpu.sync import barrier

    a_shared[unsafe_offset=local] = a[unsafe_offset=global_a]   # load
    barrier()                                                    # all loaded
    ...read a_shared freely...
    barrier()                                                    # all finished
```

`barrier()` synchronises **every thread in the block** — it is CUDA's `__syncthreads()`. Memory operations before it are visible to all threads after it.

Two rules that are not negotiable:

Every thread in the block must reach every barrier. Putting one inside an `if` that only some threads take is a hang or worse, not an error message. If you need a conditional load, do the condition *inside* the guarded region and put the barrier outside it.

You need the second barrier too. Without one before the next tile is loaded, a fast thread can overwrite shared memory a slow thread is still reading.

`syncwarp(mask)` synchronises only within a SIMD group, which is cheaper and occasionally what you want.

SIMD-group operations

Threads in the same SIMD group run in lockstep and can exchange values directly — no shared memory, no barrier. This is the fastest cooperation available.

```
from std.gpu.primitives.warp import (
    shuffle_idx, shuffle_up, shuffle_down, shuffle_xor,
    reduce, lane_group_reduce,
)
```

Operation	What it does
`shuffle_idx`	Read another lane's value by absolute lane index
`shuffle_up` / `shuffle_down`	Read from a lane a fixed distance away
`shuffle_xor`	Read from the lane whose id differs by a mask — the butterfly pattern
`reduce`	Combine a value across the whole SIMD group
`lane_group_reduce`	The same across a sub-group of lanes

The idiom worth knowing is the reduction. To sum across a SIMD group, halve the distance each step:

```
    var v = my_value
    var offset = WARP_SIZE // 2
    while offset > 0:
        v += shuffle_down(v, offset)
        offset //= 2
    # lane 0 now holds the sum
```

A block-wide reduction is this, then one value per SIMD group written to shared memory, a barrier, and one more SIMD-group reduction over those.

Which memory to reach for

Scope	Reach	Cost	Use for
Registers	one thread	fastest	everything, by default
SIMD-group shuffles	32 lanes	very fast	reductions, scans, neighbour exchange
Shared memory	one block	fast	tiles, staging, block-wide cooperation
Device buffers	whole grid, and the host	slow	inputs and outputs

The usual shape of a fast kernel: read device memory once, coalesced; keep the working set in shared memory and registers; cooperate through shuffles where possible and barriers where not; write device memory once.

Coalescing

When adjacent threads read adjacent addresses, the hardware services them as one wide transaction. When they stride, it cannot. This is usually the difference between a kernel that is memory-bound at a sensible fraction of bandwidth and one that is ten times slower for no visible reason.

In practice: map the **fastest-varying thread index to the fastest-varying data index**. If your matmul reads `B[k][n]`, let `thread_idx.x` walk `n`, not `k`. The tiled matmul in this tree carries exactly that comment — *map thread x to column for coalesced access in B*.

Divergence

Threads in a SIMD group share one instruction pointer. When they take different branches, both sides execute with the inactive lanes masked off, so a two-way branch inside a SIMD group costs the sum of both sides.

Divergence across *different* SIMD groups is free. So a condition on `block_idx`, or on `global_idx / WARP_SIZE`, costs nothing; a condition on `lane_id` costs. Data-dependent branches — like the Mandelbrot escape loop — inevitably diverge, and that is fine; it is worth knowing rather than worth avoiding at all costs.

3. Running and checking

Running: JIT or build

From a CocoaMojo distribution:

```
cocoamojo --run   prog.mojo        # JIT, straight into this process
cocoamojo --build prog.mojo        # -> ./prog, a normal binary
cocoamojo --build prog.mojo -o app # -> ./app
```

Both paths run GPU kernels. `cocoamojo` is the whole interface: no `-I` flags, no `MODULAR_*` environment variables, no Bazel. The distribution ships the compiler, the Mojo runtime, the Metal device runtime (`libCocoaMojoGPU`), the packages and `cocoa.sqlite` beside one another, and the driver points the compiler at them. Binaries from `--build` carry an rpath to `lib/`, so they run anywhere on the machine without a wrapper.

A correction worth knowing

Earlier versions of this fork's documentation — including earlier versions of this guide — said that GPU code could not be JIT-run, because "the JIT cannot resolve the GPU runtime's symbols". **That was wrong**, and the mistake is worth understanding because it is a common shape.

The JIT was never the problem. `ExecutionEngineOptions::libraryPaths` feeds an ORC dynamic-library search generator, and `-Xlinker -L`/`-l` reach it. The symbols were missing for the same reason so many things here were missing: nothing exported them. Once the device runtime existed as `libCocoaMojoGPU.dylib` rather than a hidden-visibility archive, the JIT resolved them like any other library and ran GPU kernels immediately:

```
GPU: 0.413 ms   speedup: 197.96 x
exact agreement: 100.0 % ( 0 boundary-band pixels differ)
COMPUTE-SMOKE: PASS
```

All sixteen spike checks now pass through the JIT path.

The lesson generalises: *"the JIT cannot do X"* is almost always *"nothing exported the symbol X needs"*, and the two have very different fixes.

One reason to prefer a subprocess anyway

The argument for building and exec'ing does survive, in one specific form. JIT'd code runs in the **host process's address space**. A segfault is caught by `CrashRecoveryContext`, but a call to `exit()` is not, and it would take the host down with it.

That matters if you are embedding the compiler in something long-lived — an editor, say. For running your own programs from a shell it is irrelevant.

Building against the source tree

If you are working in the repository rather than from a distribution, you drive the compiler directly and supply what `cocoamojo` would have supplied:

```
mojo build --target-accelerator apple-m4 \
  -I mojo/stdlib -I max/kernels/src -I max/mojo \
  -o /tmp/prog prog.mojo
```

`--target-accelerator` is required

Through Bazel the toolchain supplies it. Compiling directly you must pass it yourself, or comptime GPU dispatch fails with *"Unknown GPU architecture"*.

Clear the kernel cache when a change seems not to take

Compiled kernels are cached, so a rebuilt compiler can still hand you yesterday's answer. When an edit appears to do nothing, suspect this first:

```
./clear_cache.sh
```

`./rebuild.sh` does it as part of a rebuild, and can assert that a marker string is genuinely present in the binary you are about to run — so "did it actually rebuild?" has an answer that is evidence rather than hope.

One Bazel trap worth knowing: **Bazel does not forward your shell environment into compile actions**, so exporting a variable does nothing. Use `--action_env=FOO=1`, which forwards it *and* changes the action key so the action re-runs — at the cost of invalidating the whole graph, so expect a full rebuild each way.

Timing

Warm up first. The first dispatch pays for pipeline creation and first-touch residency, and including it will make your kernel look several times slower than it is.

```
    # warm
    ctx.enqueue_function(f, ...); ctx.synchronize()

    var t0 = perf_counter_ns()
    ctx.enqueue_function(f, ...); ctx.synchronize()
    var t1 = perf_counter_ns()
```

`synchronize()` before reading the clock, or you are timing the enqueue rather than the work.

Launch is synchronous by default. Setting `APPLEGPU_ASYNC_LAUNCH=1` defers the wait to `synchronize()`, which is worth about +29% on a single-chain FMA kernel and +2.9% at 64 chains — most valuable when you are launch-bound and nearly irrelevant when you are not.

Checking the answer

This deserves more care than it usually gets, because the obvious check is the wrong one.

Comparing every GPU value to every CPU value for exact equality will fail on correct code. The GPU contracts a multiply and an add into a single fused operation; the CPU does them separately, rounding in between. For most data the results are identical, but wherever the computation is iteration-sensitive the two can diverge.

The Mandelbrot spike says it plainly:

> CPU and GPU agree exactly on interior and exterior pixels. In the thin chaotic band right at the set boundary, a point is so iteration-sensitive that the GPU's fused multiply-add (vs the CPU's separate mul/add) can shift its escape count — inherent to Float32 mandelbrot, not a bug.

So it measures an **agreement rate** and requires it to be very high:

```
var rate = Float64(agree) / Float64(checked) * 100.0
print("PASS" if rate > 99.0 else "FAIL")
```

The general rule is to **classify differences before claiming victory or defeat**:

Difference	Verdict
Boundary of a chaotic region, tiny population	arithmetic — expected
Last-bit differences in float output	contraction — expected
Smooth, low-iteration region	codegen bug
Large, structured, or in a flat area	codegen bug
Exactly one block's worth of values wrong	a bounds check or an index
Everything past a certain index wrong	grid too small, or a missing `if i < n`

A picture that looks right is not evidence. A picture that survives a classification filter is.

Debugging

The runtime reads a set of environment variables, and the two you will reach for first are:

Variable	Use
APPLEGPU_TRACE_LAUNCH	Trace each dispatch — confirms the launch happened and with what dimensions
APPLEGPU_TRACE_CAPS	Report the device capabilities resolved at context creation

`APPLEGPU_KEEP_AIR` keeps the intermediate GPU module for inspection, and `APPLEGPU_TRACE_BLOB`, `APPLEGPU_TRACE_PROFILE` and `APPLEGPU_AIR_TRACE_KNOBS` cover capture blobs, timing and which knobs are in effect.

Metal's own validation is worth enabling during bring-up: this fork's dispatch path passes with Metal debug and shader validation on, and a residency mistake that is silent otherwise becomes an error there.

What this hardware supports

The reference machine is an **M4 Max**, and one distinction shapes what runs.

Apple's 16×16 `simdgroup_matrix` operations — the tensor-core-class matrix paths — require `MTLGPUFamilyApple10`, which **M5 is the first part to advertise**. On an M4 those kernels compile, and the module loads, and the driver then declines the pipeline with *"supported by GPUFamily10 and later"*.

You will meet this as a runtime error mentioning Apple M5, not as a compile failure, because everything up to pipeline creation succeeds. Guards in the kernel library are spelled `compute_capability() != 5`, and this fork answers that question by asking Metal for the family rather than trusting a marketing name.

The 8×8 matrix path does work here.

An honest word on maturity

The GPU support is a **working vertical slice**, not a finished backend. A Mandelbrot runs and agrees with the CPU; 96 of 119 in-scope GPU tests pass; the failures are triaged and named. The open work is mostly numerical correctness in specific kernels, plus the M5 surface that cannot be exercised on this machine at all.

Two consequences for you. Ordinary kernels of the shape in this section — index, guard, compute, store — are well-travelled ground. The further you go towards the kernel library's specialised paths, the more likely you are to meet something that has not been validated here yet, and the more the advice is to verify numerically against a CPU reference rather than to assume.

4. Three ferns: when a GPU kernel is worth writing

Three examples in this distribution draw the same plant. `fern/` writes one to a file, `ferns/` grows a meadow of them live, and `fernwind/` makes that meadow sway. They were built in that order, and the interesting thing is not that the last one uses the GPU — it is *where* the crossing happens, and that the first two are right to stay on the CPU.

The rule this chapter argues for, ahead of the evidence:

> Move work to the GPU when it is **wide** and you are **out of time**. Width without a deadline does not need one. A deadline with narrow work cannot use one.

Both halves matter, and the three ferns separate them cleanly.

The shape of the problem

All three draw a Barnsley fern by the **chaos game**. Four affine maps, each with a probability; start anywhere, pick a map at random, apply it, plot where you land, repeat. The picture emerges from the density of visits.

```
var r = rng.next()          # pick a map by weight
...
var nx = m.a * x + m.b * y + m.e
var ny = m.c * x + m.d * y + m.f
x = nx
y = ny                      # the next point depends on this one
```

That last line is the whole difficulty. The chaos game is a **recurrence**: point *n+1* is a function of point *n*. You cannot compute the millionth point without computing the 999,999th, so a single stream **cannot be parallelised across time**, not by a GPU and not by anything. Any thought of "just put the fern on the GPU" runs into that wall immediately.

What *is* parallel is something else, and noticing the difference is the whole chapter: **independent streams**. A hundred separate chaos games, each starting from its own seed, are a hundred independent recurrences. They do not need to talk to each other. That is the parallelism — not within a fern's history, but across many short histories.

`fern/` — no deadline, so no question

Two million points, one stream, straight down the CPU, into a histogram:

```
var hits = List[UInt32](length=W * H, fill=0)
...
hits[idx] += 1
```

Then a log curve to colour (linear brightness leaves the fronds invisible beside the stem) and a PNG.

Measured: **2,000,000 points, a 720×960 PNG encoded, and an ASCII preview printed, in 44.8 ms** total.

There is nothing here for a GPU to do. Not because the work is small — 2M iterations is not nothing — but because **nobody is waiting**. The program runs once, writes a file, exits. A GPU would add device setup, a kernel launch and a readback to a job that finishes in the time it takes to blink. The right hardware for work with no deadline is whatever is simplest to write, and that is the CPU.

`fern/` is also the example that shows what a project looks like: three files — `main.mojo`, `ifs.mojo`, `png.mojo` — side by side in one folder, because `cocoamojo` follows imports from `main.mojo` and a project's files live in the project's folder.

`ferns/` — a deadline arrives, and the CPU still wins

Now a live window: a dozen ferns growing in front of you, over a procedural lawn of fourteen thousand grass blades, under a two-octave value-noise sky. There is a deadline now — 60 frames a second, 16.7 ms a frame — and the example still does everything on the CPU. That is not laziness. It is the design.

The trick is in one comment in the source:

> *The chaos-game point (x, y) is the whole growing state — the picture so far lives in the framebuffer, not here.*

Each fern keeps only its **current point**. Every frame it advances that point a few dozen steps and plots them into a framebuffer that is **never cleared**. The picture accumulates. Nothing already drawn is ever drawn again.

So the per-frame cost is not "draw twelve ferns" — it is "add about sixty points to each of twelve ferns", on the order of **a thousand points a frame**. On the measured CPU rate below that is roughly **four microseconds** of chaos game per frame, inside a 16,700 µs budget. The lawn and sky are painted once per landscape, not per frame, for the same reason.

Measured: **240 frames at 61.1 fps.**

A GPU kernel here would be strictly worse. Fourteen thousand grass blades and a thousand chaos-game steps do not fill an M4 Max's width; you would pay a dispatch and a synchronise to save four microseconds of a sixteen-millisecond budget. **Accumulation is what keeps this cheap** — and it is also exactly what forbids the next thing.

The wall: why `ferns` cannot move

Wind means the ferns change shape. Change the shape and every point already in that framebuffer is wrong. The accumulation — the thing that made `ferns` cheap — is only valid while nothing moves.

So animating the meadow is not an optimisation problem. It is a different problem: **every fern must be drawn from scratch, every frame**, at full density. The work per frame goes from about a thousand points to about seven million.

That is the crossing. Not "the GPU is faster" — the *requirement changed*.

`fernwind/` — wide work, and now out of time

The answer is the fractal-flame one: stop trying to make one stream long, and run an enormous number of short ones.

```
comptime STREAMS = 24576
comptime BURN    = 12     # unplotted, to land on the attractor
comptime ITERS   = 280    # plotted
```

24,576 threads, each running its own small chaos game: twelve iterations discarded so the point is genuinely *on* the attractor before anyone sees it, then 280 plotted. That is **6,881,280 plotted points per frame**, and at 60 fps, **413 million points a second**.

The burn-in is worth pausing on. A chaos game started at an arbitrary point takes a few iterations to converge onto the fern; plot those and you get a scatter of wrong dots leading in from nowhere. Twelve throwaway iterations per stream is the price of 24,576 independent starting points, and it is why the picture is clean.

Threads meet only in the density buffers, through atomics:

```
_ = Atomic.fetch_add(nacc.unsafe_offset(pat), UInt32(1))
_ = Atomic.fetch_add(racc.unsafe_offset(pat), cr)
_ = Atomic.fetch_add(gacc.unsafe_offset(pat), cg)
_ = Atomic.fetch_add(bacc.unsafe_offset(pat), cb)
```

A count, and the fern's colour weighted in — so where two ferns overlap they blend **by evidence rather than by draw order**, which is a thing painters' algorithms cannot do at all.

The wind is in the mathematics, not the geometry

Nothing bends a fern. Each fern's *climb map* — the affine map that carries the plant upward, chosen 85% of the time — is rotated a fraction of a degree per frame by a gusting wind field. Because that map is applied **recursively** up the plant, a uniform rotation compounds: the stem barely moves, the tips whip. The bend is a consequence of recursion, not of any code that draws a curve.

This is the payoff for redrawing from scratch. Once the maps can change per frame, the motion is free — it costs nothing beyond the redraw you were already doing. `ferns` could never have this at any price, because its picture is its history.

The measurement

The same work — one `fernwind` frame's 6,881,280 plotted points — on each processor. M4 Max, warm, three trials each:

	per frame	rate
CPU, one thread	**26.6 ms**	259 M points/s
GPU, 24,576 streams	**0.256 ms**	26,900 M points/s

104× faster on the GPU — and the number that actually decides it is the first column against a 16.7 ms frame budget. The CPU is not hopeless here; it is *close*. At 26.6 ms it would sustain about 37 fps for the chaos game alone, before compositing 655,360 pixels or painting anything else. Close, and on the wrong side of the line.

That is what makes this a good example rather than a flattering one. The GPU is not rescuing a hopeless computation. It is turning work that misses the deadline by 1.6× into work that uses 1.5% of the frame — and hands the rest of the frame back for the shade pass, the present, and the whole of AppKit.

Measured with the same source shape as the example, dispatching and awaiting only the chaos kernel. The frame rate of `fernwind` itself cannot show this: it is vsync-locked, so it reads 60.2 fps whether the kernel takes 0.26 ms or 14 ms. **A frame-rate number tells you a deadline was met, never by how much.**

What stayed on the CPU, deliberately

`fernwind` did not move everything. The lawn and the cloud sky are still painted by the CPU, once per landscape, and uploaded — because they do not change between frames, and work that does not change should not be recomputed by anyone. Only the part that genuinely changes every frame went to the GPU, and a second small kernel composites densities over that backdrop.

The design also rests on two facts that were **proved by a probe before the example was written**: that `Atomic.fetch_add` lowers through the AIR backend without losing increments, and that host writes through `map_to_host` reach the next dispatch. Both are assumptions that would have been extremely painful to discover as bugs later, and cheap to establish first.

The rule, restated

	width	deadline	verdict
`fern/`	2M sequential points	none	**CPU** — nobody is waiting
`ferns/`	~1k points/frame	16.7 ms	**CPU** — accumulation keeps it tiny
`fernwind/`	6.9M points/frame	16.7 ms	**GPU** — 26.6 ms doesn't fit

Ask, in this order:

1. **Is anything waiting?** No deadline, no reason. Write it on the CPU.
2. **Can I avoid the work instead?** `ferns` beats `fernwind` at its own game by never redrawing. The cheapest kernel is the one you didn't dispatch.
3. **Is the work actually wide?** Not "is it big" — is it *many independent things*? A chaos game is not parallel across time and never will be; it is parallel across streams. Find the axis that is independent, or there is nothing for the hardware to do.
4. **Measure both.** Not the frame rate — the work. A vsync-locked 60 fps looks identical at 0.26 ms and 14 ms.

The GPU chapter before this one shows how to write the kernel. This one is about whether to.

CocoaMojo Programmer's Guide

This guide teaches CocoaMojo in the order you meet it: install it and run a program, learn what the language actually is at this frozen version, understand the object model Mojo has on its own, then send your first message to Cocoa, get memory right, let Cocoa call back into your code, and finally assemble and read a real windowed application. A final chapter reads the fork's own IDE — the largest cocoa-mojo program there is — for the shapes that appear at scale.

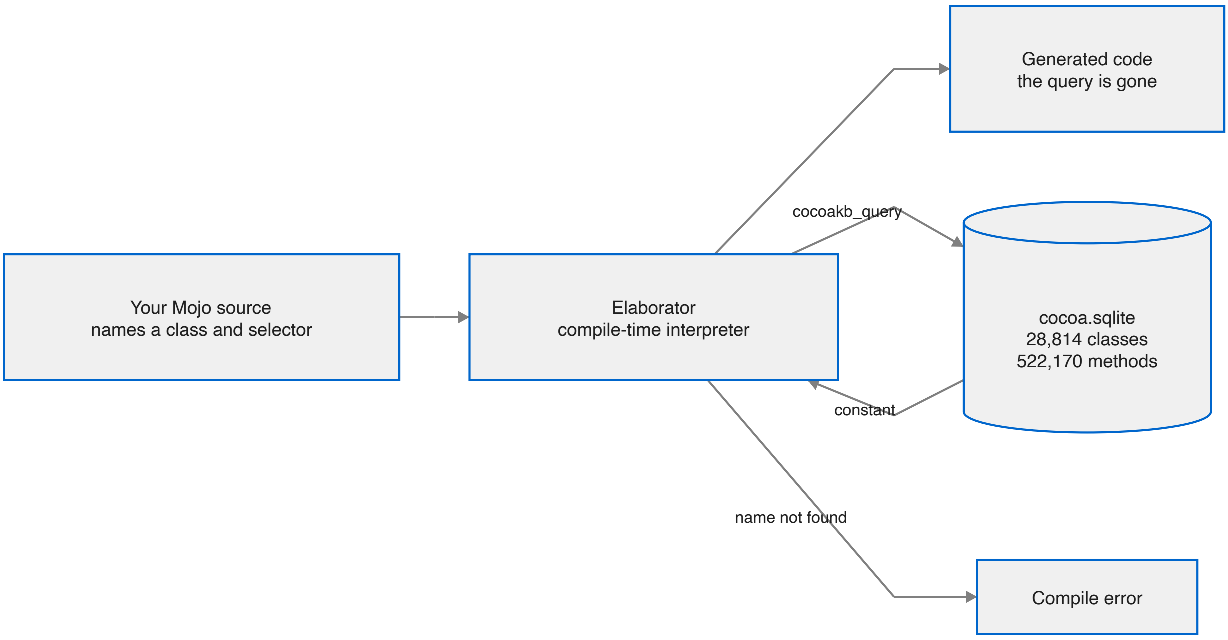
Read it in order the first time. Each chapter assumes the one before it.

When you want to look something up rather than learn it, use the Reference Manual.

What makes this different from a binding layer

Most language bridges to Cocoa work by generating declarations: a tool reads the SDK once, emits a large body of source, and from that moment the generated code and the real SDK drift apart silently.

CocoaMojo does not generate anything. The compiler holds an open connection to a database describing macOS, and resolves each fact at the point of use during elaboration.



Two consequences follow, and they are the reason the whole design exists.

The first is that **a wrong name cannot survive to run time**. There is no generated stub to be out of date, so a selector the SDK does not have has nowhere to hide.

The second is that **the compiled program has no dependency on the database**. By the time code is generated, every query has collapsed to a constant. The database is a build input in exactly the way a header file is.

1. Getting started

CocoaMojo ships as a signed, notarized disk image containing an installer and **Roast**, a Mac IDE for Mojo written in Mojo. Installing it gives you the compiler, the standard library, the language server, the debugger, the examples, and an editor to run them in.

This chapter takes you from that disk image to a running Cocoa program. If you want to build the fork from source instead, that is Appendix A; you do not need it to write anything.

What you need

Requirement	Why
Apple Silicon Mac	The fork targets `arm64` Darwin and hardcodes it deliberately
macOS 15 or later	The metadata is built from the live Objective-C runtime on your machine
Xcode 16 or later	Supplies the SDK the database is built from, and the AIR toolchain if you go near the GPU

There is an Intel build for the 2019 Mac Pro as well. Everything in this guide applies to it unchanged except the GPU chapters, which describe Apple Silicon.

Install

Open the disk image, double-click **Install Roast**, and press **Install**.

Two things happen, and the second is the slow one:

1. The toolchain is copied to `/Applications/Roast`.
2. **cocoa.sqlite is built from your Mac's SDK**, with a progress bar. It is not downloaded, because it describes *your* machine: roughly 236 MB covering 28,814 classes and 522,170 methods, read out of the live Objective-C runtime and BridgeSupport.

That second step is the whole idea of this fork. The compiler answers questions about macOS *while your program is being compiled*, and it answers them from the SDK you actually have.

If you install again later the button reads **Reinstall**, and that is also the answer to "I updated macOS" — see After a macOS update.

Where everything is

The installed toolchain:

```
/Applications/Roast/
    CocoaMojo/
        2026.08.31/          one directory per installed version
        current -> 2026.08.31
    Roast.app
```

Versions sit side by side and `current` is a symlink, so an install adds a directory and moves one link rather than overwriting what is working.

The parts you are allowed to edit live somewhere else:

```
~/Library/Application Support/Roast/
    Standard Library/stdlib      the stdlib, editable
    Examples/                    what the Examples menu opens
    IDE Source/                  Roast's own source
    Python/Environments/         per-project Python environments
```

The reason for the split is worth knowing early. An installed app's `Resources` are sealed by its code signature, so "how do I change the standard library" cannot be answered with "break the seal." Instead the first launch copies the standard library and the examples out to user space, and **everything that reads them — builds, the language server, the Examples menu, jump-to-definition — reads your copy**. The bundle keeps the pristine originals, which is what makes *Reset* a copy rather than a download.

So you can edit the standard library, and you can put it back.

Your first program

Open Roast. The **Examples** menu lists everything in the box; pick **hello** and press **⌘R**.

```
Hello from cocoa-mojo.
The first hundred integers sum to 5050
```

The console opens itself when a build starts, and **⌘0** toggles it.

That proves the toolchain. Now one that proves the *point* of it. **File ▸ New Folder**, save this as `main.mojo` inside it, and press ⌘R:

```
from std.objc import ObjCClass, ObjCObject, msg_send, autoreleasepool
from std.ffi import c_char
from std.memory import OpaquePointer


def main():
    with autoreleasepool():
        var cls = ObjCClass.lookup["NSString"]()
        var text = String("Hello from Cocoa, via Mojo")

        var s = msg_send[
            ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
        ](cls.as_object(), text.as_c_string_slice())

        var n = msg_send[Int, "NSString", "length"](s)
        print("length:", n)

        var back = msg_send[
            OpaquePointer[MutUntrackedOrigin], "NSString", "UTF8String"
        ](s)
        print("round trip:", String(unsafe_from_utf8_ptr=back.bitcast[c_char]()))
```

```
length: 26
round trip: Hello from Cocoa, via Mojo
```

Nothing in that program named a dispatch function, a type encoding, or a selector address. All of it came from the database while the program was being compiled.

A project is a folder

There is no project file and nothing to generate. A project is a folder with a `main.mojo` in it — **File ▸ Open Folder…**, then ⌘R.

Mojo has no link step: the compiler is given one file and follows its imports, so a project needs an entry point rather than a file list. Roast looks for one in this order:

1. `main.mojo` in the project root — the convention, and what the examples use
2. the file on screen, if it is in the root and declares a top-level `main`
3. the one non-test file in the root that declares a top-level `main`
4. the file on screen

Step 4 is what makes a single loose file buildable: open one file, press ⌘B, and it builds that file.

Prove the checking is real

Change `"length"` to `"lenght"` and look at it.

You do not get a runtime crash, and you do not get a silently wrong answer. You get an error naming the selector — and you get it **as you type**, because the language server asks the same database the compiler does.

That is the claim this whole design rests on: a name that does not exist cannot reach code generation. The Reference Manual lists every diagnostic that enforces it.

The command line, if you prefer one

The toolchain is not added to your `PATH`. The driver is at:

```
/Applications/Roast/CocoaMojo/current/bin/cocoamojo
```

and it is the whole interface — no `-I` flags, no `MODULAR_*` variables:

```
cocoamojo --run   hello.mojo         # compile and run
cocoamojo --build hello.mojo         # -> ./hello
cocoamojo --build hello.mojo -o app  # -> ./app
```

The distribution carries the compiler, the Mojo runtime, the Metal device runtime, the packages and `cocoa.sqlite` beside one another, and the driver wires them together. `--run` JITs; both paths work for Cocoa and for GPU code. Binaries from `--build` carry an rpath to `lib/`, so they run anywhere on the machine without a wrapper.

Add it to your `PATH` if you want it by name:

```
export PATH="/Applications/Roast/CocoaMojo/current/bin:$PATH"
```

After a macOS update

Rebuild the database: open the installer again and press **Reinstall**. An SDK update changes what the compiler should know about macOS, and nothing forces the issue — the compiler will happily keep using the old database.

`cocoakb_db_hash()` is how you check. It returns the SHA-256 of the database a given compilation consulted, so a binary can record exactly which revision it was built against.

Where to go next

Chapter 2 is not optional. This compiler rejects a good deal of Mojo that current tutorials still teach, and knowing what changed will save you more time than anything else in this guide.

If you would rather read code than prose, The examples walks through everything in the Examples menu and says what each one is for.

2. The language, as it is now

This chapter exists because most Mojo you will find written down does not compile here, for two separate reasons. Mojo itself moved and the writing did not; and this fork has since started changing the language deliberately, under the name cocoa-mojo.

Read this before you write anything substantial. Everything below was taken from the compiler's own source and standard library at the frozen commit.

This is cocoa-mojo, not upstream Mojo

The fork has stopped tracking upstream's declaration model and named itself. Where upstream removed something this port needs, it heals the language instead of working around the removal. Two keywords have come back with new, narrower meanings, and they are the first thing to learn.

The stance is recorded in the fork's own COCOA_LET_DESIGN.md: new code here is deliberately a different dialect, and migration diagnostics teach rather than promise compatibility.

`fn` is back — as the foreign-callable function

Upstream removed `fn`. This fork revived it, but not as the old strict function. Here `fn` declares a **foreign-callable** function:

- **thin** — no captured context
- **non-raising** — the C boundary has no error channel
- **C ABI** — set automatically by the keyword
- every parameter and return type must be ABI-classifiable

That is exactly the contract an Objective-C `IMP` requires, which is the point.

```
fn add_one(x: Int) -> Int:
    return x + 1

fn button_clicked(self_: P, cmd: P, sender: P):
    clicks()[] += 1
```

You do not write `abi("C")` any more. The keyword says it.

An `fn` is still an ordinary callable from Mojo — `add_one(1)` works — so the keyword costs you nothing at the call site.

`fn` in type position

`fn(...) -> R` is sugar for the old `def(...) thin abi("C") -> R`:

```
comptime IntFn = fn(Int) -> Int

def apply(f: IntFn, x: Int) -> Int:
    return f(x)
```

The standard library's IMP aliases are now spelled this way, and read as what they are:

```
comptime IMP0 = fn(P, P, /) -> None
comptime IMP1 = fn(P, P, P, /) -> None
comptime IMP0Bool = fn(P, P, /) -> Bool
comptime IMP1Bool = fn(P, P, P, /) -> Bool
comptime IMP2 = fn(P, P, P, P, /) -> None
```

What `fn` rejects

Marking one `raises` is a compile error, and the message teaches rather than just refusing:

```
'fn' declares a foreign-callable (C ABI, non-raising) function in cocoa-mojo
and may not be marked 'raises'; use 'def' for an ordinary Mojo function
```

It comes with a FixIt that replaces `fn` with `def`. `async` is rejected the same way. Old code of the `fn main() raises` shape therefore gets told what changed instead of a bare contract violation.

`let` is back — as the immutable binding

`let` declares an **immutable, scope-bound binding**.

```
let win = ObjCObject(window_addr()[])
let n = 41 + 1
```

Three things to know.

The binding is immutable; the object is not. This is Swift's rule exactly. You cannot rebind `win`, but you can send it setters all day.

It is general. The design originally proposed gating `let` on a `Retained` trait so that only reference-counted Cocoa objects could use it. When it came to implementation the opposite was chosen: `let n = 42` and `let win = NSWindow(...)` both mean "immutable binding", which is one rule instead of two.

Ownership rides the value, not the keyword. For a Cocoa object held in an `ObjCRef`, `let` gives you the same +1-at-bind and release-at-scope-exit you already had with `var`; the ARC behaviour lives in `ObjCRef`. `let` adds only the immutability of the binding.

What `let` rejects

Reassignment is a compile error. And `let` is a **function-body** form only — there is no field or file-scope `let`:

```
'let' declares an immutable binding inside a function body; use 'var' for a
field or module value
```

When to reach for it

The fork's own conversion is a good guide: sixteen never-reassigned Cocoa bindings became `let` in one file alone. If you never rebind it, say so.

`alias` is deprecated, not removed

`alias` still parses, with a warning:

```
'alias' is deprecated; use 'comptime'
```

Use `comptime`. `alias` is gone from the standard library's own code.

`comptime` does the compile-time work

Argument conventions

There is no `inout`, no `borrowed`, and no `owned`. The conventions are:

Most of them are *contextual* — soft identifiers rather than reserved words — so `mut`, `imm`, `out` and `deinit` are all still available as ordinary variable names. Only `var` is a reserved keyword.

Convention	Meaning
(none)	Borrowed immutably. The default; you do not write it.
`imm`	Borrowed immutably, said out loud. `read` is the deprecated spelling.
`mut`	Borrowed mutably.
`out`	An uninitialised result the function must initialise. Used for `__init__`.
`var`	Taken by value; the callee owns it.
`deinit`	Consumes the value and ends its lifetime. Used for `__deinit__` and for consuming methods.

In practice:

```
def __init__(out self, *, adopt: ObjCObject):
    self._obj = adopt

def _add(mut self, selector: StaticString, encoding: String, imp: P):
    ...

def register(deinit self) -> ObjCClass:
    ...

def __iter__(var self) -> Self.IteratorOwnedType:
    ...
```

The transfer sigil `^` moves a value into a consuming position:

```
var cls = b^.register()      # b is consumed here
var delegate = new_instance(cls)
```

Forgetting `^` on a `deinit` method is one of the more common early errors.

Origins, and how they are spelled

References carry an origin parameter. The names were renamed twice and the old spellings have now been removed, so a tutorial written a few months ago will use identifiers that no longer exist.

Use this	Not this
`ImmOrigin`	`ImmutOrigin`
`ImmUnsafeAnyOrigin`	`ImmutUnsafeAnyOrigin`
`ImmStaticOrigin`	`StaticConstantOrigin`
`UntrackedOrigin`	`ExternalOrigin`
`MutUntrackedOrigin`	`MutExternalOrigin`
`ImmUntrackedOrigin`	`ImmutUntrackedOrigin`, `ImmutExternalOrigin`

`MutUntrackedOrigin` is the one you will type constantly in Cocoa code, because every pointer that crosses into Objective-C is outside Mojo's lifetime tracking by definition. Most CocoaMojo files start by shortening it:

```
comptime P = OpaquePointer[MutUntrackedOrigin]
```

Pointers

`Pointer` is the type. `UnsafePointer` still exists but is deprecated in favour of it, and the pre-unification aliases — `MutUnsafePointer`, `ImmUnsafePointer`, `ImmutOpaquePointer`, `OptionalUnsafePointer` and the rest — have been removed outright.

```
var p = Pointer[UInt8, MutUntrackedOrigin](unsafe_from_address=addr)
var q = OpaquePointer[MutUntrackedOrigin](unsafe_from_address=addr)
```

The raw memory functions were renamed to carry an `unsafe_` prefix, and the old names are gone: use `unsafe_memcpy`, `unsafe_memset`, `unsafe_memset_zero`, `unsafe_destroy_n`. Likewise the `Pointer` methods are now `unsafe_deinit_pointee()`, `unsafe_write()`, `unsafe_as_noalias()`.

Function types, `thin`, and `abi("C")`

`fn` is the spelling you want for a foreign-callable function, but the longhand still exists and you will meet it in older code and in diagnostics.

```
comptime IMP1 = fn(P, P, P, /) -> None          # cocoa-mojo
comptime IMP1 = def(P, P, P, /) thin abi("C") -> None   # the longhand it sugars
```

`/` ends the positional-only parameters, `thin` means the function carries no captured context, and `abi("C")` gives it the C calling convention. You need all three for a Cocoa callback, because the Objective-C runtime calls a bare C function pointer with no closure environment — which is precisely why `fn` bundles them into one keyword.

A `thin` function type can also carry trailing `where` clauses constraining the parameters it declares, so a generic algorithm can state what it requires of a function handed to it rather than restating the constraint at every binding site.

One consequence worth knowing: **a thin fn is exactly a global block.** An Objective-C block with no captures is `_NSConcreteGlobalBlock` with no copy/dispose helpers, so the standard library can build a block around any `fn` at no cost. That is how block-only Cocoa APIs are reachable today — see Concurrency and blocks.

Parameters and generics

Parameters go in square brackets and are resolved at compile time. Arguments go in parentheses.

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R:
    ...
```

That signature shows most of what you need: a type parameter, string parameters, a defaulted `Bool` parameter, a variadic type-parameter pack, and a value pack `*args: *Ts` bound to it.

Overloading works on parameter lists as well as argument lists, which is how `ObjCClassBuilder.add_method` accepts five different IMP shapes under one name.

Structs and traits

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64
```

`@fieldwise_init` synthesises the memberwise initialiser. Conformances go in the parentheses. The ones you will meet most in Cocoa code are `Copyable`, `Movable`, `AnyType` and `TrivialRegisterPassable`.

Two more renames that will bite: `ImplicitlyDestructible` and `ImplicitlyDeletable` are now `Deinitable`, and `@explicit_destroy` is no longer required, because implicit deletion is the default.

Destructors are `__deinit__`, taking `deinit self`:

```
def __deinit__(deinit self):
    if not self._obj.is_nil():
        ...
```

Other removals you are likely to trip over

Gone	Use instead
`InlineArray`	`Array`
`SIMD.size`, `Array.size`, `SIMDSize`	`length`, `SIMDLength`
`String.as_string_slice()`	`StringSpan(my_string)`
`as_immutable()`, `get_immutable()`	`as_imm()`
`ImmutSpan`	`ImmSpan`
`ConditionalType`	the ternary `T if cond else U`
`trait_downcast()`	a `where` clause or `comptime assert` with `conforms_to(...)`
`.mojopkg` files	`.mojoc` files
`memcmp`	`unsafe_memcmp`
`List.steal_data()`	`unsafe_take_allocation()`
`OwnedPointer.take()`	`into_inner()`
`Variant.take()`	`unwrap()`
`std.gpu.profiler` / `ProfileBlock`	`perf_counter_ns()`, or a real GPU profiler

The module system also tightened. Importing same-named functions from different modules to build one overload set is now an error, and intra-package access without an explicit `import` is an error. Both had deprecation periods that have now expired.

`async` on an `fn`

`fn` cannot be `async`; use `def`. The rest of the async story is unchanged from upstream, and nothing in CocoaMojo depends on it.

A note on `async`

`AnyCoroutine`, `Coroutine` and `RaisingCoroutine` are no longer in the prelude and the module defining them is private. `async def` still works and the compiler still synthesises those types — you simply cannot name them. Mojo's async support is unfinished and nothing in CocoaMojo depends on it.

What you can rely on

The point of the freeze is that this list will not move under you. Code written against this chapter keeps compiling for as long as you keep the compiler. That is the trade the fork makes: no upstream fixes and no new features, in exchange for a language that stays still long enough to build something on.

3. Mojo's own object model

Before any Cocoa appears, it is worth being clear about what Mojo gives you on its own — because it is not what most object-oriented experience will lead you to expect, and because the shape of the Cocoa layer is a direct response to the gap.

Mojo has no classes of its own. It has structs with methods, traits, and generics, and everything is resolved at compile time. There is no inheritance, no vtable, no runtime object graph, and no dynamic dispatch anywhere in the language.

That is not a deficiency to work around. Most of what you want from objects is here; what is missing is precisely the part Objective-C is unusually good at, which is why the two compose so well.

`class` means something specific here

In this fork `class` is not a Mojo class. It declares an **Objective-C** class — a real one, registered with the ObjC runtime, with dynamic dispatch and inheritance and everything else Mojo does not have.

That is the division of labour in one keyword. `struct` is Mojo's value type, resolved statically; `class` is Cocoa's reference type, resolved by the runtime. They are different animals and the language now says so.

This chapter is about `struct`, because that is what Mojo gives you on its own and what everything else is built from. Chapter 6 is about `class`.

Structs

A struct is a value type with a fixed layout known at compile time.

```
struct Counter:
    var count: Int
    var label: String

    def __init__(out self, label: String):
        self.count = 0
        self.label = label

    def increment(mut self):
        self.count += 1

    def describe(self) -> String:
        return self.label + ": " + String(self.count)
```

Fields are declared with `var`. Methods are ordinary `def`s whose first parameter is `self`, and the argument convention on `self` says what the method does:

First parameter	Meaning
`self`	Reads the value. The default.
`mut self`	Mutates it in place.

`var self`	Takes it by value; the method owns it.
`out self`	Initialises it. Used by `__init__`.
`deinit self`	Consumes it, ending its lifetime.

`Self` names the enclosing type, which matters in generic code and in trait requirements.

Static methods take no `self`:

```
    @staticmethod
    def zero() -> Self:
        return Counter("")
```

`@fieldwise_init`

When the initialiser would just assign each field in order, ask for it:

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64

var p = CGPoint(3.0, 4.0)
```

This is the shape almost every Cocoa geometry struct takes.

The lifecycle

Here Mojo diverges sharply from older writing about it, and from most other languages. There are four operations, and **three of them are spelled as __init__ overloads**:

```
struct Buffer(Copyable, Movable, Deinitable):
    var data: Pointer[UInt8, MutUntrackedOrigin]
    var size: Int

    def __init__(out self, size: Int):              # construct
        ...

    def __init__(out self, *, copy: Self):          # copy
        ...

    def __init__(out self, *, deinit move: Self):   # move
        ...

    def __deinit__(deinit self):                    # destroy
        ...
```

If you have seen `__copyinit__` and `__moveinit__`, those names are gone. The keyword-only `copy:` and `deinit move:` arguments replace them, and there are zero uses of the old spellings left in the standard library.

Copying is explicit at the call site too:

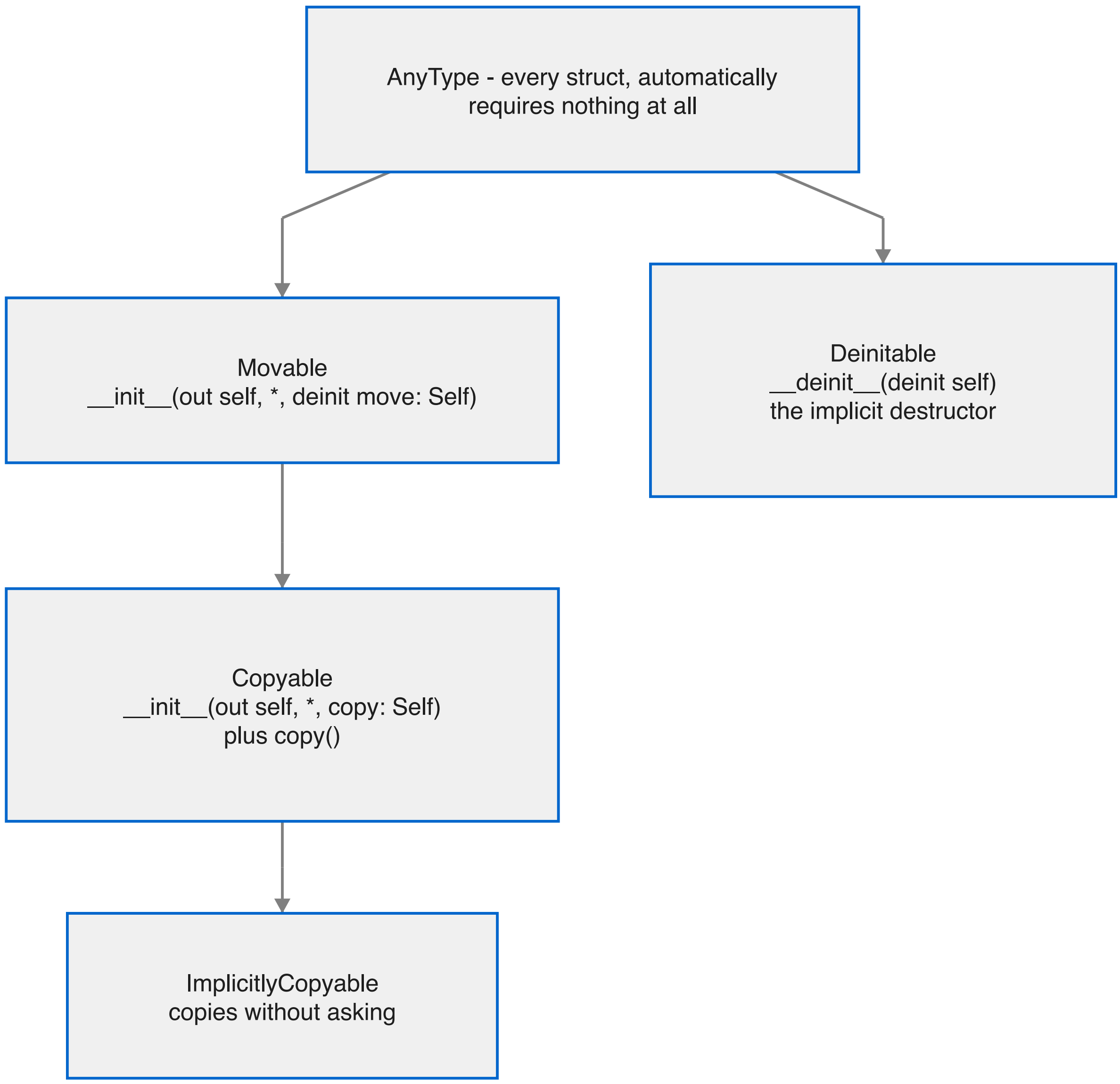
```
var b = a.copy()
```

`copy()` is a convenience for `Self(copy=self)` provided by the `Copyable` trait, and **overriding it is not allowed** — the trait says so directly. You customise the constructor, not the convenience.

Moving uses the transfer sigil:

```
var b = a^
```

The four traits everything rests on



AnyType is the floor. Every struct conforms to it automatically, and it requires nothing — *not even a destructor*. That is the unusual part, and the next section is about why.

Movable requires the move constructor. It also carries a compile-time flag, `__move_ctor_is_trivial`, that the compiler usually generates: true when the value can be moved by copying its bits with no side effects.

Copyable(Movable) requires the copy constructor and inherits movability. Copying is explicit by default.

ImplicitlyCopyable(Copyable) opts a type into being copied without an explicit `.copy()`.

Deinitable is the implicit destructor. Every type gets it by default.

Linear types, and why `AnyType` requires nothing

Most languages with strong lifetimes require every type to provide at least a trivial destructor, so the compiler can destroy a value whenever it decides the value is dead. Mojo's floor is lower than that on purpose.

A type may conform to `AnyType` but **not** to `Deinitable`. Such a type is called *linear*, and it has no implicitly-callable destructor at all. The compiler will not quietly clean it up; if you let one go out of scope without consuming it, that is a **compile error**.

```
struct MustBeCommitted(Deinitable where False):
    ...
    def commit(deinit self): ...
    def roll_back(deinit self): ...
```

The `Deinitable where False` conformance is how a type opts out. The effect is that the user must *choose* — commit or roll back — and cannot drift into neither by forgetting.

This is a real tool, not a curiosity. A linear type is a guard that some explicit action must happen later, enforced by the compiler rather than by review. You will not need it often; when you do, nothing else substitutes.

Traits

A trait declares requirements. A body of `...` means "conforming types must provide this".

```
trait Equatable:
    def __eq__(self, rhs: Self) -> Bool: ...
```

Three things traits can do that make them carry most of the weight classes would.

They inherit. `trait Comparable(Equatable)` requires everything `Equatable` does, plus its own.

They carry default implementations. A trait method with a real body is a default, and conforming types get it free:

```
trait Comparable(Equatable):
    def __lt__(self, rhs: Self) -> Bool: ...

    @always_inline
    def __gt__(self, rhs: Self) -> Bool:
```

```
        return rhs < self
```

Implement `__lt__` and `__eq__`; get `__gt__`, `__le__` and `__ge__` for nothing. This is where inherited behaviour lives in Mojo.

They carry associated types. A `comptime` member constrained by a trait:

```
trait IterableOwned:
    comptime IteratorOwnedType: Iterator

    def __iter__(var self) -> Self.IteratorOwnedType: ...
```

The conforming type chooses what `IteratorOwnedType` is, and refers to it as `Self.IteratorOwnedType`.

Composition

Traits combine with `&`, anywhere a trait is expected:

```
def once[T: Movable & Deinitable, //](...):
```

Conditional conformance

A generic type can conform to a trait *only when its parameter does*. This is the most expressive thing in the system, and `Optional` uses it heavily:

```
struct Optional[T: AnyType](
    Boolable,
    Copyable where conforms_to(T, Copyable),
    Deinitable where conforms_to(T, Deinitable),
    Equatable where conforms_to(T, Equatable),
    Movable where conforms_to(T, Movable),
    ...
):
```

`Optional[Int]` is copyable because `Int` is. `Optional[SomeLinearType]` is not, and the compiler knows without being told twice.

Generics

Struct and function parameters go in square brackets and are compile-time values:

```
struct Optional[T: AnyType]:
    ...

def largest[T: Comparable](a: T, b: T) -> T:
    return a if b < a else b
```

`Some[Trait]`

When a parameter exists only to name the argument's type, `Some` says so more briefly:

```
def foo[T: Intable, //](x: T) -> Int:   # the long way
def foo(x: Some[Intable]) -> Int:       # the same thing
```

The `//` marks parameters that are *inferred only* — never written at the call site.

Some[Trait] is not an existential. It is an alias that expands to an inferred generic parameter, so the call is still resolved and specialised at compile time. There is no boxing, no vtable and no dynamic dispatch hiding in it. `Some[Writer]` in a signature means "some specific type that writes, chosen at compile time", not "any writer, decided at run time".

That distinction is the whole difference between Mojo's object model and Objective-C's, and it is worth holding on to for the next chapter.

Operators

The dunder protocol will be familiar from Python. This compiler's standard library defines and dispatches:

Group	Methods
Arithmetic	`__add__` `__sub__` `__mul__` `__truediv__` `__floordiv__` `__mod__` `__pow__`
Unary	`__neg__` `__pos__` `__abs__` `__invert__`
Bitwise	`__and__` `__or__` `__xor__` `__lshift__` `__rshift__`
Comparison	`__eq__` `__ne__` `__lt__` `__le__` `__gt__` `__ge__`
Container	`__getitem__` `__setitem__` `__len__` `__contains__` `__iter__` `__next__`
Conversion	`__int__` `__float__` `__bool__` `__str__` `__hash__`
Call and scope	`__call__` `__enter__` `__exit__`

`__enter__` and `__exit__` are what make `with autoreleasepool():` work — a context manager is just a struct with those two methods, which is exactly how `autoreleasepool` is built.

Text formatting goes through the `Writable` trait and a `write_to` method rather than only `__str__`.

What this means for Cocoa

Lay the two models side by side and the division of labour is obvious.

	Mojo	Objective-C
Keyword here	`struct`	`class`
Unit	value semantics	reference semantics

Layout	fixed at compile time	object header plus ivars, discoverable at run time
Inheritance	none	single, deep, pervasive
Dispatch	static, always	dynamic, always — `objc_msgSend`
Polymorphism	traits and generics, resolved at compile time	messages, resolved at run time
Lifetime	scope-based, compiler-enforced	reference counting
Adding a method later	impossible	routine — categories, runtime class building

Mojo has no runtime object system, and Cocoa is nothing *but* a runtime object system. So CocoaMojo does not try to make Mojo object-oriented. It does two narrower things:

It wraps Objective-C references in Mojo structs. `ObjCRef` is an ordinary struct with a `__deinit__` that calls `objc_release`. Cocoa's reference counting becomes Mojo's scope-based lifetime, with no new machinery on either side — the next chapter but one is entirely about this.

It builds real Objective-C classes. When Cocoa needs to send your code a message, a `class` declaration becomes a genuine registered ObjC class whose method implementations are Mojo. The dynamic dispatch is real; it just lands on static Mojo code.

Everything in the rest of this guide is one of those two moves. Knowing which one you are looking at makes the rest considerably easier to read.

4. Calling Cocoa

Every Objective-C method call is one C function call:

```
objc_msgSend(id self, SEL op, ...)
```

Binding that is not hard because of the call. It is hard because of everything the call depends on — does this class have this selector, what does it return, which register file does each argument travel in — and all of that lives in the SDK, where it is traditionally transcribed by hand, once, wrongly.

`msg_send` gets those facts from the database while your program compiles.

First: load the framework

Foundation arrives free — something in every process drags it in. **AppKit does not**, unless the binary was linked against it. In a JIT-run program (`mojo run`) nothing was, so `objc_getClass("NSApplication")` returns nil and every message to it silently no-ops. The app "runs", exits without a window, and produces no diagnostic anywhere.

That failure shape cost the fork real time. Call this first in anything windowed:

```
if not load_framework["AppKit"]():
    raise Error("could not load AppKit")
```

It is a `dlopen` with `RTLD_NOW`, idempotent and cheap after the first call.

Looking up a class

```
var cls = ObjCClass.lookup["NSString"]()
if cls.is_nil():
    print("NSString not registered — is Foundation linked?")
```

The name is a *parameter*, in square brackets, because it is resolved at compile time. `is_nil()` is worth checking once at startup: a nil class means the framework was never loaded, and every subsequent message will silently do nothing.

`ObjCClass` and `ObjCObject` are distinct types even though both are pointers at the C ABI, so you cannot pass one where the other is expected. Convert deliberately:

```
var as_obj = cls.as_object()
```

Sending a message

```
var n = msg_send[Int, "NSString", "length"](s)
```

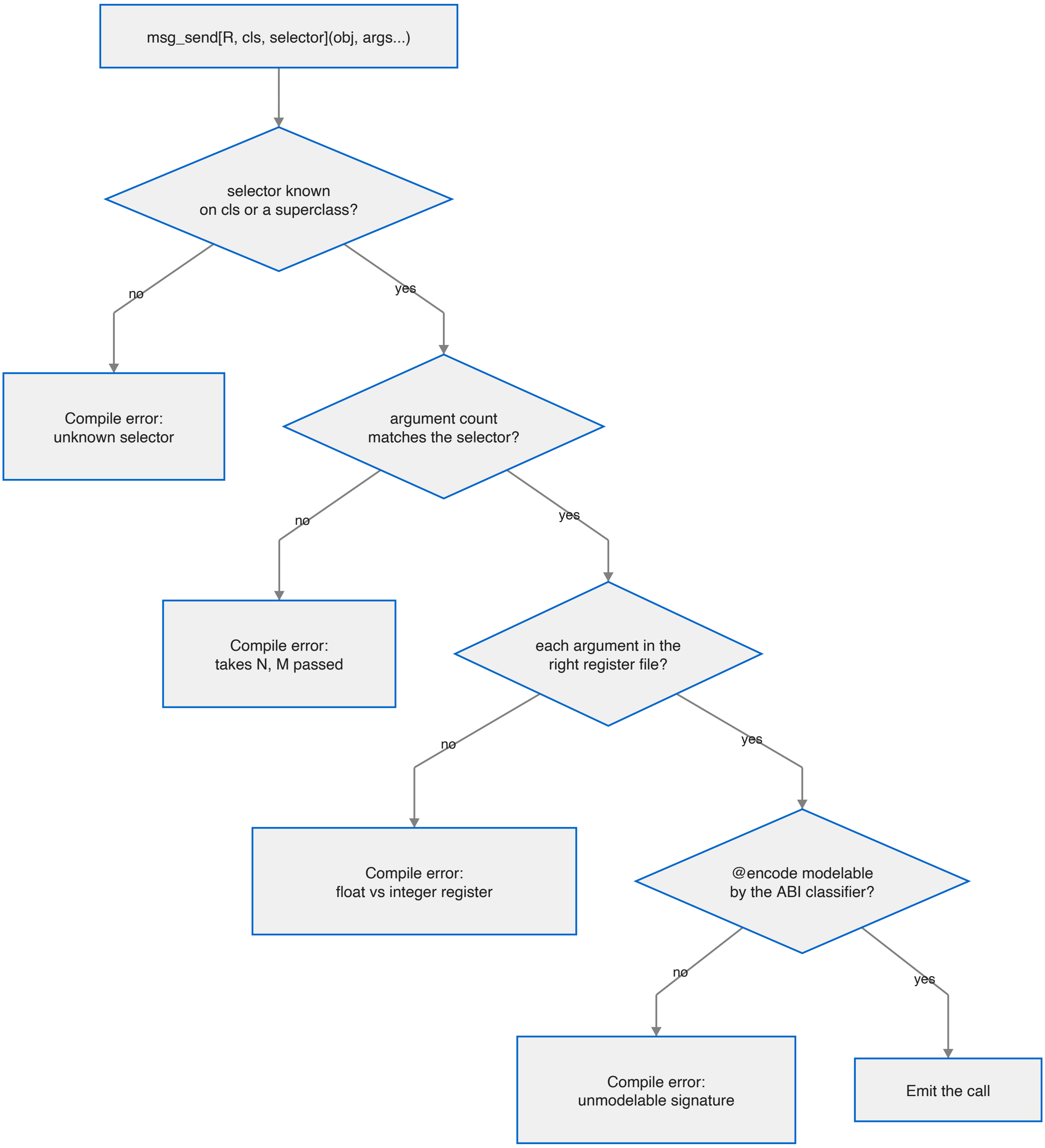
Read the parameters left to right: the **return type**, the **class the selector is looked up on**, and the **selector**. Then the receiver in parentheses.

For a class method, add `is_class=True`:

```
var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

Arguments follow the receiver. The colons in the selector tell you how many to expect — `stringWithUTF8String:` takes one, `length` takes none — and the compiler checks that you passed the right number.

What gets checked, and when



Four checks, all before code generation. Taking them in turn:

The selector must exist. The lookup walks the superclass chain in the metadata, so asking `NSMutableString` about `length` correctly finds `NSString`'s definition. A misspelling has nowhere to hide.

The argument count must match. This is the check that saves you from the worst class of bug. Call `characterAtIndex:` with no index and the callee reads whatever happened to be in the argument register — a plausible-looking number, a crash much later, or nothing at all. Here it is a compile error, and the fork ships it as a must-fail test.

Each argument must be in the right register file. On AAPCS64 a scalar float travels in `v0`–`v7` and everything else in the general-purpose registers. Passing an `Int` where the ABI wants a `Float64` is not a conversion; it is reading the wrong register. `msg_send` flags the cases it is certain about — a scalar float against an integer class, and the reverse — and stays quiet about structs and unknowns so there are no false positives.

The signature must be modelable. If the ABI classifier cannot make sense of a method's `@encode` string it answers `"?"`, and that is a compile error rather than a guess. This is the check that matters least often and matters most when it fires.

Return types

The return type parameter is what the C ABI sees, so pick the Mojo type that matches the Objective-C one:

Objective-C	Mojo return parameter
`id`, any object	`ObjCObject`
`NSUInteger`, `NSInteger`	`Int`
`BOOL`	`Bool`
`unichar`	`UInt16`
`double`, `CGFloat`	`Float64`
`const char *`	`OpaquePointer[MutUntrackedOrigin]`
`void`	assign to _ and ignore

A `void` method still returns something as far as Mojo is concerned, so the idiom is:

```
_ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](app, delegate.ptr())
```

You will write `_ =` constantly. It is not a wart; it is the compiler refusing to let you silently discard a value.

Struct returns, and why arm64 makes them boring

On x86-64, a method returning a large struct must go through a completely different entry point, `objc_msgSend_stret`, with a hidden buffer pointer in `rdi` and the receiver shifted to `rsi`. Getting that wrong corrupts the stack.

On arm64 there is no `objc_msgSend_stret` and no `objc_msgSend_fpret`. They do not exist in this machine's libobjc, because AAPCS64 does not need them: a small aggregate comes back in `x0`–`x1`, a homogeneous float aggregate in `v0`–`v3`, and anything larger through the `x8` indirect-result register. All of that happens through the ordinary send.

So `cocoakb_msgsend_variant` always answers `"objc_msgSend"` here. The query is kept anyway, because its *other* job survives: an unmodelable signature still answers `"?"` and still fails the build.

A concrete case worth internalising: `-[NSValue rectValue]` returns a `CGRect`, which is 32 bytes. On x86-64 that is a `_stret` call. Here it classifies as `h4` — a homogeneous float aggregate of four doubles — and comes back in `v0`–`v3` from a plain `objc_msgSend`.

Protocol-typed receivers: `send`

`msg_send` needs a class name to look the selector up on. Sometimes you do not have one. Every Metal object is declared as a protocol — `id<MTLDevice>`, `id<MTLTexture>` — and so is every Cocoa delegate. The concrete class is a private implementation detail you cannot name.

For those, use `send`, which is keyed by selector alone:

```
var tex = send[ObjCObject, "newTextureWithDescriptor:"](device, desc.ptr())
```

The dispatch stub and the argument classes come from any class in the database that implements the selector, which is consistent across implementors. You keep the argument-count check and the register-file check. The only thing you give up is verification that this particular receiver responds to it.

The trade is explicit, and the error when a selector is implemented by nothing at all is clear:

```
std.objc: no class in the metadata implements selector 'newTextureWithDesc:',
so its dispatch ABI is unknown. Check the selector spelling.
```

Selectors

`sel[...]` registers a selector and caches it:

```
var s = sel["applicationDidFinishLaunching:"]()
```

The resolved `SEL` is cached in a per-selector global slot, deduplicated by name in the KGEN lowering, so every call site for a given selector shares one slot. After the first send, resolving a selector is a load and a branch-predicted-away null check — not a hash lookup and not a runtime registry call.

You rarely call `sel` directly; `msg_send` does it for you. You need it when talking to `class_addMethod` or `respondsToSelector:`.

Strings

`NSString` is the type you will convert most, and there are three levels of convenience.

The raw send, when you want to see the machinery:

```
var s = msg_send[
    ObjCObject, "NSString", "stringWithUTF8String:", is_class=True
](cls.as_object(), text.as_c_string_slice())
```

`nsstring`, for handing an autoreleased string to an AppKit setter that will retain it:

```
_ = msg_send[ObjCObject, "NSWindow", "setTitle:"](win, nsstring("Life").ptr())
```

And `NSString`, a leak-safe owning wrapper, when the string outlives the statement:

```
var greeting = NSString("Hello")
print(greeting.length())
print(greeting.to_string())
```

Going the other way, `ns_to_string` is the direction that used to be missing:

```
var text = ns_to_string(some_nsstring)
```

It copies out as UTF-8, so the result does not depend on the pool that owns the `NSString`. Underneath it is still `-UTF8String`, which you can call yourself when you want the raw pointer:

```
var p = msg_send[
    OpaquePointer[MutUntrackedOrigin], "NSString", "UTF8String"
](s)
var text = String(unsafe_from_utf8_ptr=p.bitcast[c_char]())
```

Errors: `NSError` becomes `raises`

Cocoa's error convention predates the exceptions it never adopted. A fallible method takes a trailing `error:` out-parameter (`NSError **`) and signals failure **through its return value** — nil for object returns, `NO` for `BOOL` returns. The error object is only meaningful when the return value says failure.

Checking the out-parameter instead of the return value is the classic Cocoa bug. Ignoring it throws the diagnosis away — which is exactly what this codebase used to do, with an `err_out()` sink that discarded every message.

Two wrappers convert the convention to Mojo's:

```
# Object convention: nil means failure.
var contents = msg_send_raising[
    "NSString", "stringWithContentsOfFile:encoding:error:", is_class=True
](ns_string_cls, path.ptr(), Int(4))

# BOOL convention: NO means failure. Returns nothing.
msg_send_raising_check[
    "NSString", "writeToFile:atomically:encoding:error:"
](nsstring("round trip"), path.ptr(), Bool(True), Int(4))
```

Do not pass the error argument. The wrapper creates the stack slot and appends it for you. Pass the message arguments without it.

On failure you get a Mojo `Error` whose message carries Cocoa's own diagnosis — the `localizedDescription`, plus domain and code:

```
stringWithContentsOfFile:encoding:error:: The file "really-not-here.txt"
couldn't be opened because there is no such file. (NSCocoaErrorDomain 260)
```

Everything `msg_send` checks is still checked. The selector must exist, the register-file classes are verified, and the argument count includes the error slot the wrapper supplies — all at compile time.

Two limits worth knowing. The wrappers are declared at explicit arities up to five leading arguments, because Mojo permits nothing after an unpacked `*args` so the slot cannot be appended to a forwarded pack. That covers the SDK's convention comfortably, and a new arity is four lines. And if an API returns failure without writing an error, you get a message saying so rather than a crash.

Enum constants

Cocoa's named constants come from BridgeSupport, resolved at compile time:

```
comptime titled = cocoakb_enum_value["NSWindowStyleMaskTitled"]()
comptime utf8 = cocoakb_enum_value["NSUTF8StringEncoding"]()   # 4
```

Use these instead of typing the numbers. The one place you will be tempted to cheat is style masks, where `15` is quicker to write than four lookups — the example applications do exactly that, and it is the kind of shortcut that survives right up until Apple renumbers something.

Extern object constants

Constants like `NSForegroundColorAttributeName` are not values the metadata can hand over at compile time. They are globals whose address the linker resolves. `extern_object` takes a link-time reference to the data symbol and loads the pointer out of it:

```
var key = extern_object["NSForegroundColorAttributeName"]()
```

Struct layouts

Declare the struct in Mojo, then assert that your declaration agrees with the SDK:

```
@fieldwise_init
struct CGPoint(Copyable, Movable):
    var x: Float64
    var y: Float64

@fieldwise_init
struct CGSize(Copyable, Movable):
    var width: Float64
    var height: Float64

@fieldwise_init
struct CGRect(Copyable, Movable):
    var origin: CGPoint
    var size: CGSize

comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()
comptime assert align_of[CGRect]() == cocoakb_struct_align["CGRect"]()
```

```
comptime assert cocoakb_field_offset["CGRect", "size"]() == 16
```

This is the part people skip, and it is the part that pays. A wrong field offset does not fail loudly — it reads a filename out of the middle of a timestamp. Most sample code you will find online quotes 32-bit offsets; `WIN32_FIND_DATAW`'s equivalent trap exists on every platform. Assert, and the build tells you.

Once the assertions pass you can pass the struct by value straight through a send, and the C ABI does the register allocation:

```
win = msg_send[
    ObjCObject, "NSWindow", "initWithContentRect:styleMask:backing:defer:"
](
    win,
    CGRect(CGPoint(100.0, 100.0), CGSize(1080.0, 720.0)),
    Int(15),
    Int(2),
    Bool(False),
)
```

5. Ownership and memory

Cocoa manages memory by reference counting. An object is created at +1, every `retain` adds one, every `release` takes one away, and at zero it is deallocated. Autorelease pools defer a release to the end of a scope.

Mojo has value semantics and deterministic destruction. CocoaMojo binds the two so that the Objective-C reference count follows the lifetime of the Mojo value holding it — which means **you opt out of correct memory management, never into it**.

`ObjCRef` owns a +1

```
struct ObjCRef(Movable):
    ...
```

An `ObjCRef` holds one owned reference. It releases when the Mojo value is destroyed. There are two ways to make one, and picking the right one is the whole skill.

```
var owned = ObjCRef(adopt=obj)    # you already own a +1
var shared = ObjCRef(retain=obj)  # you only borrowed it; add a +1
```

The rule that decides which is the oldest rule in Cocoa, and it is mechanical:

Selector you called	You own the result?	Use
`alloc`, `new`, `copy`, `mutableCopy`	yes, +1	`adopt=`
anything else	no, borrowed	`retain=`

Get this backwards in the `adopt` direction and you over-release, which crashes somewhere unrelated. Get it backwards in the `retain` direction and you leak.

The classic +1 chain

```
def make_array() -> ObjCRef:
    var cls = ObjCClass.lookup["NSMutableArray"]()
    var allocated = msg_send[
        ObjCObject, "NSMutableArray", "alloc", is_class=True
    ](cls.as_object())
    var inited = msg_send[ObjCObject, "NSObject", "init"](allocated)
    return ObjCRef(adopt=inited)
```

`alloc` returns +1, `init` consumes and returns it, and `adopt=` takes over. The caller cannot forget to release, because there is nothing to forget: the `ObjCRef` releases when it dies.

The fork's ownership test cycles a million of these and watches memory stay flat.

Copying is an explicit retain

`ObjCRef` is `Movable` but not implicitly `Copyable`. Sharing an Objective-C object means retaining it, and the design makes that visible:

```
var a = make_array()
var b = a.copy()     # a visible +1
```

If you want to move rather than share, use the transfer sigil:

```
var b = a^
```

Autorelease pools

```
with autoreleasepool():
    var s = make_autoreleased()
    print("live inside the pool:", not s.is_nil())
# pool drained here
```

`autoreleasepool` pushes a pool on entry and pops it on exit. It guards against a double pop internally, because popping a token twice corrupts the pool page and produces a hard crash a long way from the cause.

Wrap `main` in one. Wrap any loop that creates many temporary Cocoa objects in one. Do **not** wrap an AppKit callback in one — AppKit's own event dispatch already has a pool active, and every object you read from an event is autoreleased by the caller.

Returning an object without leaking

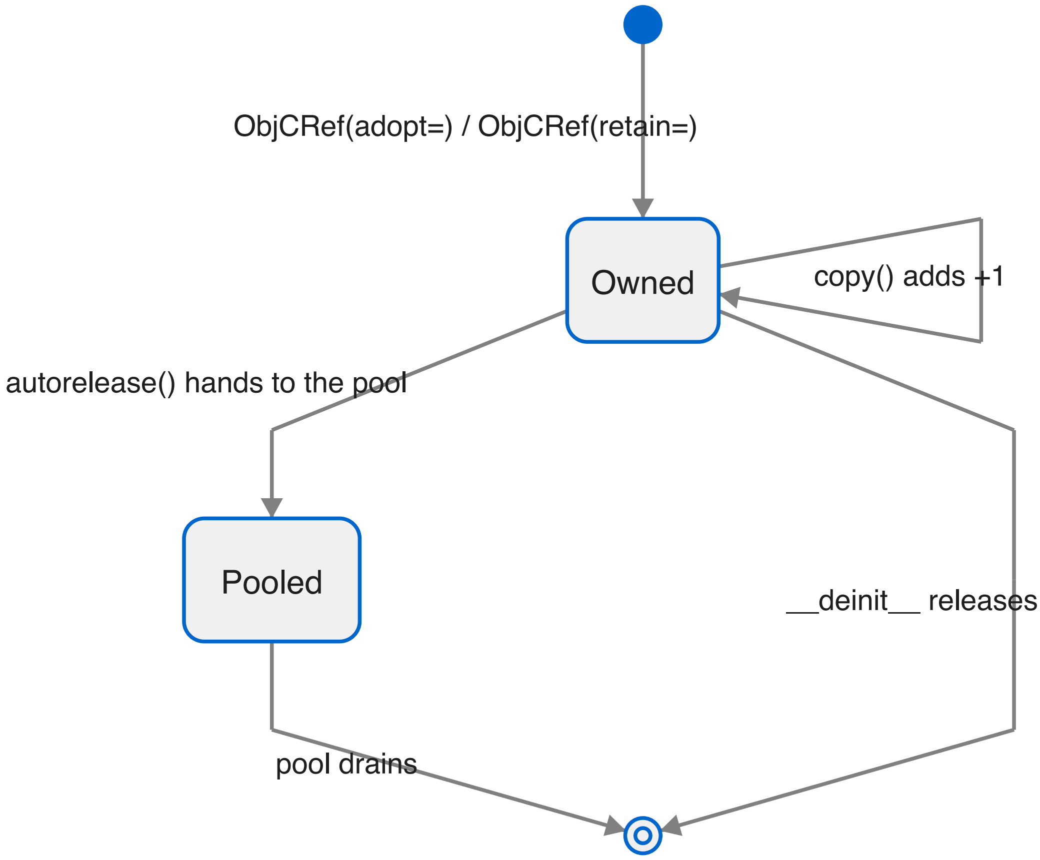
Sometimes you want to hand an object back without making the caller own it — exactly what an Objective-C method returning an autoreleased object does. `autorelease()` consumes the `ObjCRef` and gives the object to the current pool:

```
def make_autoreleased() -> ObjCObject:
    var owned = make_array()
    return owned^.autorelease()
```

Note the `^`. `autorelease` takes `deinit self`, so the reference is consumed; without the sigil you get a compile error.

The returned `ObjCObject` is valid until the pool drains. Using it afterwards is a use-after-free, and no amount of compile-time checking can save you from that one — it is the one place in CocoaMojo where the discipline is yours.

The lifetime, as a state machine



`let` for the bindings you never rebind

Everything above works the same whether you bind with `var` or `let`. The ARC behaviour lives in `ObjCRef`, not in the keyword.

```
let obj = make_array()          # +1 at bind, released at scope exit
let s = nsstring("hello")
```

What `let` adds is that the *binding* cannot be reassigned, which is worth having precisely because Cocoa code is full of handles you obtain once and then only send messages to. If you never rebind it, say so; the compiler will hold you to it.

The object stays mutable. `let win = ...` then `setTitle:` is fine — you are mutating the object, not rebinding the name.

Weak references

`ObjCRef` is the strong half of the story. `ObjCWeakRef` is the other half, and you need it more often than you might expect, because **Cocoa's delegate convention is weak**. A window does not own its delegate; an observer does not own its target. Holding an `ObjCRef` where Cocoa expects weakness is a retain cycle that leaks both sides.

```
var strong = make_object()
var weak = ObjCWeakRef(strong.object())

var loaded = weak.load()        # an ObjCRef: the object at +1, or nil
if loaded.is_nil():
    print("gone")
```

It is a **zeroing** weak reference: while the object lives, `load()` returns it; after the last strong reference goes, `load()` returns nil. A weak reference to nil is legal and simply loads nil.

Three details that matter in use.

load() returns an owned ObjCRef, not a bare handle. It goes through `objc_loadWeakRetained`, so the object cannot be deallocated between your nil-check and your use. That is the entire reason to load a weak reference rather than peek at it.

Copying is explicit, matching `ObjCRef`:

```
var weak2 = weak.copy()         # an independent registration
```

There is one small heap allocation per weak reference, and the reason is worth understanding. The Objective-C runtime tracks a weak reference *by the address of the slot holding it*. Mojo values move. If the slot were a struct field, any move would leave the runtime pointing into a dead stack frame — a corruption with no diagnostic. So the slot lives on the heap, its address stays stable for the life of the value, and moving the struct moves only a pointer. Delegates are few; the allocation is not a problem.

A rough edge

Assigning into an uninitialised `var` of a type with a `__deinit__` destroys garbage first and crashes. This is pre-existing and not specific to weak references, but it bites here because the natural shape —

```
var weak: ObjCWeakRef          # then assign later
```

— is exactly that pattern. Bind it in a helper scope and return it instead.

Buffers that Cocoa callbacks must reach

This one is not about Objective-C at all, and it will catch you.

Mojo destroys a value at its **last use**, not at the end of scope. So a `List` whose `.unsafe_ptr()` you stash in a global is freed immediately after that line, and every stored pointer dangles. The symptom arrives much later as a corrupted allocator, nowhere near the cause.

When memory has to outlive Mojo's view of it, own it outside Mojo:

```
def alloc_zeroed(count: Int, size: Int) -> Int:
    var sym = _sym["calloc"]()
    var call = Pointer(to=sym).unsafe_bitcast[
        def(Int, Int, /) thin abi("C") -> P
    ]()[]
    return Int(call(count, size))
```

Explicit, unowned, and correct. The alternative — hoping a `List` lives long enough — is not a smaller amount of unsafety, only a less visible one.

Where callback state lives

Cocoa callbacks are bare C function pointers. They get no closure and no context argument you did not arrange yourself. State they need must therefore live somewhere reachable by name.

```
comptime g_running = named_global["life.running", Int]
comptime g_gen = named_global["life.gen", Int]

# read and write through the pointer
g_running()[] = 1
var generation = g_gen()[]
```

`named_global` gives you one zero-initialised process global per name, shared across every call site, because KGEN deduplicates the global by name. It is the plainest possible answer to a real constraint, and it is better than the alternative of smuggling a pointer through `setRepresentedObject:` and casting it back.

A short checklist

Use `adopt=` after `alloc`, `new`, `copy`, `mutableCopy`; `retain=` otherwise. Use `^` when consuming an `ObjCRef`. Wrap `main` and long loops in `autoreleasepool`, but not AppKit callbacks. Allocate anything a callback must reach outside Mojo's ownership, and reach it through `named_global`.

6. Letting Cocoa call you

Everything interactive in Cocoa works the same way: you supply an object, and the framework sends it a selector. Target/action, delegates, notification observers, timers — all of it.

To supply such an object you need an Objective-C class. In cocoa-mojo you declare one with `class`, and the compiler does the rest.

`class` declares an Objective-C class

The whole of it is a declaration:

```
class ExampleActions:
    def buttonClicked_(self, sender: ObjCObject):
        clicks()[] += 1
```

That is a real Objective-C class. The compiler registers it with the runtime, derives the selector, works out the type encoding, builds the trampoline, and gives you an instance:

```
let actions = ObjCObject(ExampleActions().__objc_id)
```

No `ObjCClassBuilder`, no encoding string, no `IMP`, and no `cmd: P` slot.

Method names become selectors

A trailing underscore is a colon.

Mojo method	Selector
`acceptsFirstResponder`	`acceptsFirstResponder`
`buttonClicked_`	`buttonClicked:`
`mouseDown_`	`mouseDown:`
`applicationShouldTerminateAfterLastWindowClosed_`	`applicationShouldTerminateAfterLastWindow`

The number of underscores must equal the number of arguments after `self`, and a mismatch is a compile error naming both counts.

That diagnostic earns its place in the *other* direction. Writing `drawRect` where you meant `drawRect_` derives a nullary selector, registers perfectly, and then never receives a single draw — the framework keeps sending `drawRect:` to a class that answers `drawRect`. Nothing crashes, nothing appears. It is now a compile error.

A leading underscore means the method is Mojo's own and never reaches the runtime. Without that rule every snake_case helper in a class would derive a nonsense selector like `my:helper` and then be rejected for a colon it never wanted — which would make a Cocoa class a hostile place to put a private method. `_tab_width` is simply private, exactly as Mojo and Python already read a leading underscore, and the dunders come along for free.

To spell a selector that the underscore rule cannot produce, say so:

```
    @objc("insertText:replacementRange:")
    def insert_text(self, text: ObjCObject, range: NSRange): ...
```

Superclass and protocols

```
class LifeView(NSView):          # subclass an SDK class
class LifeDelegate:              # no base means NSObject
class Editor(NSView, NSTextInputClient):   # first base is the superclass,
                                            # the rest are protocols
```

Protocols are adopted through `class_addProtocol` — real conformance, not merely having the methods. AppKit does ask: `conformsToProtocol:` reportedly cost a day on `NSTextInputClient` before that was understood.

An unknown superclass is caught at compile time, so a typo in a class name fails the build rather than producing a class that inherits from nothing.

Encodings are looked up, then derived

If the selector already exists on the superclass chain, the encoding comes from the SDK database, and a signature that disagrees with it is a compile error **quoting the database's encoding**. If the SDK has never heard of the selector — which is the normal case for target/action and delegate methods you invent — the encoding is derived from your Mojo signature.

Either way you do not write `"v@:@"` again.

Methods may raise

This is the part that changes how the code reads:

```
class ExampleActions:
    def buttonClicked_(self, sender: ObjCObject):
Closed:
        with autoreleasepool():
            _ = msg_send[ObjCObject, "NSTextField", "setStringValue:"](...)
```

No `try`, and no `fn`. A `class` method body is a `def` and **may raise**; the synthesized trampoline catches at the Objective-C boundary, because unwinding into `objc_msgSend` is undefined behaviour. On a raise the boundary reports — `NSLog` and continue, by default — and returns a zero value.

You can still write `fn` methods where you want the strict contract and no catch machinery. But the reason the examples read cleanly is that the common case no longer needs one.

Fields

A class holds per-instance state:

```
class Tally(NSObject):
    var hits: Int
    var high_water: Int

    # Mutating methods declare `mut self`, exactly as on a struct.
    def isProxy(mut self) -> Bool:
        self.hits += 1
        if self.hits > self.high_water:
            self.high_water = self.hits
        return True

    def selectedRange(self) -> NSRange:
        return NSRange(self.hits, self.high_water)
```

Three instances of `Tally` count separately, and the counting happens when the Objective-C runtime dispatches to the method — not when Mojo calls it. That distinction matters more than it sounds, and the next section is about it.

Where the state lives: the object gets **one ivar of sizeof(Self)**, 8-aligned, at an offset the runtime settles at registration. There is no allocation beyond the object itself. Every trampoline moves the incoming `id` along by that offset, so `self` inside a method *is* the box.

`mut self` is required to write a field, exactly as on a struct.

The rules, v1

Fields have just landed, and the contract is narrower than it will be. Stated so nobody finds out the hard way:

Fields must be default-constructible, and zero must be a valid value. The runtime zero-fills the object; `__init__` default-constructs every field so both views agree. This is the `named_global` rule you already know, now applied per instance.

Field initializers are not honoured. `var x: Int = 3` does not parse, and even where it did the box would hold the default rather than the 3.

Destruction does not run. `dealloc` is not hooked yet, so a field's `deinit` never fires. A field owning heap memory lives as long as the object and leaks when the object dies — the same lifetime story as the `named_global`s fields replace. Fine for app-lifetime objects, wrong for transient ones.

The constructor-copy trap

This is the one to internalise. `Tally()` hands back a class *value*, which is a copy of the box's ground state plus the `id`. Reading `__objc_id` from it is correct. **Mutating a field through it writes the copy, not the object.**

Watch it happen:

```
var p = Probe()
var o = ObjCObject(p.__objc_id)

p.isProxy()                 # mutate through the Mojo value
# the Mojo value says:   True
```

```
# the object says:       False

_ = msg_send[Bool, "NSObject", "isProxy"](o)    # mutate through the runtime
# the object says:       True
```

So the state a Cocoa callback sees is the state the *runtime* wrote. Since callbacks are the whole point of a class, this is usually invisible: AppKit sends messages, the trampoline resolves the box from the real `id`, and everything agrees. It bites the moment you try to seed a field from Mojo after construction and wonder why the view never sees it.

Until the handle type arrives, treat the value `Tally()` returns as a way to get an `id` and nothing more.

What fields replace

State that used to live in process globals:

```
comptime clicks = named_global["example.clicks", Int]
```

`examples/life/main.mojo` still carries fourteen of these, and most are now convertible. Weigh it against the two limits above: a counter or a flag is a clean win, while anything owning memory keeps the leak it already had and gains nothing until `dealloc` runs.

Still not in this version

Nested classes, class-level `comptime` parameters, and Mojo-trait conformances. A class is not a struct.

Instances

`ClassName()` registers the class if it is not registered yet, then allocs and inits. Registration is idempotent — a second instance costs one `objc_getClass`.

```
var probe = Probe()
var obj = ObjCObject(probe.__objc_id)
```

`__objc_id` is the raw `id`. A class reference is register-passable, retains on copy and releases on destruction — which is what an Objective-C reference *is*, and the same shape `ObjCRef` already had.

Sending to your own selectors

One asymmetry to know. `msg_send` checks the selector against the SDK, so it cannot send a selector you invented:

```
# buttonClicked: is ours — msg_send will not have it
var responds = msg_send[Bool, "NSObject", "respondsToSelector:"](
    obj, sel_dynamic("buttonClicked:")
)
```

`sel_dynamic` registers a selector by name at run time, and `respondsToSelector:` is what a button asks before it sends anything anyway.

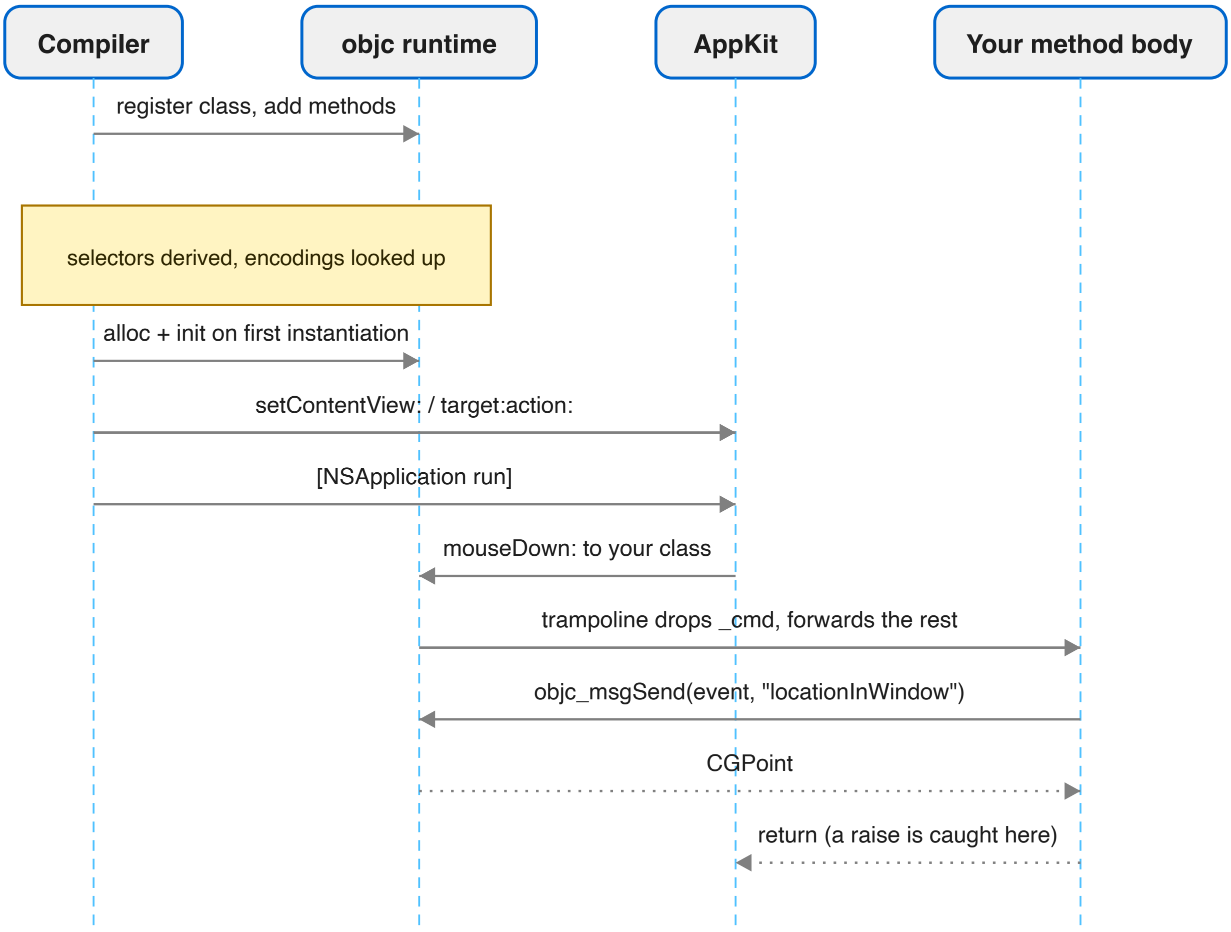
The older way: `ObjCClassBuilder`

Everything above replaces a mechanism that still exists and still works. You will meet it in `spikes/`, which predate `class`:

```
var vb = ObjCClassBuilder["NSView"]("LifeView")
vb.add_method["mouseDown:"](on_mouse_down)
vb.add_method["tick:", encoding="v@:@"](on_tick)
var LifeView = vb^.register()
```

with callbacks written as `fn`s carrying the `self_: P, cmd: P` prefix by hand. It is the layer `class` is built on, and it remains the escape hatch for a method shape the class syntax does not cover. For new code, declare a class.

The round trip



The trampoline is the piece you never see. It carries the `(self, _cmd, args…)` shape the runtime requires, drops the `_cmd` slot, forwards the rest to your method, and catches anything the body raises so it cannot unwind into `objc_msgSend`.

Verifying it worked

`respondsToSelector:` is the cheap check, and worth doing once during bring-up:

```
var responds = msg_send[Bool, "NSObject", "respondsToSelector:"](inst, s)
print("responds:", responds)
```

If that prints `False`, the usual causes are a selector spelled differently from the one you added, or a `register()` that never ran.

Delegates

A delegate is an object implementing the right selectors — so it is a class with those methods on it:

```
class LifeDelegate:
    def applicationShouldTerminateAfterLastWindowClosed_(
        self, app: ObjCObject
    ) -> Bool:
        return True

# ...
let delegate = ObjCObject(LifeDelegate().__objc_id)
_ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](app, delegate.ptr())
```

Because the selector is a real AppKit one, its encoding comes from the database and you never type `"c@:@"` — which, incidentally, is the encoding people most often get wrong, since `BOOL` is `c` and not `B`. Get the signature wrong and you get a compile error quoting what the SDK expects.

Where a delegate must *declare* conformance rather than merely have the methods, name the protocol as a base:

```
class Editor(NSView, NSTextInputClient):
    ...
```

Reading arguments out of an event

Callback arguments arrive as raw pointers. Wrap and send:

```
def event_has_shift(event: ObjCObject) -> Bool:
    var flags = msg_send[Int, "NSEvent", "modifierFlags"](event)
    return (flags & 131072) != 0    # NSEventModifierFlagShift
```

Arguments arrive typed now — a `class` method declares `event: ObjCObject` rather than a raw `P` — so most of the wrapping the old callbacks needed is gone.

That magic number should be `cocoakb_enum_value["NSEventModifierFlagShift"]()`. The example applications hardcode several of these, and it is the one habit in them worth not copying.

Reading a keystroke is the same pattern with a string on the end:

```
class LifeView(NSView):
  def keyDown_(self, event: ObjCObject):
    var chars = msg_send[
        ObjCObject, "NSEvent", "charactersIgnoringModifiers"
    ](ObjCObject(Int(event)))
    if chars.is_nil():
        return
    var p = msg_send[P, "NSString", "UTF8String"](chars)
    if Int(p) == 0:
        return
    var s = String(unsafe_from_utf8_ptr=p.unsafe_bitcast[c_char]())
    if len(s.as_bytes()) == 0:
        return
    var c = s.as_bytes()[0]
    if c == UInt8(ord(" ")):
        toggle_running()
```

Three nil checks before touching anything. AppKit will hand you a nil string for a dead key, and a null `UTF8String` for an empty one.

When the caller is not Objective-C

Everything above is Cocoa calling you through the Objective-C runtime: you declare a `class`, the runtime finds your selector, and dispatch does the work. Some of the most interesting parts of macOS do not work that way. They take a **C function pointer** and call it directly — no object, no selector, no runtime in between.

This is where the fork's `fn` earns its revived meaning. `fn` is thin, non-raising, and C ABI: that is exactly an Objective-C `IMP`, and it is exactly a C callback too. So a Mojo `fn` *is* a C function pointer, and can be handed to a C API with nothing in between — no shim, no trampoline, no C file in the build.

CoreAudio's render callback is the sharpest case, because it also has a deadline. Declare the signature as a type, and the function is that type:

```
comptime P = OpaquePointer[MutUntrackedOrigin]
comptime AURenderCallback = fn(P, P, P, UInt32, UInt32, P, /) -> Int32

fn render(
    ref_con: P, action_flags: P, timestamp: P,
    bus: UInt32, frames: UInt32, io_data: P,
) -> Int32:
    ...
    return 0
```

Installing it means writing the function's address into the struct CoreAudio expects. Read it out of a slot holding the value rather than bitcasting the function itself — the slot's eight bytes are the pointer:

```
var cbfn: AURenderCallback = render
let fn_addr = Pointer(to=cbfn).unsafe_bitcast[Int]()[]
var cbs = external_call["calloc", P](Int(2), Int(8))
```

```
cbs.unsafe_bitcast[Int]()[unsafe_offset=0] = fn_addr
cbs.unsafe_bitcast[Int]()[unsafe_offset=1] = Int(state)   # the refCon
```

What a real-time thread forbids

`render` runs on a thread CoreAudio owns, every few milliseconds, with a hard deadline: 512 frames at 48 kHz is 10.7 ms, and a buffer not filled in time is a gap you can hear. Three rules follow, and none of them is advice:

- **No allocation.** Everything the callback touches is allocated before the unit starts.
- **No locks.** The drawing thread reads the same state to paint meters; a torn read costs one wrong pixel for one frame, where a held lock costs a click in the speaker. Take the torn read.
- **No raising.** An `fn` cannot raise, which is the language enforcing the rule rather than the programmer remembering it.

The callback carries no captured state — a C function pointer has nowhere to put any. That is what `refCon` is for: the last field of the struct above is handed back as the first argument on every call, so the whole synthesiser hangs off one pointer and nothing needs a global.

An unbounded loop is a hang, not a slow path

Ordinary code survives a loop that runs too long; a real-time thread does not. A range-reduction written as *subtract 2π until in range* is fine for a filter coefficient and ruinous for a vibrato whose argument grows with a frame counter: the loop gets one iteration longer every fifty frames, and one day the audio simply stops. Reduce in one step instead, and clamp anything derived from data:

```
let turns = x * 0.15915494309189535     # 1 / 2*pi
var t = x - 6.283185307179586 * Float64(Int(turns))
```

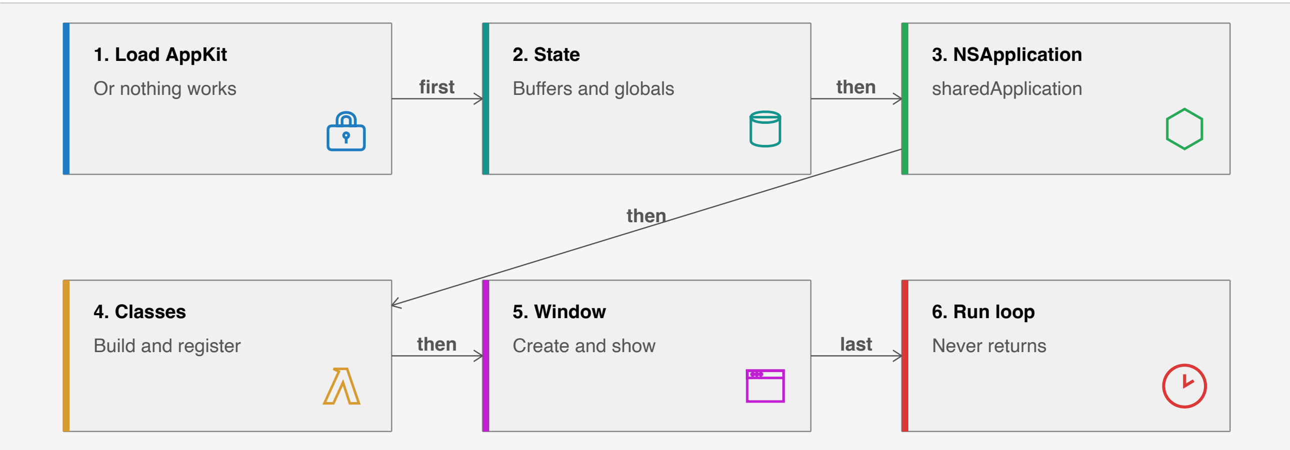
The same reasoning applies to a note number arriving from a file. Clamping it is not defensive decoration: unclamped, an octave normalisation is a loop over the octave count, and a nonsense value is not a wrong pitch but a stopped speaker.

`examples/chip/` is the whole of it — a three-voice synthesiser whose oscillators are phase accumulators, with the render callback above — and `examples/abcplayer/` reads ABC notation and schedules it to the sample through the same path.

7. A complete application

Everything so far assembles into one shape. This chapter is that shape, in the order it has to happen, with the reasons.

The skeleton



Order matters. A class must be registered before anything is instantiated from it, and the application object must exist before a window is made key. Getting either backwards produces a silent failure rather than an error.

Step 0 — load AppKit

```
def main() raises:
    if not load_framework["AppKit"]():
        raise Error("could not load AppKit")
```

Nothing linked AppKit into a `mojo run` process, so without this every AppKit class lookup returns nil and every message to it silently does nothing. The program starts and exits with no window and no diagnostic. Put it first.

Step 1 — state, before anything else

```
    g_alive()[] = alloc_zeroed(CELLS, 1)
    g_next()[] = alloc_zeroed(CELLS, 1)
    g_frame()[] = alloc_zeroed(PIXELS, 4)
    g_running()[] = 1
    randomize()
```

Do this first because the callbacks you are about to install can fire the moment the run loop starts, and a callback that reads a null global crashes on the first frame.

Step 2 — the application object

```
    with autoreleasepool():
        var NSApplication = ObjCClass.lookup["NSApplication"]()
        var app = msg_send[
            ObjCObject, "NSApplication", "sharedApplication", is_class=True
        ](NSApplication.as_object())
        _ = msg_send[Bool, "NSApplication", "setActivationPolicy:"](app, Int(0))
```

`setActivationPolicy:` with `0` is `NSApplicationActivationPolicyRegular` — the difference between a real application with a Dock icon and menu bar, and a process that puts a window on screen that cannot be focused. Skipping it is the most common reason a first CocoaMojo window appears dead.

Step 3 — classes

The delegate:

```
        var db = ObjCClassBuilder("LifeDelegate")
        db.add_method["applicationShouldTerminateAfterLastWindowClosed:"](
            should_terminate
        )
        var delegate = new_instance(db^.register())
        _ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](
            app, delegate.ptr()
        )
```

The view, whose event handlers are Mojo functions:

```
        var vb = ObjCClassBuilder["NSView"]("LifeView")
        vb.add_method["mouseDown:"](on_mouse_down)
        vb.add_method["mouseDragged:"](on_mouse_dragged)
        vb.add_method["keyDown:"](on_key_down)
        vb.add_method["acceptsFirstResponder"](accepts_first_responder)
        var LifeView = vb^.register()
```

`acceptsFirstResponder` returning `True` is what makes the view eligible to receive key events at all. Without it `keyDown:` never fires and the view looks broken in a way that gives no clue.

Step 4 — window and view

```
        var win = msg_send[ObjCObject, "NSWindow", "alloc", is_class=True](
            ObjCClass.lookup["NSWindow"]().as_object()
        )
        win = msg_send[
            ObjCObject,
            "NSWindow",
            "initWithContentRect:styleMask:backing:defer:",
        ](
```

```
            win,
            CGRect(CGPoint(100.0, 100.0), CGSize(1080.0, 720.0)),
            Int(15),
            Int(2),
            Bool(False),
        )
        g_window()[] = win.addr()
```

The `15` is a style mask — titled, closable, miniaturisable, resizable — and should really be four `cocoakb_enum_value` lookups combined. The `2` is `NSBackingStoreBuffered`.

Then the view, and making the window visible:

```
        var view = msg_send[ObjCObject, "NSView", "alloc", is_class=True](
            LifeView.as_object()
        )
        view = msg_send[ObjCObject, "NSView", "initWithFrame:"](view, frame)
        _ = msg_send[ObjCObject, "NSWindow", "setContentView:"](win, view.ptr())
        _ = msg_send[ObjCObject, "NSWindow", "makeKeyAndOrderFront:"](
            win, ObjCObject(0).ptr()
        )
        _ = msg_send[
            ObjCObject, "NSApplication", "activateIgnoringOtherApps:"
        ](app, Bool(True))
```

Step 5 — the run loop

```
    var app2 = msg_send[
        ObjCObject, "NSApplication", "sharedApplication", is_class=True
    ](ObjCClass.lookup["NSApplication"]().as_object())
    _ = msg_send[ObjCObject, "NSApplication", "run"](app2)
```

`run` does not return until the application terminates. Note that it is deliberately **outside** the `autoreleasepool` block: the pool should drain the setup objects before the loop begins, and the loop maintains its own pools per event cycle.

Re-fetching `sharedApplication` rather than reusing `app` is not superstition — `app` was bound inside the `with` block that has now ended.

What you get

The fork ships three applications built exactly this way.

`spikes/life/life.mojo` is Conway's Life with mouse drawing, pause and single step, and cells coloured by age so you can see a pattern's structure rather than a flat mask. Six hundred lines, and the only non-Cocoa part is the simulation.

`spikes/playground/playground.mojo` is a Mojo editor and runner written in Mojo — a thousand lines, a split view, syntax highlighting, and a child process.

`examples/mandelbrot/main.mojo` draws through a `CAMetalLayer`, with every pixel computed and coloured by a Mojo kernel on the Apple GPU. The GPU stack this once waited for works now: it times one CPU core against the GPU (a couple of hundred times apart on an M4 Max), then holds 60fps while it zooms. `examples/chip/` and `examples/abcplayer/` are the real-time counterpart: a synthesiser and an ABC-notation player whose audio is produced by a Mojo `fn` serving as CoreAudio's render callback, on a thread with a 10.7 ms deadline (chapter 6).

`examples/fluid/` is the deeper GPU example — Stable Fluids, about 35 dependent dispatches a frame, no shader anywhere.

Debugging, honestly

There is no source-level debugging here yet. What you have is `print`, `respondsToSelector:` during bring-up, and the compile-time checks doing more work than they would in most languages.

The three failure modes worth recognising:

A window appears but ignores the keyboard: `acceptsFirstResponder` is missing or returns `False`.

A window appears but cannot be focused and has no Dock icon: `setActivationPolicy:` was not called.

A crash inside `objc_msgSend` with a plausible-looking receiver: an object was released while Cocoa still held it. Look for an `ObjCRef` that went out of scope, or a `new_instance` result that was never retained.

The first two are configuration. The third is the one the compile-time checks cannot reach, which is why chapter 5 spends so long on it.

8. Concurrency and blocks

Grand Central Dispatch is the concurrency story for a Cocoa app: UI work belongs to the main queue and everything else hops between queues. It is a C API in libSystem, so there is nothing to link.

The reason it lands cleanly here is a coincidence of contracts. Every GCD operation exists in two forms — a block form, and an `_f` form taking a bare C function pointer plus a context word — and **the _f form is exactly the revived fn**: thin, non-raising, C ABI. So most of `std.objc.dispatch` is direct calls with no adaptation layer at all.

```
from std.objc.dispatch import (
    Semaphore, async_f, global_queue, main_queue, sync_f, with_block,
)
```

Queues

```
var q = global_queue()      # the default-priority concurrent queue
var m = main_queue()        # where UI work belongs
```

`main_queue()` is worth a note: the main queue is not a function in C but a global, `_dispatch_main_q`, and the `dispatch_get_main_queue()` macro takes its address. The library references it the same way it references the `msg_send` stubs — as a link-time symbol.

Dispatching an `fn`

```
fn set_flag(ctx: P) -> None:
    named_global["app.flag", Int]()[] = 42

sync_f(q, ctx, set_flag)     # runs before returning
async_f(q, ctx, set_flag)    # returns immediately
```

The context word is how you get data to the work function, because an `fn` has no closure. In practice you will often reach for `named_global` instead, the same way Cocoa callbacks do — see Where callback state lives.

`Pointer` is non-nullable, so when the work function ignores its context you still have to hand it a real address rather than null. Any stable one will do.

Waiting: `Semaphore`

```
var sem = Semaphore()
var ctx = P(unsafe_from_address=sem._sem)

async_f(q, ctx, add_and_signal)
async_f(q, ctx, add_and_signal)
sem.wait()
sem.wait()
```

One `wait()` per expected signal. This is the rendezvous the dispatch spike uses to prove three concurrent `async_f` calls all ran.

Blocks

Plenty of modern Cocoa is block-only, and blocks have their own ABI — isa, flags, invoke pointer, descriptor, copy and dispose helpers. That sounds like a lot of machinery to build.

It is not, because of the correspondence the design turns on: **a thin fn is exactly a global block.** No captures means `_NSConcreteGlobalBlock` and no copy/dispose helpers at all, so the library can build the 32-byte block literal around any `fn` on the stack.

```
fn block_work() -> None:
    var n = named_global["app.count", Int]()
    n[] = n[] + 1

with_block[block_work](q, wait=True)     # sync: no copy needed
with_block[block_work](q, wait=False)    # async: the runtime Block_copy's it
```

The `wait=True` path never escapes the frame, so the stack literal is enough. The `wait=False` path does escape, and dispatch's own `Block_copy` memmoves the 32 bytes off your frame — safe precisely because there are no captures to deep-copy and the descriptor is immortal.

What is not here yet

Capturing closures as heap blocks. That needs synthesized copy and dispose helpers riding the existing closure emitter, and it is explicitly a later phase. If an API demands a block that captures, you are outside what the library covers today.

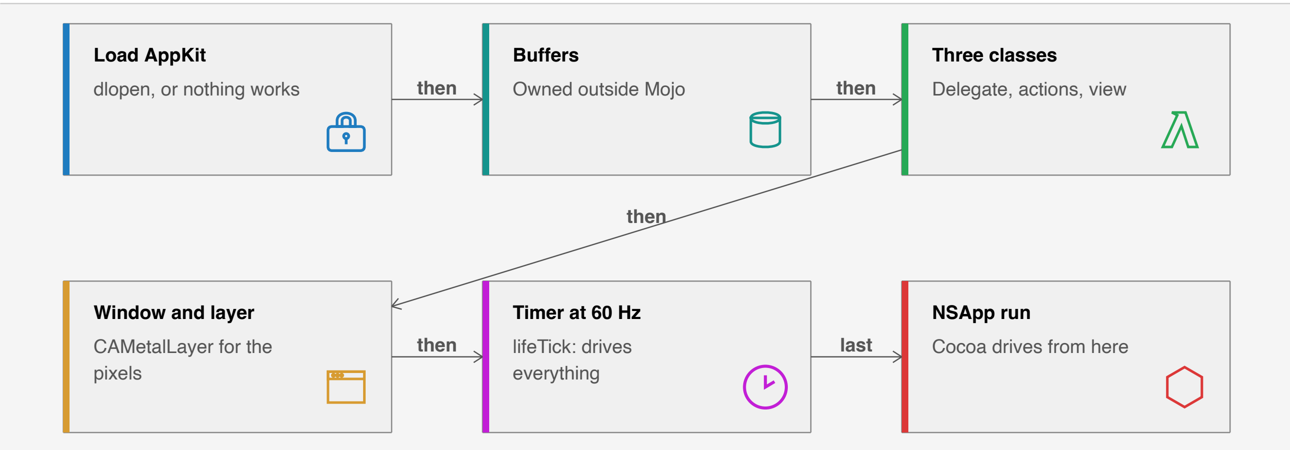
The practical consequence is the one you have already met everywhere else in this guide: state that a callback needs must live somewhere reachable without a closure. `named_global` is the answer here too.

9. A demo, walked through

This chapter reads one complete program: `examples/life/main.mojo` in the fork. Conway's Life, 644 lines, in a real window — mouse drawing, keyboard control, a 60 Hz timer, and cells coloured by age so you can see the structure of a pattern rather than a flat mask. Rendering goes through a `CAMetalLayer`.

It is worth reading because it is the current shape of a CocoaMojo application: three `class` declarations, no `ObjCClassBuilder`, no encoding strings, no `cmd` slots.

Everything below is quoted from the source.



The three classes

That is the part worth studying, and it is short.

A delegate

```
class LifeDelegate:
    def applicationShouldTerminateAfterLastWindowClosed_(
        self, app: ObjCObject
    ) -> Bool:
        return True
```

No base, so it subclasses `NSObject`. One method, whose trailing underscore makes the selector `applicationShouldTerminateAfterLastWindowClosed:`. AppKit declares that selector, so the encoding is looked up rather than derived, and a wrong signature would be a compile error quoting the SDK.

Returning `True` is what lets `[NSApp run]` ever finish: closing the window ends the app.

A timer target, and a lesson in its docstring

```
class LifeActions:
    """The timer's target.
```

```
    Named `lifeTick:` rather than `tick:` on the compiler's advice: the SDK
    declares a `tick:` on CASecureFlipBookLayer taking a double, and a
    selector we invent that collides with one the SDK knows would have been
    registered with that shape -- `v@:d` where we meant `v@:@`. The
    diagnostic said so."""

    def lifeTick_(self, timer: ObjCObject):
        advance_tick()
```

This is the single best illustration in the tree of what the compile-time checking buys.

The obvious name for a timer callback is `tick:`. But `tick:` already exists in the SDK — on `CASecureFlipBookLayer`, taking a `double` — and because encodings are *looked up* when the selector is known, the class would have registered `v@:d` while the code meant `v@:@`. The timer would then have handed a double where an object pointer was expected.

That is a silent, hard-to-find corruption in the old world. Here it was a diagnostic, and the fix was to pick a name nobody else declares — at which point the encoding is derived from the Mojo signature, correctly.

A view subclass

```
class LifeView(NSView):
    def mouseDown_(self, event: ObjCObject):
        var p = event_point(event.ptr())
        paint_at(p.x, p.y, event_has_shift(event.ptr()))

    def mouseDragged_(self, event: ObjCObject):
        ...

    def rightMouseDown_(self, event: ObjCObject):
        var p = event_point(event.ptr())
        paint_at(p.x, p.y, True)

    def acceptsFirstResponder(self) -> Bool:
        return True

    def keyDown_(self, event: ObjCObject):
        handle_key(event.ptr())
```

`NSView` as the first base makes it a subclass. Six methods, six selectors, every encoding from the SDK.

`acceptsFirstResponder` has no underscore and no arguments, so it is the nullary selector `acceptsFirstResponder`. Returning `True` is what makes the view eligible for key events at all — without it `keyDown_` never fires and nothing indicates why.

Note also what is *not* in the class body. `handle_key` and `event_point` are free functions, and the source says why:

> Key handling, kept a free function so the class body stays a list of selectors rather than a wall of logic.

That is a good instinct. A Cocoa class is an interface to the framework; the logic behind it does not have to live inside it.

State lives in globals, for now

```
comptime g_alive = named_global["life.alive", Int]   # UInt8*  per cell 0/1
comptime g_next = named_global["life.next", Int]     # UInt8*  scratch
comptime g_age = named_global["life.age", Int]       # UInt16* generations survived
comptime g_frame = named_global["life.frame", Int]   # UInt32* BGRA pixels
comptime g_running = named_global["life.running", Int]
comptime g_speed = named_global["life.speed", Int]
```

Fourteen of these, and they predate class fields. Most are now convertible: `g_running`, `g_speed`, `g_tick` and the rest are counters and flags on the view, which is exactly what a field is for.

The ones to leave alone are the buffer pointers. A field's `deinit` does not run at `dealloc` yet, so a field owning memory leaks with the object — and these already own memory deliberately, outside Mojo, for the reason the next section gives. Moving them into fields would buy nothing and cost the clarity of saying so.

The `life.` prefix on every key is the discipline that keeps two subsystems from colliding, and it stays worth having for whatever remains a global.

`main`, in order

Load AppKit first

```
def main() raises:
    if not load_framework["AppKit"]():
        raise Error("could not load AppKit")
```

Nothing linked AppKit into a JIT-run process, so without this the `NSApplication` lookup is nil and every message to nil quietly does nothing: the app starts and exits having drawn nothing.

Allocate outside Mojo

```
    g_alive()[] = alloc_zeroed(CELLS, 1)
    g_frame()[] = alloc_zeroed(PIXELS, 4)
```

Mojo destroys a value at its **last use**, not at end of scope, so a `List` whose `unsafe_ptr()` you stash in a global is freed immediately and every stored pointer dangles — surfacing much later as a corrupted allocator, nowhere near the cause. Owning the memory outside Mojo makes the lifetime explicit and correct.

Instantiate the classes

```
        # Instantiating a class is what registers it, so the delegate exists
```

```
    # in the runtime by the time it is handed over.
    var delegate = ObjCObject(LifeDelegate().__objc_id)
    _ = msg_send[ObjCObject, "NSApplication", "setDelegate:"](
        app, delegate.ptr()
    )

    var view_instance = ObjCObject(LifeView().__objc_id)
    var actions = ObjCObject(LifeActions().__objc_id)
    _ = external_call["objc_retain", P](actions.ptr())
```

Three lines where the old version had three builders, six `add_method` calls and two hand-written encodings.

Two details worth taking away.

Instantiation is registration. There is no separate register step, so the class is in the runtime by the time you hand the instance to AppKit.

Retain what Cocoa holds by bare pointer. `actions` is the timer's target and must outlive this scope; the explicit `objc_retain` is correct for an object that lives as long as the application.

A difference from the builder path

```
    # `LifeView()` is already allocated and initialised -- that is what
    # instantiating a class does -- so the frame is set rather than passed
    # to initWithFrame:.
    var view = view_instance
    _ = msg_send[ObjCObject, "NSView", "setFrame:"](view, frame)
```

Worth noticing because it changes the code you write. With `ObjCClassBuilder` you got a class back and did `alloc` then `initWithFrame:` yourself. `LifeView()` has already done both, so you set the frame afterwards instead.

The timer

```
    _ = msg_send[
        ObjCObject, "NSTimer",
        "scheduledTimerWithTimeInterval:target:selector:userInfo:repeats:",
        is_class=True,
    ](
        ObjCClass.lookup["NSTimer"]().as_object(),
        Float64(1.0 / 60.0),
        actions.ptr(),
        sel["lifeTick:"]().ptr(),
        actions.ptr(),
        Bool(True),
    )
```

`sel["lifeTick:"]()` is the one place the selector appears as a value rather than as a `msg_send` parameter — target/action wants the `SEL` itself.

`Float64(1.0 / 60.0)` and not `1.0 / 60.0`: the interval travels in a float register, and the register-file check would have caught an integer literal.

Into the run loop

```
    var app2 = msg_send[
        ObjCObject, "NSApplication", "sharedApplication", is_class=True
    ](ObjCClass.lookup["NSApplication"]().as_object())
    _ = msg_send[ObjCObject, "NSApplication", "run"](app2)
```

Outside the `with` block, so the setup pool drains before the loop begins; the loop keeps its own pools per event cycle. `app` was bound inside the block that has now ended, hence `app2`.

`run` does not return.

What the drawing does

The parts this chapter has skipped are ordinary Mojo. `evolve()` runs the Life rules over the two cell buffers; `render()` writes BGRA pixels into `g_frame`; `present()` blits that into a `CAMetalLayer` drawable. The Metal work uses `send` rather than `msg_send`, because a `MTLDevice` is a protocol-typed object whose concrete class is not something you can name:

```
        var queue = send[ObjCObject, "newCommandQueue"](display_dev)
```

What to take from it

The Cocoa-facing surface of a 644-line application is three class declarations totalling about thirty lines, and every selector and encoding in them was checked at compile time. The rest is Mojo.

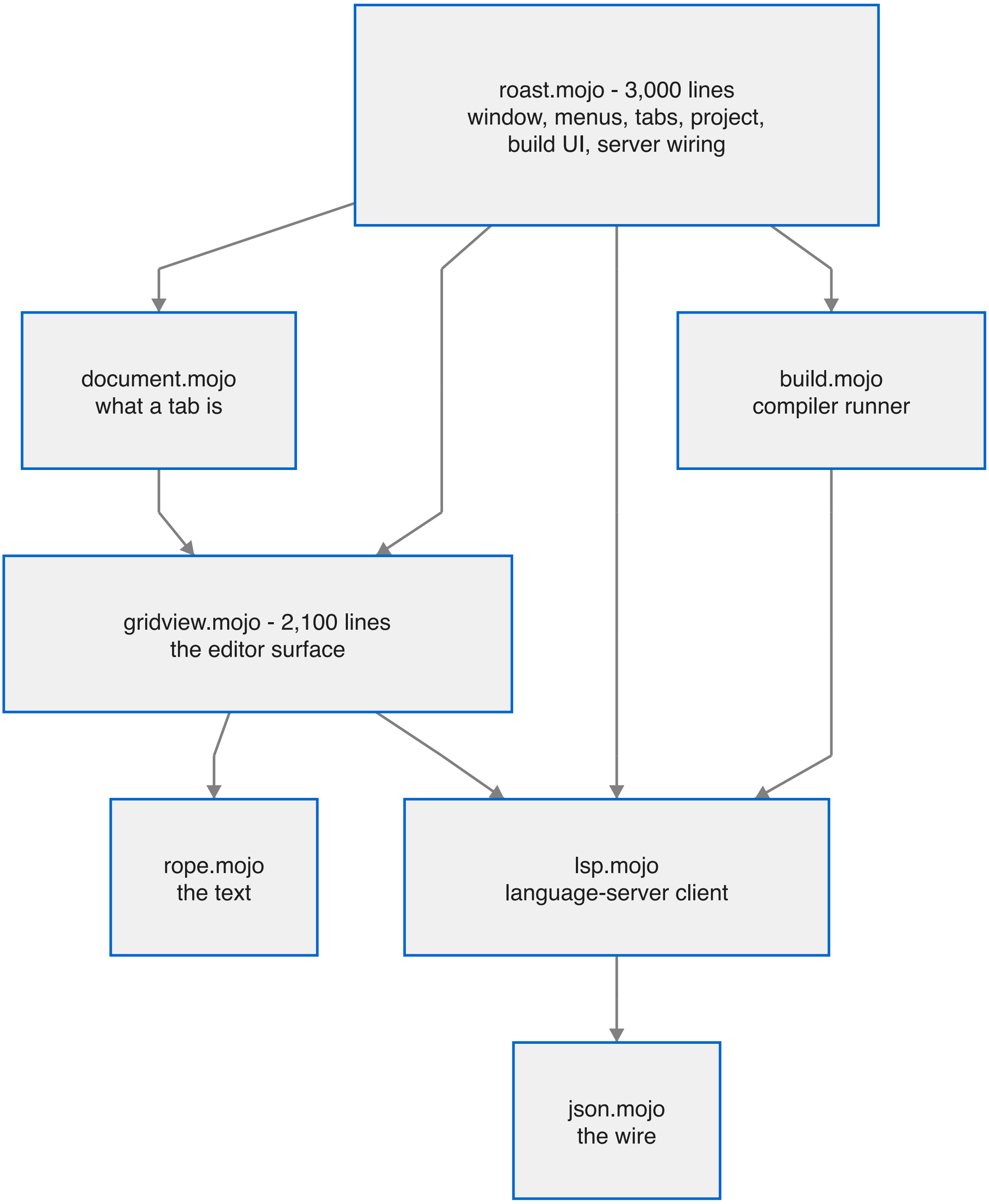
That ratio is the point of the whole design.

10. Roast: the IDE, read as a program

Chapter 9 read a demo. This chapter reads a tool: **Roast**, the fork's own IDE, written in cocoa-mojo, and the largest program in the dialect — about 9,000 lines across seven modules and their test suites. It has tabs, a project sidebar, a build-and-run console, completions and diagnostics from a language server, find, undo, zoom, and an Examples menu. It regularly edits its own source.

It earns a chapter for a different reason than Life did. Life shows how small a Cocoa program can be. Roast shows the shapes that appear when a program gets big *without* stopping being cocoa-mojo: where state lives when callbacks have no closures, what the revived `let` does to code that mutates containers, how a process is driven without blocking the main thread, and how an application proves itself in CI with nobody at the keyboard.

Everything below is quoted from the source at commit `3275387`. The program moves; the shapes are the point.



Each module below gets a section, and each section is really about one piece of the language.

The rope: structs, sharing, and `ArcPointer`

The text is a persistent rope — a B-tree over UTF-8 leaves in which **nodes are immutable and shared**. An edit copies only the path from the touched leaf to the root and returns a new root; every other node is shared with the previous version.

```
struct Node(Movable, Deinitable):
    """One rope node: either a run of text, or a list of children."""

    var is_leaf: Bool
    var text: String
    var kids: List[ArcPointer[Node]]
    var nbytes: Int
    var nlines: Int  # newline characters beneath this node, not line count
    var nutf16: Int  # UTF-16 units beneath this node -- what Cocoa counts in
```

This is chapter 3's material doing real work. `Node` is an ordinary Mojo struct; `ArcPointer` makes children shareable between trees; and the three cached counts are what make every question the editor asks — *which line is this offset on, where does line 40,000 start, what is this range in UTF-16* — an O(log n) walk instead of a scan.

One property pays three times over, and the editor is designed around it:

- **Undo is a stack of old roots.** Structural sharing makes a thousand-entry history cost kilobytes, so there are no command objects and no inverse operations to get wrong.
- **A snapshot is one pointer copy** — `rope.copy()` retains the root — so saving, searching, and sending text to the language server all read a consistent tree while the main thread keeps editing.
- **A save cannot tear**, because the tree it serialises cannot change.

Byte offsets throughout, and the dialect's slice spelling keeps everyone honest: the rope indexes bytes (`s[byte=a:b]`), and only the view layer thinks in characters (`s[codepoint=:keep]`). The third cached count exists because Cocoa's text system counts UTF-16 units — `selectedRange` is asked on essentially every keystroke, and answering it from cached per-node counts rather than by walking a copy of the prefix is the difference between a 2.4 µs keystroke and a multi-megabyte one.

The editor surface: a `class` that draws

The whole editor is one custom view. Its declaration is chapter 6 in a single line:

```
class RoastGridView(NSView, NSTextInputClient):
```

The base list names the superclass and the protocol, and the protocol matters: implementing the selectors is not conforming, and AppKit asks `conformsToProtocol:` before it will speak NSTextInputClient to a view. All eleven protocol methods are implemented, because typing ASCII works with almost any half of the protocol — dead keys, option-e composition and every CJK input method go through marked text, and a client that answers `hasMarkedText` without maintaining a marked range corrupts the buffer for exactly the people using those input methods.

Drawing is arithmetic, on purpose. A fixed-pitch font means

```
x = column * advance          y = line * line_height
document height = line_count * line_height
```

so there is no layout pass, ever. `drawRect_` draws the lines the *damage* covers — not the viewport:

```
                # The dirty rect, not the viewport. For a keystroke they are
```

```
                 # the same thing; for the caret blink the dirty rect is a
                 # four-point sliver, and drawing the viewport instead meant
                 # re-lexing every visible line twice a second to blink a
                 # cursor.
```

Keyboard input never touches the buffer directly. Every key goes to the input context, which turns it into either committed text or a *command selector* — and commands are where an editor's real bugs live, so they are one function, `apply_command`, driven directly by a windowless test suite:

```
    def keyDown_(self, event: ObjCObject):
        """Every key goes to the input context, never straight to the buffer.

        Interpreting the event ourselves would work for ASCII and break every
        input method: it is `interpretKeyEvents:` that turns a keystroke into
        `insertText:`, a command selector, or marked text mid-composition.
        """
```

One detail worth stealing: the shifted movement keys arrive as the same selectors with an `AndModifySelection:` suffix. Roast strips the suffix once, and every motion — present and future — gains its selecting twin for free.

State a callback can reach: `named_global`, then fields

A Cocoa callback is a C-ABI `fn` with no closure. Whatever it needs, it must be able to *find*, and Roast shows the two answers the dialect has, in the order it grew them.

The first is `named_global`: one zero-initialised process global per name. The contract is strict — a zero-initialised global of any type is all zeros — and the codebase treats that as a design constraint rather than a nuisance:

```
# The buffer lives in a one-element global list rather than a raw heap slot.
# A zero-initialised global of any type is all zeros, and assigning a value
# with a destructor over zeros would run that destructor on garbage. A List is
# the exception worth using: zero-initialised it *is* a valid empty list, so
# the first buffer is appended and every later one replaces element zero --
# destroying a real Rope, which is correct.
comptime g_buffer = named_global["roast.buffer", List[Rope]]
```

That pattern — `Int` for object addresses, one-element `List` for anything with a destructor — repeats across the program's 89 globals.

The second answer is newer: **fields on the class**. State that is *per view by nature* now lives in the view's box:

```
    var caret: Int
    """the insertion point, in bytes from the start of the rope"""
    var anchor: Int
    """the other end of the selection; equal to the caret when there is none"""
```

The migration is instructive because of what it did *not* do: none of the 149 call sites moved. Each global became a function of the same name returning a pointer, choosing storage by whether a view exists yet:

```
def g_caret() -> Pointer[Int, MutUntrackedOrigin]:
    """`caret`, on the view. Spelled as it always was, so no call site moved.
    ...
    """
    var id = g_grid()[]
    if id == 0:
        return _pre_caret()
    return Pointer(to=box_ref[RoastGridView](id)[].caret)
```

The fallback is not a transition artefact — the editing test suite drives the whole editor without ever making a view, deliberately, because the risk there is the arithmetic and not the Objective-C. And the test the migration was *for* could not previously have been written at all: two views, two carets, and editing one leaves the other alone.

`let`, `var`, and a bug that shipped

The revived `let` is an immutable binding **to a place**, not a snapshot of a value. The codebase documents the trap twice, and then a review caught it a third time, live.

The documented version, from the highlighter:

```
            # `var`, not `let`. In cocoa-mojo `let x = y` binds to y rather
            # than copying it, so `let start = i` followed i as the loop
            # advanced and the keyword span came out empty.
```

The live version was in `popup_accept`, the code that types a completion into the buffer when you press Enter. It bound `let text` into the completion list, then called `hide_popup()` — *which clears that list* — then inserted `text`. That is a read of freed memory, and it inserted the right text anyway, because freed heap memory usually still holds its bytes. The worst kind of working.

The interesting part is why it compiled. Bind into a *tracked* list and the compiler refuses:

```
error: use of invalidated interior reference 'l["element"]'
```

But every long-lived structure in a Cocoa program routes through `named_global`, whose pointers carry `MutUntrackedOrigin` — and untracked origins are precisely where invalidated-reference analysis goes blind. The fix is one word, now wearing its reason:

```
    # `var`, not `let`. The revived `let` binds to the list slot, and
    # hide_popup() clears that list ... `var` copies while the slot is
    # still alive.
    var text = g_comp_insert()[][sel]
```

The rule this teaches: **let into a container you are about to mutate is wrong, and the compiler can only tell you so when the origin is tracked.** Inside the `named_global` world, the discipline is yours.

Two child processes, zero blocked threads

Roast runs a language server for its whole life and a compiler on demand. Both are `NSTask`s with pipes, and both are drained *without blocking* from the same timer, because a server thinking hard must not be an editor that has stopped responding.

The non-blocking read is a lesson in ABI honesty, quoted at length because the comment is the best in the codebase:

```
@always_inline
def readable(fd: Int) -> Bool:
    """Is there anything to read right now?

    This exists instead of setting O_NONBLOCK, and the reason is an arm64 ABI
    trap worth writing down. `fcntl` is variadic -- int fcntl(int, int, ...) --
    and on arm64 a variadic argument is passed on the stack, not in a register.
    Calling it through a fixed three-argument signature puts O_NONBLOCK in x2,
    where fcntl never looks, so the flag is silently not set and the next read
    blocks the main thread until the server happens to say something. The
    editor stops responding and nothing in the code looks wrong.

    `poll` takes a pointer, a count and a timeout, all fixed, so it has no such
    hazard. POLLIN is 1; a zero timeout means ask and return.
    """
```

The server conversation is JSON-RPC with `Content-Length` framing — a header, a blank line, and that many *bytes* — and the client is strict about protocol order in a way worth copying. Nothing is sent before the server answers `initialize`; the client counts completed handshakes, and the application watches the count and announces every open document when it moves:

```
def announce_open_documents():
    """Send didOpen for every open tab.

    load_file announces a file when the server is ready at the time -- which
    it is not at startup (the handshake takes a poll cycle), and not for any
    tab that was open when a project change swapped in a fresh server process.
    ...
    """
```

Replies are matched by request id, and a completion reply additionally records *which document* it was asked about — a reply that is not superseded can still be about a file the user has switched away from, and a late answer pasted into the wrong buffer is worse than no answer.

Documents: the working set

With one buffer, globals were the right answer. With tabs they are exactly wrong — and rather than thread a document handle through forty call sites, `document.mojo` keeps the globals as the **working set** and makes a `Document` a *saved state*. Switching tabs stashes the working set into the document being left and loads the one being entered. Two functions, one place, and every draw loop and input handler keeps working unchanged.

The rope is what makes this honest: stashing a document's text is one pointer copy, and so is its whole undo history, because the entries share structure.

Every tab change goes through one function, so a rule added later cannot be forgotten at one of the nine places a tab can change:

```
def switch_document(index: Int) -> Bool:
    """Change tabs, flushing the outgoing document first.

    Every tab change goes through here rather than calling
    `document.switch_to` directly, so the flush cannot be forgotten at one of
    the nine places a tab can change.
    """
```

The flush exists because edits are debounced — three idle ticks before a `didChange` goes out — so a tab edited and left quickly would strand its text, and the server would go on reporting diagnostics for a version of the file that no longer exists anywhere. Exactly the case that looks like the tool lying to you.

The shell: five classes and a timer

All of Cocoa's callback surface lands on five classes: the delegate, one target for every menu and toolbar action (which is also the sidebar's data source), the editor view, the completion popup's view, and the tab strip. The tab strip is the one to read — it draws itself, hit-tests its own close boxes, scrolls, and keeps its label attributes as fields built lazily in its own box.

The program's heartbeat is one timer tick that drains both children and notices state changes by serial number rather than by flag:

```
    def timerTick_(self, timer: ObjCObject):
```

Everything asynchronous in Roast — server messages, compiler output, build completion, completion replies, handshake completion — is a counter someone bumps and the tick compares. No locks, no queues of closures: the run loop is the concurrency model, which is chapter 8's advice applied to a whole application.

Behaviour rules live in one place each. Opening an example *is* a project switch — save what is dirty, root the project, open its files, close what belongs to the old project — and the close rule survived a data-loss review:

```
    # Still dirty means the save above did not happen -- the panel was
    # cancelled, or the write failed. Closing it anyway would discard exactly
    # the text the person just declined to write down, so it stays open
    # alongside the new project instead.
```

Proving it with nobody at the keyboard

Roast's test philosophy is the part most worth stealing, and it is two rules.

Rule one: logic tests need no window. The rope, the JSON, the editing commands, the build driver and the LSP client all run headless — `edit_test.mojo` alone makes 122 assertions about caret arithmetic, UTF-8 boundaries, selection rules, undo bounds and clipboard round trips by driving `apply_command` directly. When a bug is found in the app, the fix lands with a check here, which is why the file keeps growing.

Rule two: the application reports its own state. A screenshot needs an unlocked screen and a human; instead the app prints what AppKit actually thinks — window visible, tab gap, menu count, sidebar rows — and the harness greps. Behaviour that needs a click gets a *door*: an environment variable that does exactly what the click does, through the same code path. The Examples menu is even driven as a real menu — the harness finds the item by title and calls `performActionForItemAtIndex:` — because a door that merely reproduces the action's logic goes on passing after the menu stops reaching it. That exact gap is how "the example opens one file of three" once shipped.

Unattended runs are also *polite*. A harness launch is still a real GUI process on a real desktop, and as a Regular app it took the screen from whoever was working. With the autoclose door set, the app declares itself an Accessory and never claims the front:

```
# Accessory (1) gives the same window and the same
# AppKit behaviour with no Dock icon and no claim on the front.
let headless = g_autoclose()[] != 0
```

What to take from it

Reading Roast end to end, five shapes recur, and they are the chapter:

1. **Cache what you will be asked.** Three counts on every rope node turn every editor question into a descent.
2. **State has an owner.** Application-wide state in `named_global`s, per-view state in the view's box, per-document state in the document — and accessors that let call sites not care which.
3. **let binds places.** Reach for `var` the moment the source of a binding can change under it; inside untracked origins nobody else will catch it.
4. **Never block the run loop.** Children on pipes, `poll` before `read`, serials instead of callbacks.
5. **Every behaviour gets a door.** If CI cannot reach a code path without a human, that path will ship broken — and in this program's history, it did.

Appendix A. Building from source

You do not need this to write CocoaMojo programs. The installer gives you a complete toolchain, and the standard library it installs is editable. This appendix is for working on the fork itself: the compiler, the AIR backend, the IDE.

What you need

Requirement	Why
Apple Silicon Mac, macOS 15+, Xcode 16+	As for the installed toolchain
A checkout of MojoCocoa	The compiler and standard library
A checkout of CocoaBaseMCP	Builds `cocoa.sqlite`

The two checkouts are expected to sit beside each other. Everything below assumes that layout; if yours differs, one environment variable fixes it.

Build the SDK database

The database is built from *your* machine — the live Objective-C runtime plus BridgeSupport — not downloaded. It takes about twelve seconds.

```
python3 ../CocoaBaseMCP/build.py
```

You get roughly 236 MB describing 28,814 classes and 522,170 methods. This is the same database the installer generates; it is separated here only because a source tree has no installer to do it.

Build the compiler

```
./bazelw build --config=build-mojo --config=release //KGEN/tools/mojo:mojo
```

Both --config flags matter. Without them the build succeeds and produces a compiler that cannot declare an Objective-C class, along with a `libLLVM` containing no symbols — a failure that looks like a source problem and is not.

For everything a distribution needs, use the release script, which builds the compiler, the shared libraries, `CompilerRT`, the language server and the debugger, assembles `dist/CocoaMojo`, and verifies the result:

```
./tools/release.sh
```

Point the compiler at the database

```
export MODULAR_MOJO_MAX_COCOAKB_PATH=/path/to/cocoa.sqlite
```

An assembled distribution carries its own copy and needs no variable; `bin/cocoamojo` exports this before every build.

Run a program from the source tree

There is a wrinkle worth knowing before it wastes your afternoon: the raw `mojo-full` binary cannot locate `std.mojoc` on its own. Run Mojo through the Bazel wrapper, which supplies the standard-library import paths:

```
./bazelw run //KGEN:mojo -- run /absolute/path/to/program.mojo
```

Note the **absolute** path. Bazel runs the wrapper in its own working directory, so a relative path will not resolve the way you expect.

Run the checks

The interesting half of the verification suite is the half that must *fail*:

```
./spikes/run-cocoa-checks.sh
```

Twelve spikes that must compile and run, and four that must be **rejected at compile time** — sixteen checks in all. A run in which the must-fail spikes quietly succeed is a failed run, because the design's entire claim is that unknown names cannot reach code generation.

The four must-fail checks each pin one guarantee: an unknown metadata name (`must_fail.mojo`), a wrong argument count (`must_fail_argcount.mojo`), a raising `fn` (`must_fail_fn_raises.mojo`), and reassigning a `let` (`must_fail_let_assign.mojo`).

Build the IDE and an installer

Roast is built from `ide/roast.mojo` by the freshly built compiler:

```
./tools/make-app.sh
```

The disk image — payload, installer, signing, notarization, stapling and verification — is built from the `RoastInstaller` checkout beside this one:

```
MOJOCOCOA=/path/to/MojoCocoa ../RoastInstaller/tools/make-release.sh
```

Signing identity and notary profile are personal and live in an ignored `tools/signing.local.sh`. `--no-notarize` builds everything but the round trip.

CocoaMojo Reference Manual

Look-up material, organised by what you are trying to find. If you are learning rather than checking, start with the Programmer's Guide.

Section	Contents
1. Language reference	The dialect this frozen compiler accepts: keywords, conventions, declarations, and a complete list of what was removed
2. std.objc	Every exported type and function, with signatures
3. cocoakb queries	Every compile-time metadata query, its parameters and its result
4. Diagnostics	The compile errors this layer produces, what each means, and how to fix it
5. Deviations from Modular's compiler	Every way this fork changes the language, the argument for each, and what it costs — plus what is upstream's churn rather than ours
6. Patches applied to the compiler	The work inside the compiler that made those changes real, and the defects fixed on the way

Version

Everything here describes the compiler at the fork point of MojoCocoa, Mojo 1.1.0. The fork does not track upstream, so this reference does not go stale in the usual way — it describes one compiler that does not change.

Facts were taken from the source tree: `mojo/stdlib/std/objc/`, `mojo/stdlib/std/sys/_cocoakb.mojo`, `KGEN/lib/MojoParser/`, and the verification spikes under `spikes/`.

Notation

Parameters appear in square brackets and are resolved at compile time. Arguments appear in parentheses. A parameter with `= value` is defaulted.

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R
```

Throughout, `P` abbreviates `OpaquePointer[MutUntrackedOrigin]`, following the convention used in the standard library itself.

1. Language reference

The dialect accepted by this frozen compiler. Where it differs from published Mojo documentation, this document describes the compiler.

Declarations

Functions

Two keywords, with different contracts.

```
def name(arg: T, ...) -> R:          # ordinary Mojo function
def name(arg: T) raises -> R:
def name[param: T](arg: T) -> R:

fn name(a: P, b: P) -> Bool:         # foreign-callable: thin, C ABI, non-raising
```

`raises` marks a `def` that can raise. `abi("C")` on a `def` gives the C calling convention and belongs before the return arrow; `fn` implies it.

fn — the foreign-callable function. Thin (no captures), non-raising, C ABI, all parameter and return types ABI-classifiable. Exactly the Objective-C `IMP` contract. Callable from Mojo as an ordinary function. May not be marked `raises` or `async`.

Bindings

```
var x = expr         # mutable binding
let x = expr         # immutable binding, function body only
comptime X = expr    # compile-time binding
```

`let` is immutable in the *binding*, not the object: `let win = ...` still permits `setTitle:`. It is general — it applies to values as well as reference-counted objects. Ownership semantics for Cocoa objects live in `ObjCRef`, not in the keyword. There is no field or file-scope `let`.

Function types

```
fn(T1, T2, /) -> R                   # sugar for the line below
def(T1, T2, /) thin abi("C") -> R
def[w: Int](Int) thin -> None where (w > 0, "width must be positive")
```

`/` ends the positional-only parameters. `thin` means no captured context. A `thin` function type may carry trailing `where` clauses constraining the parameters it declares; the clause binds to the innermost function type, so a declaration-level `where` after a function-type result needs that result parenthesised:

```
def make[n: Int]() -> (def() thin -> None) where n > 0: ...
```

Binding a constrained function to a function type that declares no matching `where` clause is an error. Passing an unconstrained function where a constrained type is expected is allowed and free.

Compile-time bindings

```
comptime NAME = expression
comptime Alias = SomeType[Param]
```

`alias` still parses but is deprecated: `'alias' is deprecated; use 'comptime'`.

Compile-time control flow

```
comptime if cond:
    ...
else:
    ...

comptime for i in range(n):
    ...

comptime assert condition, "message"
```

`@parameter if` and `@parameter for` still parse but are deprecated. The `@parameter` decorator on parametric closures is now `@__parameter`.

Structs

```
@fieldwise_init
struct Name(Trait1, Trait2):
    var field: T

    def __init__(out self, ...):
    def method(self) -> T:
    def mutator(mut self):
    def consume(deinit self) -> T:
    def __deinit__(deinit self):
```

Struct parameters are declared in brackets after the name:

```
struct ObjCClassBuilder[superclass: StaticString = "NSObject"]:
```

`__new__` is not supported on structs; use `__init__`.

Argument conventions

Convention	Meaning
(omitted)	Immutable borrow. The default.

`imm`	Immutable borrow, written explicitly.
`mut`	Mutable borrow.
`out`	Uninitialised result the callee must initialise.
`var`	By value; callee owns it.
`deinit`	Consumes the value, ending its lifetime.

`read` is the deprecated spelling of `imm` and warns with a FixIt. `inout`, `borrowed` and `owned` do not exist.

`imm`, `mut`, `out`, `deinit` and `where` are **contextual** — soft identifiers, not reserved words — so they remain usable as ordinary names. `var` is a reserved keyword.

Function types do not accept `deinit`; use `var` there.

The transfer sigil `^` moves a value into a consuming position:

```
var cls = builder^.register()
```

Reserved words

The compiler reserves 64 keywords. Most you will use; a handful are lexed and classified but have no parse handling at all, and a block of double-underscore names are compiler internals.

Ordinary

```
_        alias    and      as       assert   async    await    break
comptime continue def      elif     else     except   finally  fn
for      from     if       import   in       is       let      not
or       pass     raise    ref      return   struct   trait    try
var      while    with     lambda
```

`alias` warns and is superseded by `comptime`; `fn` and `let` have the cocoa-mojo meanings described above.

Reserved but not implemented

```
del      global   match    case     nonlocal   yield
```

Lexed and classified as statement keywords and then never parsed, so they are unavailable as identifiers without buying you anything. There is no pattern matching, no `del`, no generators, and no `global`/`nonlocal`.

`class` was in this list and no longer is — see below.

`class`

Declares an **Objective-C class**, not a Mojo one.

```
class Name(Superclass, Protocol, ...):
    var field: Type                        # no initializer yet
    def method_(self, arg: T) -> R: ...    # may raise; the boundary catches
    def mutator_(mut self, arg: T): ...    # mut self to write a field
    fn strict_(self, arg: T): ...          # fn's contract, unchanged
    @objc("real:selector:")
    def renamed(self, a: T, b: U): ...
```

The first base is the superclass and defaults to `NSObject`; the rest are protocols, adopted through `class_addProtocol`. Method names map to selectors by turning a trailing `_` into `:`, with the underscore count required to match the argument count. A **leading** underscore marks a method as Mojo's own, never exposed to the runtime.

Encodings are looked up from the SDK when the selector exists on the superclass chain — a disagreement is a compile error quoting the database — and derived from the Mojo signature otherwise.

`ClassName()` registers the class if needed, then allocs and inits; registration is idempotent. `__objc_id` is the raw `id`. A class reference is register-passable, retains on copy and releases on destruction.

Fields are stored inline: one ivar of `sizeof(Self)`, 8-aligned, at the offset the runtime settles at registration. Each trampoline advances the incoming `id` by that offset, so `self` in a method is the box. Writing a field requires `mut self`.

Field rules in this version: the type must be default-constructible and zero must be a valid value for it; field initializers are not honoured; `dealloc` is not hooked, so a field's `deinit` never runs and an owning field leaks with the object. And `ClassName()` returns a *copy* of the ground state plus the `id` — reading `__objc_id` from it is correct, mutating a field through it writes the copy rather than the object.

Not in this version: nested classes, class-level `comptime` parameters, Mojo-trait conformances.

Compiler internals

```
__comptime_assert         __extension               __functions_in_module
__generator_type          __get_address_as_owned_value
__get_address_as_uninit_lvalue                      __get_current_function_name
__get_litref_as_mvalue    __get_mvalue_as_litref    __is_run_in_comptime_interpreter
__mlir_region             __mojo_crash              __struct_field_ref
conforms_to               origin_of                 type_of
```

`conforms_to`, `origin_of` and `type_of` are the three you will actually write — they appear in `where` clauses and `comptime assert`s throughout the standard library. `__comptime_assert` is deprecated in favour of `comptime assert`. The rest are the compiler talking to itself.

Contextual, not reserved

```
imm      mut      out      deinit   where    read
```

These are soft identifiers: they mean something in a signature and remain usable as ordinary names everywhere else. `read` is deprecated in favour of `imm`.

Origins

Spelling	Removed alias
`ImmOrigin`	`ImmutOrigin`
`ImmUnsafeAnyOrigin`	`ImmutUnsafeAnyOrigin`
`ImmStaticOrigin`	`StaticConstantOrigin`
`UntrackedOrigin`	`ExternalOrigin`
`MutUntrackedOrigin`	`MutExternalOrigin`
`ImmUntrackedOrigin`	`ImmutUntrackedOrigin`, `ImmutExternalOrigin`

Also in use: `MutAnyOrigin`, `MutOrigin`.

Pointers

`Pointer[T, Origin]` is the type. `OpaquePointer[Origin]` is the untyped one. `UnsafePointer` exists but is deprecated in favour of `Pointer`.

Removed aliases: `MutUnsafePointer`, `ImmUnsafePointer`, `ImmutUnsafePointer`, `ImmutOpaquePointer`, `ImmutPointer`, `OptionalUnsafePointer`. Use `MutPointer`, `ImmPointer`, `ImmOpaquePointer`, `OptionalPointer`.

Construction and use:

```
Pointer[UInt8, MutUntrackedOrigin](unsafe_from_address=addr)
OpaquePointer[MutUntrackedOrigin](unsafe_from_address=addr)
p[]                                  # dereference
Pointer(to=value).unsafe_bitcast[T]()
p.bitcast[T]()
```

Renamed methods: `unsafe_as_noalias()`, `unsafe_deinit_pointee()`, `unsafe_deinit_pointee_with()`, `unsafe_write()`, `unsafe_write_move_from()`. `Pointer.type` is now `Pointer.T`.

Renamed free functions: `unsafe_memcpy`, `unsafe_memset`, `unsafe_memset_zero`, `unsafe_uninit_move_n`, `unsafe_uninit_copy_n`, `unsafe_destroy_n`, `unsafe_memcmp`.

Traits in common use

`AnyType`, `Copyable`, `Movable`, `Deinitable`, `TrivialRegisterPassable`.

`ImplicitlyDestructible` and `ImplicitlyDeletable` were renamed to `Deinitable`. `@explicit_destroy` is no longer required. The `register_passable` and `escaping` function effects are no longer supported.

Decorators

`@fieldwise_init`, `@always_inline`, `@staticmethod`, `@export("name")`, `@deprecated(...)`, `@__parameter`.

MLIR escape hatches

Used by the standard library where no surface syntax exists:

```
__mlir_op.`pop.global_alloc`[...]()
__mlir_op.`pop.extern_ptr_symbol`[...]()
__mlir_attr[`#kgen.param.expr<...>`]
```

These are internal, carry no stability guarantee, and are documented here only so that reading the standard library is possible.

Modules

Directories may have namespace semantics: one directory name may resolve across distinct locations on disk that share the name.

Importing same-named functions from different modules to form one overload set is an error. Intra-package access without an explicit `import` is an error.

`.mojopkg` is gone; the package format is `.mojoc`.

Removed library APIs

Removed	Replacement
`InlineArray`	`Array`
`SIMD.size`, `Array.size`, `TypeList.size`, `SIMDSize`	`length`, `SIMDLength`
`String.as_string_slice()`	`StringSpan(s)`
`String.set_byte_length()`	*(internal, none)*
`as_immutable()`, `get_immutable()`	`as_imm()`
`ImmutSpan`	`ImmSpan`
`ConditionalType`, `std.utils.type_functions`	`T if cond else U`
`trait_downcast()`	`conforms_to(...)` in `where` or `comptime assert`
`memcmp`	`unsafe_memcmp`

`List.steal_data()`, `OwnedPointer.steal_data()`	`unsafe_take_allocation()`
`OwnedPointer.take()`	`into_inner()`
`Variant.take()`, `Variant.unsafe_take()`	`unwrap()`, `unsafe_unwrap()`
`b64decode(validate=...)`	always validates; drop the parameter
`std.gpu.profiler`, `ProfileBlock`	`perf_counter_ns()`, external profiler
`benchmark.run[func]()`	`run(f)`
`AnyCoroutine`, `Coroutine`, `RaisingCoroutine`	*(unnameable; async def still works)*
async task API in `std.runtime.asyncrt`	`initialize_runtime()`, `parallelism_level()` from `std.runtime`

2. std.objc

The complete exported surface. Throughout, `P` abbreviates `OpaquePointer[MutUntrackedOrigin]`.

```
from std.objc import (
    ObjCClass, ObjCObject, SEL, sel, msg_send, send, autoreleasepool,
    load_framework,
    ObjCRef, ObjCWeakRef,
    NSString, nsstring, extern_object, ns_to_string,
    msg_send_raising, msg_send_raising_check,
    ObjCClassBuilder, IMP0, IMP1, IMP0Bool, IMP1Bool, IMP2,
    new_instance, named_global, sel_dynamic,
)

# GCD lives in its own module
from std.objc.dispatch import (
    Semaphore, async_f, global_queue, main_queue, sync_f, with_block,
)
```

Runtime handles

SEL

A registered Objective-C selector — an interned string pointer.

Member	Signature
`ptr`	`def ptr(self) -> P`

Conforms to `TrivialRegisterPassable`.

ObjCClass

A handle on a class object.

Member	Signature
`lookup`	`@staticmethod def lookup[name: StaticString]() -> ObjCClass`
`is_nil`	`def is_nil(self) -> Bool`
`as_object`	`def as_object(self) -> ObjCObject`

`lookup` calls `objc_getClass`. A nil result means the framework is not loaded.

ObjCObject

A borrowed `id`.

Member	Signature

is_nil	def is_nil(self) -> Bool
addr	def addr(self) -> Int
ptr	def ptr(self) -> P

Construct directly from an address with `ObjCObject(Int(raw_pointer))`.

Selectors

sel

```
def sel[name: StaticString]() -> SEL
```

Registers the selector once and caches the result in a per-selector global slot, deduplicated by name in the KGEN lowering. After the first send the cost is one load and a null check.

sel_dynamic

```
def sel_dynamic(name: StaticString) -> P
```

Registers a selector by name at run time, returning the raw pointer. Accepts custom selectors the SDK does not know. Use when calling `class_addMethod` directly.

Message sending

msg_send

```
def msg_send[
    R: AnyType,
    cls: StaticString,
    selector: StaticString,
    is_class: Bool = False,
    *Ts: AnyType,
](obj: ObjCObject, *args: *Ts) -> R
```

Sends `selector` to `obj`, returning `R`.

Parameter	Meaning
R	Return type; must be register-passable
cls	Class the selector is looked up on; inheritance-resolved
selector	The selector, colons included
is_class	True for a + method
Ts	Argument types, inferred

Compile-time checks: the selector must exist on `cls` or a superclass; the argument count must equal the selector's declared count; each argument must be in the register file the ABI expects, for the cases that can be determined certainly; and the `@encode` signature must be modelable.

`send`

```
def send[R: AnyType, selector: StaticString, *Ts: AnyType](
    obj: ObjCObject, *args: *Ts
) -> R
```

Sends to a **protocol-typed** receiver whose concrete class is unknown at compile time — every Metal object, every delegate. The dispatch stub and argument classes come from the database keyed by selector alone.

Argument count and register-file checks still apply. Receiver-class verification does not.

Ownership

`ObjCRef`

An owning handle on one +1 reference. Conforms to `Movable`; deliberately not implicitly `Copyable`.

Member	Signature	Notes
init	`def __init__(out self, *, adopt: ObjCObject)`	Takes ownership of an object already at +1
init	`def __init__(out self, *, retain: ObjCObject)`	Adds a +1 to a borrowed object
`copy`	`def copy(self) -> Self`	Explicit retain
`object`	`def object(self) -> ObjCObject`	Borrowed handle, valid for this ref's lifetime
`is_nil`	`def is_nil(self) -> Bool`	
`autorelease`	`def autorelease(deinit self) -> ObjCObject`	Hands to the current pool; consumes the ref
deinit	`def __deinit__(deinit self)`	Releases if not nil

Use `adopt=` after `alloc`, `new`, `copy`, `mutableCopy`; `retain=` otherwise.

Backed by the ARC entry points `objc_retain`, `objc_release`, `objc_autorelease`, so it interoperates with ARC code.

`autoreleasepool`

A scoped pool.

```
with autoreleasepool():
```

```
    ...
```

Pushes on `__enter__`, pops on `__exit__`, and guards against a double pop — popping a token twice corrupts the pool page.

Foundation

`NSString`

A leak-safe owning wrapper. Conforms to `Movable`.

Member	Signature
init	`def __init__(out self, *, adopt: ObjCObject)`
init	`def __init__(out self, text: String)`
`object`	`def object(self) -> ObjCObject`
`length`	`def length(self) -> Int`
`to_string`	`def to_string(self) -> String`
`equals`	`def equals(self, other: NSString) -> Bool`
`appending`	`def appending(self, other: NSString) -> NSString`

`length` is UTF-16 code units, matching `-[NSString length]`.

`nsstring`

```
def nsstring(s: String) -> ObjCObject
```

An **autoreleased** `NSString`. For handing to AppKit setters, which retain. Use inside an autorelease pool.

`extern_object`

```
def extern_object[name: StaticString]() -> ObjCObject
```

The object held in an extern Cocoa constant, such as `NSForegroundColorAttributeName`. These are globals whose address the linker resolves, not compile-time values; this takes a link-time reference to the data symbol and loads the pointer out of it.

Defining classes

IMP type aliases

Alias	Signature
IMP0	fn(P, P, /) -> None
IMP1	fn(P, P, P, /) -> None
IMP0Bool	fn(P, P, /) -> Bool
IMP1Bool	fn(P, P, P, /) -> Bool
IMP2	fn(P, P, P, P, /) -> None

The first two arguments are always `self` and `_cmd`.

ObjCClassBuilder

Superseded by the class keyword for new code; retained as the lower-level mechanism and as the escape hatch for method shapes `class` does not cover.

```
struct ObjCClassBuilder[superclass: StaticString = "NSObject"]
```

Member	Signature
init	`def __init__(out self, name: String)`
add_method	`def add_method[selector: StaticString, encoding: StaticString = ""](mut self, imp: IMPn)`
register	`def register(deinit self) -> ObjCClass`

`add_method` is overloaded across the five IMP shapes. When `encoding` is omitted the SDK's `@encode` for the selector is used, with frame offsets stripped; an unknown selector with no `encoding` is a compile error.

`register` consumes the builder, so call it as `builder^.register()`.

new_instance

```
def new_instance(cls: ObjCClass) -> ObjCObject
```

`+[cls new]` — an owned (+1) instance. Retain it explicitly if it must outlive the enclosing scope while Cocoa holds a bare pointer to it.

named_global

```
def named_global[name: StaticString, T: AnyType]() -> Pointer[T, MutUntrackedOrigin]
```

A zero-initialised process global of type `T`, shared by name across every call site, because KGEN deduplicates the global. The standard answer to callbacks having no closure.

```
comptime g_running = named_global["life.running", Int]
```

```
g_running()[] = 1
```

Frameworks

`load_framework`

```
def load_framework[name: StaticString]() -> Bool
```

`dlopen` of `/System/Library/Frameworks/<name>.framework/<name>` with `RTLD_NOW`. Returns whether the handle is non-null. Idempotent and cheap after the first call.

Foundation arrives in every process; AppKit does not unless the binary linked it, which a `mojo run` process did not. Without this, `ObjCClass.lookup` for an AppKit class returns nil and every message to it silently no-ops.

Errors

`msg_send_raising`

```
def msg_send_raising[
    cls: StaticString, selector: StaticString,
    T0: AnyType, ..., is_class: Bool = False,
](obj: ObjCObject, a0: T0, ...) raises -> ObjCObject
```

Sends an object-returning `...error:` selector and raises when the result is nil. **The trailing error argument is created and appended by the wrapper — pass the message arguments without it.**

`msg_send_raising_check`

```
def msg_send_raising_check[
    cls: StaticString, selector: StaticString,
    T0: AnyType, ..., is_class: Bool = False,
](obj: ObjCObject, a0: T0, ...) raises
```

The `BOOL` convention: raises when the result is `NO`. Returns nothing. Same error-slot handling.

Both are declared at explicit arities up to five leading arguments, because nothing may follow an unpacked `*args`. The raised `Error` carries the `NSError`'s `localizedDescription`, domain and code, copied into a Mojo `String`. An API that returns failure without writing an error produces a message saying exactly that.

Weak references

ObjCWeakRef

A zeroing weak reference. Conforms to `Movable`.

Member	Signature	Notes
init	`def __init__(out self, target: ObjCObject)`	Registers via `objc_initWeak`; a nil target is legal
`load`	`def load(self) -> ObjCRef`	The object at +1 if alive, else a nil `ObjCRef`
`copy`	`def copy(self) -> Self`	An independent registration
deinit	`def __deinit__(deinit self)`	`objc_destroyWeak` plus `free`

The registration slot is heap-allocated — one pointer-sized `malloc` per weak reference — because the runtime tracks weak references by the *address of the slot* and Mojo values move. A struct field would leave the runtime pointing into a dead frame after any move.

`load()` goes through `objc_loadWeakRetained` and therefore returns an owned reference, so the object cannot be deallocated between the nil-check and the use.

Use this wherever Cocoa expects weakness — delegates and observer targets — because an `ObjCRef` there is a retain cycle.

Strings, the other direction

ns_to_string

```
def ns_to_string(s: ObjCObject) -> String
```

`NSString` to Mojo `String`, by copy, UTF-8. The result does not depend on the pool owning the `NSString`.

std.objc.dispatch

Grand Central Dispatch. Imported directly, not re-exported from `std.objc`.

Name	Signature	Notes
`DispatchFn`	`fn(_RawPtr) -> None`	The work-function contract
`global_queue`	`def global_queue() -> _RawPtr`	Default-priority concurrent queue
`main_queue`	`def main_queue() -> _RawPtr`	`&_dispatch_main_q`, as the C macro takes it
`sync_f`	`def sync_f(queue, context, work: DispatchFn)`	Runs before returning
`async_f`	`def async_f(queue, context, work: DispatchFn)`	Returns immediately
`Semaphore`	`struct Semaphore(Movable)`	`.wait()`; one wait per expected

		signal
`with_block`	`def with_block[work: fn() -> None](queue, *, wait: Bool)`	Builds a real ObjC block around an `fn`

`with_block` works because a thin `fn` is exactly a global block — `_NSConcreteGlobalBlock`, no captures, so no copy/dispose helpers. The `wait=True` path uses a stack literal; `wait=False` lets the runtime `Block_copy` the 32 bytes off the frame.

Capturing closures as heap blocks are not implemented.

Method type encodings

The strings `class_addMethod` expects, as returned by the database with frame offsets stripped.

Encoding	Meaning
`v@:`	`void method(id self, SEL _cmd)`
`v@:@`	`void method(id self, SEL _cmd, id arg)`
`c@:`	`BOOL method(id self, SEL _cmd)`
`c@:@`	`BOOL method(id self, SEL _cmd, id arg)`
`@@:`	`id method(id self, SEL _cmd)`

Single-character type codes: `v` void, `c` `BOOL` (a signed char), `@` object, `:` selector, `i` int, `q` long long, `Q` unsigned long long, `d` double, `f` float, `*` C string, `^` pointer to.

`BOOL` is `c`, not `B`. This is the encoding most often written wrongly by hand.

3. cocoakb queries

Compile-time queries against `cocoa.sqlite`. Every one resolves to a constant before code generation, so a name the metadata does not know is a compile error rather than a wrong answer.

```
from std.sys._cocoakb import cocoakb_struct_size, cocoakb_field_offset
```

The module is underscore-prefixed because it is compiler-adjacent, not because it is off limits — the verification spikes import it directly.

Layout

`cocoakb_struct_size`

```
def cocoakb_struct_size[name: StaticString]() -> Int
```

Size in bytes of a Cocoa struct. `name` is spelled as the runtime spells it, for example `"CGRect"`.

`cocoakb_struct_align`

```
def cocoakb_struct_align[name: StaticString]() -> Int
```

Alignment in bytes.

`cocoakb_field_offset`

```
def cocoakb_field_offset[type_name: StaticString, field: StaticString]() -> Int
```

Byte offset of a field within a struct. This is what makes a declaration checkable rather than merely plausible.

```
comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()
comptime assert cocoakb_field_offset["CGRect", "size"]() == 16
```

Constants

`cocoakb_enum_value`

```
def cocoakb_enum_value[name: StaticString]() -> Int
```

The value of an enum member, from BridgeSupport. Signed, so a value that must sign-extend as a pointer does behaves correctly, and a flag mask narrowed by the caller keeps its bits either way.

```
comptime assert cocoakb_enum_value["NSUTF8StringEncoding"]() == 4
```

cocoakb_constant_type

```
def cocoakb_constant_type[name: StaticString]() -> StaticString
```

The declared type encoding of an extern-symbol constant, for example `"@"` for an object. Constants like `NSFontAttributeName` are runtime addresses, not compile-time values; this reports the type, and `extern_object` fetches the value.

Classes and methods

cocoakb_superclass

```
def cocoakb_superclass[name: StaticString]() -> StaticString
```

cocoakb_method_encoding

```
def cocoakb_method_encoding[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

The verbatim `@encode` signature, inheritance-resolved: the lookup walks the superclass chain, so asking `NSMutableString` about `length` finds `NSString`'s definition. Returns the raw form including frame offsets, for example `"Q16@0:8"`.

cocoakb_msgsend_variant

```
def cocoakb_msgsend_variant[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

Which `objc_msgSend` entry point the send must use. Returns `"objc_msgSend"`, `"objc_msgSend_stret"`, or `"objc_msgSend_fpret"`; `"?"` marks a signature the ABI classifier could not model, and callers assert against it.

On arm64 the answer is always `"objc_msgSend"`. AAPCS64 has neither `_stret` nor `_fpret` — an aggregate return travels in `x0`–`x1`, in `v0`–`v3` when it is a homogeneous float aggregate, or through the `x8` indirect-result register. The query is retained because its other job — rejecting unmodelable signatures — still matters.

cocoakb_method_ret_class

```
def cocoakb_method_ret_class[
```

```
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

The AAPCS64 return classification, in the token vocabulary below.

`cocoakb_method_arg_classes`

```
def cocoakb_method_arg_classes[
    cls: StaticString, selector: StaticString, is_class: Bool = False
]() -> StaticString
```

Comma-joined classifications of the arguments beyond `self` and `_cmd`. An empty string means the selector takes none; `"g"` means one; `"g,g"` two.

Selector-keyed queries

For protocol-typed receivers whose concrete class is unknown at compile time.

`cocoakb_selector_variant`

```
def cocoakb_selector_variant[selector: StaticString]() -> StaticString
```

The dispatch variant, taken from any class implementing the selector. An empty result or `"?"` means no class in the metadata implements it.

`cocoakb_selector_arg_classes`

```
def cocoakb_selector_arg_classes[selector: StaticString]() -> StaticString
```

`cocoakb_selector_encoding`

```
def cocoakb_selector_encoding[selector: StaticString]() -> StaticString
```

The `@encode` signature by majority across implementing classes, for example `"v24@0:8@16"`. Used by `ObjCClassBuilder` to type a Mojo method when defining a class at run time.

POSIX

The same database describes the C library.

Query	Returns
`cocoakb_posix_sig[name]()`	The full C type as clang reports it, e.g. `"int (const char *, int, ...)"`
`cocoakb_posix_ret_class[name]()`	AAPCS64 return classification

cocoakb_posix_arg_classes[name]()	AAPCS64 argument classifications

Provenance

cocoakb_db_hash

```
def cocoakb_db_hash() -> StaticString
```

The SHA-256, lowercase hex, of the database this compilation consulted. A binary can record exactly which metadata revision produced it, which is the reproducibility pin and also how you notice a stale database after a macOS update.

The AAPCS64 token vocabulary

These tokens come from `derive_method_abi.py` in CocoaBaseMCP. They are **not** the SysV eightbyte vocabulary, and reading them as if they were puts a homogeneous float aggregate in the wrong register file.

Token	Meaning	Register file
`v`	void	—
`g`	general-purpose register	integer
`f`	scalar float	float, `v0–v7`
`h2`, `h3`, `h4`	homogeneous float aggregate of k members	float, k consecutive v registers
`i1`, `i2`	small integer struct	integer
`b`	by-value pointer	integer
`x`	indirect result via `x8`	—
`s`	struct	integer
`m`	memory	—
`?`	unmodelable; a compile error	—

A value goes in a `v` register when it is a scalar float or a homogeneous float aggregate. Everything else goes in the integer file. Testing whether every byte of a token is `f` would be the SysV reading and would misclassify an HFA.

Worked examples from the verification spike:

Query	Result
`cocoakb_method_ret_class["NSString", "length"]()`	`"g"` — one GPR
`cocoakb_method_ret_class["NSValue", "rectValue"]()`	`"h4"` — `CGRect` in `v0–v3`

cocoakb_method_encoding["NSMutableString", "length"]()	"Q16@0:8"

4. Diagnostics

The compile errors this layer produces, what each one means, and what to do. Messages are quoted from the source that emits them.

Language-level

`'fn' declares a foreign-callable (C ABI, non-raising) function in cocoa-mojo and may not be marked 'raises'; use 'def' for an ordinary Mojo function`

`fn` is the foreign-callable function. The C boundary has no error channel, so a raising `fn` cannot exist. Comes with a FixIt replacing `fn` with `def`.

This is also the migration message: code from the era when `fn` was a general strict function (`fn main() raises`) gets told what changed rather than a bare contract violation.

`'fn' declares a foreign-callable function and cannot be 'async'; use 'def'`

Same contract, same remedy.

`unknown tokens at the end of a declaration`, on a class field

A field initializer. `var x: Int = 3` is not accepted in this version; declare `var x: Int` and let it take its default. See [chapter 6](#).

`expression must be mutable for in-place operator destination`, in a class method

The method needs `mut self` to write a field, exactly as a struct method would.

`'let' declares an immutable binding inside a function body; use 'var' for a field or module value`

`let` has no field or file-scope form. Use `var` there.

Reassigning a `let`

Rebinding a `let` is a compile error. The object remains mutable — this is about the binding, not the value. See [the language chapter](#).

`'alias' is deprecated; use 'comptime'`

A warning, not an error. `alias` still parses.

`'__new__' is not supported on structs; use '__init__' instead`

`the 'register_passable' function effect is no longer supported`

`the 'escaping' function effect is no longer supported`

`implicit deletion. '@explicit_destroy' is no longer required.`

Remove the decorator; implicit deletion is now the default.

`std.objc` compile-time assertions

These come from `comptime assert` inside `msg_send`, `send` and `ObjCClassBuilder`, so they fire during elaboration and name the problem precisely.

Unknown or unmodelable signature

```
std.objc: 'SELECTOR' on CLASS has an @encode signature the ABI classifier
could not model, so std.objc cannot pick a dispatch stub for it. Call it by
hand with a checked external_call if you know the layout.
```

`cocoakb_msgsend_variant` answered `"?"`. Rare, and the escape hatch is the one the message names: call it yourself with `external_call`, having worked out the layout.

Wrong argument count

```
std.objc: 'SELECTOR' on CLASS takes N argument(s), but M were passed.
```

Count the colons in the selector. `stringWithUTF8String:` takes one; `initWithContentRect:styleMask:backing:defer:` takes four. This is the check that prevents reading an unset argument register.

Wrong register file — float where an integer is expected

```
std.objc: argument I of 'SELECTOR' on CLASS is a float, but the ABI expects an
integer/pointer register here. Check the argument type.
```

Wrong register file — integer where a float is expected

```
std.objc: argument I of 'SELECTOR' on CLASS is an integer, but the ABI expects
a float register here. Pass a Float32/Float64.
```

The usual cause is an integer literal where a `CGFloat` is wanted. Write `Float64(0)` rather than `0`. These checks fire only where the classification is certain, so an absence of error is not proof of correctness for struct arguments.

Selector implemented by nothing

```
std.objc: no class in the metadata implements selector 'SELECTOR', so its
dispatch ABI is unknown. Check the selector spelling.
```

From `send`. Either the selector is misspelled, or the framework declaring it was not scanned when the database was built.

Unknown selector encoding when defining a class

Raised by `ObjCClassBuilder.add_method` when `encoding` is omitted and the SDK does not know the selector. Supply it:

```
b.add_method["myCustomAction:", encoding="v@:@"](handler)
```

cocoakb query failures

A name the database does not contain fails the query itself. The `must_fail.mojo` spike is exactly this:

```
comptime bogus = cocoakb_struct_size["NSDefinitelyNotAStruct"]()
```

The build fails. It does not return zero, and it does not return a plausible size.

If a name you believe is real fails, work through these in order.

Is the database built? On an installed toolchain the installer built it; if it is missing or was interrupted, run the installer again and press *Reinstall*. From a source tree: `python3 ../CocoaBaseMCP/build.py`.

Is the compiler pointed at it? An installed toolchain carries its own copy and needs no variable — `bin/cocoamojo` exports it before every build. From a source tree, `MODULAR_MOJO_MAX_COCOAKB_PATH` must name an existing `cocoa.sqlite`.

Is it stale? Rebuild after a macOS update — *Reinstall*, or `build.py` again. `cocoakb_db_hash()` tells you which revision a compilation used.

Is the spelling the runtime's? The database is built from the live Objective-C runtime and BridgeSupport, so it holds the runtime's names, not a header's typedef.

Runtime failures the compiler cannot catch

Three things remain yours.

A nil class. `ObjCClass.lookup` returns nil when the framework is not loaded. Every subsequent message to it does nothing, silently. Check `is_nil()` once at startup.

Use after the pool drains. An object handed to a pool by `autorelease()` is valid only until that pool exits. Nothing checks this.

An object released while Cocoa still holds it. Typically an `ObjCRef` that went out of scope, or a `new_instance` result never retained, where Cocoa keeps a bare pointer. The symptom is a crash inside `objc_msgSend` with a plausible-looking receiver.

Symptoms with no error at all

Symptom	Cause
Window appears but ignores the keyboard	`acceptsFirstResponder` missing or returning `False`
Window cannot be focused, no Dock icon	`setActivationPolicy:` not called with `0`
Callback never fires	Selector added under a different spelling, or `register()` never ran
`respondsToSelector:` returns `False`	Same as above; check the selector string character by character
Allocator corruption far from any Cocoa call	A `List` pointer stashed after its last use; allocate outside Mojo instead
`autorelease pool page corrupted`	A pool token popped twice
A class field written from Mojo is not visible to Cocoa	`ClassName()` returns a copy of the box plus the `id`; mutating through it writes the copy. Write fields from methods the runtime dispatches to
A class field owning memory never frees	`dealloc` is not hooked yet, so a field's `deinit` does not run
Windowed app starts and exits with no window and no message	`load_framework["AppKit"]()` was not called; every message to the nil class silently no-ops
A crash when assigning into a `var` declared but not initialised	Pre-existing: assigning into an uninitialised `var` of a type with `__deinit__` destroys garbage first. Bind in a helper scope and return instead

5. Deviations from Modular's compiler

This fork is not a patched Mojo that happens to build on a Mac. It changes what the language *is* in specific, deliberate ways, and this document states every one of them and argues for it.

Each entry says what upstream does, what this compiler does instead, **why that is the right answer for this target**, and what the choice costs. The cost column is not decoration — a deviation with no cost is usually one nobody thought about.

> **Read the last section first if you are debugging.** A good deal of what looks like a deviation is upstream's own churn, inherited by freezing at a commit. Those are listed at the end under Not ours, and blaming the fork for them will send you looking in the wrong place.

The shape of the divergence

Deviation	Kind
fn is the foreign-callable function	Revived keyword, narrower contract
let binds by reference	Revived keyword
class declares an Objective-C class	Reserved word given a meaning
@objc	New decorator
cocoakb: the compiler asks the SDK	New compiler capability
An Apple GPU target	New backend and runtime
std.objc	New standard-library package
One host, one CPU, one accelerator	Narrowed scope

Three of these are keywords, and they are the ones that will bite a reader coming from current Mojo documentation, because the code still *parses*.

fn is the foreign-callable function

Upstream used `fn` for the strict-mode function — declared argument mutability, no implicit raising — and has since folded that distinction away.

Here `fn` means something narrower and load-bearing: thin (no captures), non-raising, C ABI, every parameter and return type ABI-classifiable. That is exactly the Objective-C `IMP` contract, and exactly a C function pointer.

```
comptime AURenderCallback = fn(P, P, P, UInt32, UInt32, P, /) -> Int32
```

Why this is right. A language that talks to an operating system needs a way to say *this function is callable by C*, and it needs the compiler to enforce it rather than hope. Every Cocoa callback, every `IMP`, and every C function pointer the platform hands you has the same contract, so the contract deserves a keyword rather than a decorator and a convention.

Making it a declaration makes it checkable. A `fn` marked `raises` is a compile error — one of the four must-fail checks in the verification suite — instead of an exception unwinding into CoreAudio's render thread, where there is no handler and no way to report one. `examples/chip/` installs a Mojo `fn` as CoreAudio's render callback with no C shim anywhere, which is the whole argument in one file.

What it costs. `fn` cannot call a `def`, because a `def` may raise. That reaches further than it first appears: `std.math.sin` is a `def`, so the chip synthesiser carries a hand-written sine series. The restriction is real and it is the price of the guarantee.

`let` binds by reference

Upstream removed `let` entirely.

Here `let` is an immutable, function-scope binding that **names storage rather than copying it**. `let x = g[]` does not snapshot `g`; it is another way to say the same location.

Why this is right. Cocoa code is mostly borrowed references. A `let` that copied would either retain and release an object nobody wanted duplicated, or silently take a snapshot of something the runtime is still mutating. Naming the storage is the honest description of what a borrowed reference *is*, and it makes the common case free — no copy, no lifetime question.

Keeping ownership out of the keyword is deliberate. `let` says nothing about retain and release; `ObjCRef` does. One concept per construct means you can read a line and know which question it answers.

What it costs. This is the sharpest edge in the language, and it is not hypothetical — it has cost real debugging time five separate times in this project. A value read into a `let` and then mutated at the source reads back *mutated*:

```
for a in range(1, len(times)):
    let t = times[a]        # names the slot; the shift below writes over it
    var b = a - 1
    while b >= 0 and times[b] > t:
        times[b + 1] = times[b]
```

That sorts `5 3 9 1` into `5 5 9 9`. `var t = times[a]` is correct and is the only difference. There is no diagnostic, because nothing is wrong — the binding is doing exactly what it is for.

Read the value **out** before touching the source, or bind a genuine copy with `var`. The compiler does catch the related case where the container is *grown* rather than written — that is `use of invalidated interior reference`, and it names both the read and the invalidation.

`class` declares an Objective-C class

Upstream reserved `class` for a Python-style class that never arrived; it was lexed, classified, and never parsed.

Here it declares a real Objective-C class, registered in the runtime.

```
class ExampleActions:
    def buttonClicked_(self, sender: ObjCObject):
```

```
    ...
```

The selector comes from the method name — an underscore is a colon, so `buttonClicked_` answers `buttonClicked:`. The type encoding is derived: from the SDK when the selector is one Cocoa declares, and from the signature when it is one you invented. Instantiating the class registers it.

Why this is right. Cocoa is a callback architecture. You cannot put a window on screen without being called back — by a button, a timer, a delegate, an Apple Event. Every other language reaching Cocoa expresses this with generated glue: a bridging header, a class-builder API, hand-written `v@:@` strings, an `IMP` and a `_cmd` slot.

All of that is bookkeeping the compiler already has the information to do. It knows the method names, it knows the signatures, and through `cocoakb` it knows what the SDK declares. Making the class a *declaration* rather than a sequence of library calls is what removes the bridging header — not as a convenience, but because a declaration can be checked and a sequence of calls cannot.

The check is real in a way worth stating: an invented selector that collides with one the SDK already declares is caught. `life/` names its timer callback `lifeTick:` rather than `tick:` because `CASecureFlipBookLayer` declares `tick:` taking a double, and the compiler said so — a collision that would otherwise have registered the method with the wrong encoding.

What it costs. Fields have no initializers yet, and the class model took the largest single share of the compiler work in this fork.

`@objc`

A decorator, on methods and on classes, that makes the Objective-C exposure explicit where the default is not what you want. New here; upstream has no equivalent because it has nothing to expose to.

cocoakb: the compiler asks the SDK

This is the deviation the rest of the fork is built on, and it is not a language change at all — it is a change to what the compiler *does*.

Upstream compiles Mojo. **Here** the compiler additionally holds an open connection to a database describing macOS, and resolves questions against it during elaboration, at the point of use:

```
comptime assert size_of[CGRect]() == cocoakb_struct_size["CGRect"]()
```

Fifteen-odd builtins answer struct sizes and field offsets, enum values, superclasses, method encodings, which `objc_msgSend` variant a return type requires, and the hash of the database a compilation consulted.

Why this is right. The alternative is code generation, and generated bindings drift. A tool reads the SDK once, emits a large body of source, and from that moment the emitted code and the real SDK diverge silently — usually discovered as a crash on a machine with a different macOS.

A compile-time query cannot drift, because there is no intermediate artifact to go stale. The answer is produced when your program is compiled, from the SDK on the machine compiling it. That is why a selector typo is a compile error, why a struct that has changed shape fails to build rather than corrupting a call, and why an argument in the wrong register file is rejected instead of producing garbage.

It also means the database is built from *your* Mac rather than downloaded — roughly 236 MB describing 28,814 classes and 522,170 methods, read out of the live Objective-C runtime and BridgeSupport.

What it costs. The compiler has a build input it must be pointed at. Get it wrong and every Cocoa name in the program fails to resolve, which looks like a source error and is a configuration one. `cocoakb_db_hash()` exists so a binary can record which revision it was built against.

An Apple GPU target

Upstream targets NVIDIA and AMD. **Here** there is a complete Metal/AIR backend — roughly 4,750 lines of lowering, legality checking and object emission — and an Apple GPU runtime of about 2,750 lines beneath it.

Why this is right. Mojo's premise is that one source file specialises to whatever silicon you point it at. On a Mac the silicon is the Apple GPU, and without this backend the premise is not merely unimplemented — it is untestable on the hardware most people actually own. A kernel here is ordinary Mojo: no shading language, no source compiled at run time, no second toolchain.

The claim is checkable rather than asserted. `examples/tiled-matmul` and the rest of the carried GPU examples are Modular's own, running **unmodified** — `DeviceContext`, `TileTensor`, `enqueue_function`, `barrier()` and `AddressSpace.SHARED` all survive the new backend as written.

What it costs. LLVM is target-agnostic by construction, so it cannot see AIR's rules. Every defect this backend has shipped was legal LLVM IR that was illegal for the target, discovered one crash at a time from an error naming nothing. That is why the backend carries a data-driven legality firewall with per-rule permit/log/fail and recorded evidence for each rule — see Patches.

`std.objc`

About 9,800 lines of standard library that upstream has no reason to carry: class lookup, message dispatch in its typed and dynamic forms, autorelease pools, ownership through `ObjCRef`, selectors, framework loading, named globals, and the typed calling layer.

Why this is right. The compiler deviations above give you a *checked* way to reach Cocoa; this package is the surface you actually write against. Keeping it in the standard library rather than a side package means the compiler and the library are versioned together, which matters when half the guarantees are enforced at compile time.

One host, one CPU, one accelerator

Upstream supports Linux on x86-64 and aarch64, and macOS on ARM64.

Here the target is Apple Silicon macOS, and the 2019 Intel Mac Pro with its Radeon Pro Vega II is a *separate fork*, described the same way and shipped as its own image.

Why this is right. The alternative to narrowing is a matrix nobody can test. Each port here targets one host, one CPU and one accelerator, and each is described with the same three statements: what runs, what does not, and what has merely been built rather than tested. A port that claims five platforms and has measured one is a port that will be wrong about four of them.

What it costs. Shared lowering changes have to be made in each fork, and the NVIDIA, AMD and Snapdragon paths still in this tree are reference material rather than supported targets.

Not ours: upstream churn we inherited

Freezing at a commit inherits whatever upstream had already changed by then. The following differ from a good deal of published Mojo writing, **including Modular's own documentation**, and none of them is a decision of this fork:

- `alias` is deprecated in favour of **comptime**
- `@parameter if` and `@parameter for` have given way to **comptime if** and **comptime for**
- `InlineArray` became `Array`; `.size` became `.length`
- `trait_downcast()` gave way to `conforms_to(...)`
- `memcmp` became `unsafe_memcmp`; `List.steal_data()` became `unsafe_take_allocation()`
- the async task API moved out of `std.runtime.asyncrt`

The full list is in the [Language reference].

Also not a compiler deviation, but worth stating once: the contributor process files — `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, the issue and PR templates, `CODEOWNERS`, the CLA workflow — were **removed** rather than left pointing at Modular's trackers and review teams. This is an unaffiliated fork; nothing here goes upstream, and an issue filed against Modular for a defect in this AIR backend wastes someone's afternoon. `LICENSE` and `Licenses/` stay, because the tree is overwhelmingly Modular's Apache-2.0 code and a derivative work has to ship the licence.

6. Patches applied to the compiler

Deviations says what this fork changed about the language and why. This document covers the work inside the compiler that made those changes real, and the defects fixed along the way.

Ninety-three commits touch the compiler, the backend, the GPU runtime or the standard library, between 23 and 31 August 2026. They divide as:

Area	Commits	What it is
`[Air]` / `[AIR]`	36	The Metal/AIR backend: lowering, legality, object emission
`[KGEN]`	30	The frontend: the class model, `cocoakb`, and frontend defects
`[AppleGPU]`	10	Kernel launch, argument binding, capability reporting
`[AsyncRT]`	7	The Metal runtime: residency, batching, async launch
`[stdlib]`	7	Standard-library fixes, mostly around `class` and `String`
`[Kernels]`, `[Support]`	3	Example corpus repair, debug annotation

The rest of this document explains the ones worth understanding. To list them all: `git log --oneline --grep='^\[\(KGEN\|Air\|AppleGPU\|AsyncRT\|stdlib\)\]'`.

Building the class model

Forty-one commits mention the class model in some form, which makes it the largest single body of work in the fork. It landed as numbered sprints and each one is a self-contained claim:

Sprint	What it established
1	The parser accepts `class`
2	Classes resolve to types; each method's selector is derived
2b	Protocols, registration, the method walk, trampolines
3	Fields live in the box, proven at runtime, and emptied when the object dies
4	The SDK answers, and the parser can ask it

Two of these are worth drawing out.

Trampolines and the ABI oracle. A method Cocoa calls arrives through the Objective-C dispatch machinery, which means the compiler must emit a function with exactly the shape the runtime expects — receiver, `_cmd`, then arguments, in the right register file. Getting that wrong does not produce a diagnostic; it produces a corrupted call. The approach taken was to make **clang the ABI oracle**: rather than reason about classification rules, ask the toolchain that already implements them, and make a refusal speak (*"Trampoline refusals speak; clang becomes the ABI oracle"*).

A class travels in a register, and still retains. A class value has to be passable like any other Mojo value while remaining a reference-counted Objective-C object. Getting both at once — register-passable *and* correctly retained — is the commit that makes `class` usable as an ordinary type rather than a special case you must handle carefully.

Frontend defects fixed while building this:

- **@staticmethod crashed the compiler.** It is now a `+` method, which is what it always meant in Objective-C terms.
- **Fields were constructed into the local, not the box** — so the object the runtime held and the object your code wrote to were different memory.
- **A returned class could not parameterise a type**, and the fix required the compiler to query the *returned* class, which turned out to be knowable at compile time.
- **cocoakb_query folded too late**, so the SDK could not choose a type. Folding it early is what lets a database answer participate in elaboration rather than merely be checked afterwards.
- **box_ref: nil is a state, not a hazard** — reading a field of a nil class reference should be a defined outcome, not a trap.

The AIR backend

This is the largest area by commit count and the one with the most instructive failures. The theme running through all of it:

> LLVM's verifier is target-agnostic by construction, so it cannot see any of AIR's rules. Every defect this backend shipped was **legal LLVM IR that was illegal for the target**, and each was found one crash at a time, days apart, from an error naming nothing.

That sentence is why the backend has a **data-driven legality firewall** rather than a pile of `if`s: a table of rules, each carrying its own action (permit / log / fail) and its own evidence for why it exists. A rule added as `fail` on day one is a rule that stops the build on its first false positive, and a gate that cries wolf gets switched off — so rules start at `log` and are promoted once a full sweep is clean.

Representative fixes, each a distinct failure mode:

Address spaces. AIR has no generic address space, so a kernel left with its stores in `addrspace(1)` and its loads in `addrspace(0)` *writes correctly and reads zero* — an output buffer full of zeroes, no diagnostic, and nothing any verifier objects to. Hence: deviceize **every** captured pointer rather than the one the frontend unpacked first; follow pointers whose descriptor arrives through a `select` or `phi`; reach captured pointers with `inttoptr` rather than `addrspacecast`; re-legalise after inlining, because whatever the inliner just pulled in from a callee has never been through it; and drop no-op `addrspacecast`s, which make `metallib` reject the whole module.

Barriers. A workgroup barrier is one dynamic rendezvous for the whole threadgroup, so selecting or skipping it per thread can silently expose partially written threadgroup memory. Divergent barriers are now rejected, and barriers are marked convergent **at declaration creation, before the optimiser runs** — marking them later is too late, because the optimiser has already moved them. A kernel that reaches a barrier no longer claims `nosync`.

Missing instructions. AIR has no three-way compare, and InstCombine synthesises `llvm.scmp` out of ordinary comparison code — so a kernel acquires one without the author writing anything unusual, and it surfaces at the reader as `Undefined symbols: llvm.scmp.i32.i32`. These are now expanded into selects during legalisation, with a rule that fails closed on any that survive. The same shape recurs:

`llvm.vector.reduce.*` and vector interleave were lowered, int↔float casts routed through `air.convert`, and `llvm.stepvector` folded.

Symbols and types. AIR declarations need an exact type key, and the symbol stems must match what Apple's reader expects — including refusing to guess integer signedness rather than picking one. The `simdgroup_matrix` family needed mangling, and per-signature tags had to be stripped from symbol names.

Three defects that only surface when the metallib becomes a pipeline. The most expensive class: the module verifies, `metallib` accepts it, `air-opt` accepts it, and the failure appears only when the driver builds a compute pipeline — which is why the backend now retains artifacts per kernel and keeps diagnostics attached to their cause.

The Apple GPU runtime

Precise residency, and a selector that was never active. Metal needs to be told which allocations a dispatch may touch. The original approach declared every live root buffer to every compute encoder — one `useResource` per buffer per dispatch, under the registry lock. Correct, and linear in live allocations. An `MTLResidencySet` inverts that: one set per device, attached to each command queue, edited when an allocation is created or destroyed, with nothing left on the dispatch path.

The instructive part is the follow-up. **The selector was wrong, so the optimisation had never executed.** The code asked for `makeResidencySetWithDescriptor:error:` where `MTLDevice` exposes `newResidencySetWithDescriptor:error:`. `respondsToSelector:` answered no, the set was never created, and every launch fell back to the walk.

The results stayed correct throughout, which is exactly what hid it: *the fallback is the old code, and the old code works*. Every measurement of "precise residency" before that fix, in both repositories, was the walk timed against itself, and the ~25% improvement recorded for it was noise between two runs of identical code.

Measured properly afterwards, in µs per dispatch:

Live buffers	Walk	Residency set
256	18.56	3.46
1,024	73.35	3.25
4,096	289.53	3.37

Linear against flat, and −99% at the top end.

Launch and batching. Metal launches now default to queued execution, asynchronous launch is safe to leave on, and queued dispatches are batched into command buffers. The measured effect is clearest on `examples/fluid`, whose step is about 35 dependent dispatches: launch overhead, not arithmetic, is what such a step is made of.

Binding. Captured device pointers are bound as buffers rather than bytes, and bound **from the kernel's argument contract rather than from the value** — the value does not always know what the kernel expects. Reflection is authoritative for compiler-generated kernels, copies are routed by host/device rather than by `MTLBuffer` handle, and oversized threadgroup storage is rejected up front rather than failing later.

Capability reporting. The ABI capability table is generated from the source and checked, so what the runtime claims and what it implements cannot drift apart. `MULTIPROCESSOR_COUNT` is answered from measured cores.

Standard library

Two `String` fixes worth naming because both are the kind that survive review:

- **String._realloc_mutable asked the allocator for zero.** A zero-size allocation is permitted to return null or a pointer you must not use, and either answer is a fault later, somewhere else.
- **String doubled when copying rather than when growing**, which is the wrong side of the operation and turns append-heavy code quadratic.

The rest are the class model reaching the library: `std.objc.typed` for calling Cocoa as calls, `box_ref` reaching a class's fields from outside a method, and the runtime half of class registration, made provable on its own.

What this history is good for

Two things, beyond the record.

First, the failure modes repeat. Address spaces, convergence, and optimiser-synthesised intrinsics account for most of the AIR defects, and each was found the same way — a program that ran and produced the wrong answer, rather than one that failed. If you are extending the backend, those three are where to look first.

Second, several entries above are corrections of *earlier entries*. The residency selector invalidated its own measurements; a legality rule was recorded as counter-evidence against a theory that turned out to be wrong (*"unmapped vector intrinsics are not the defect"*); a status note was corrected for overstating what a gate proved. That is the shape of honest work on a backend reconstructed black-box, and the commit history is kept in that shape on purpose.